

# Yamba — Cahier de recette n° 3 : l'API

Yamba — documentation générée le 06/09/2026 depuis `docs/recette/RECETTE-03-API.md`

## Table des matières

1. Objet et périmètre
  - 1.1 Ce que ce cahier prouve
  - 1.2 Ce que ce cahier ne couvre pas
  - 1.3 Les quatre cahiers
  - 1.4 Documents de référence
2. Prérequis et outillage
  - 2.1 Ce qu'il faut sur le poste
  - 2.2 Démarrer la plateforme
  - 2.3 Rejouer le jeu d'essai
  - 2.4 Les comptes du jeu d'essai
  - 2.5 Les deals du jeu d'essai, par statut
  - 2.6 Préparer le terminal
  - 2.7 Tenir la session entre deux appels
  - 2.8 Une seule porte d'entrée
  - 2.9 Forcer le fournisseur de paiement factice
3. Conventions du cahier
  - 3.1 Numérotation
  - 3.2 Verdicts
  - 3.3 Gravité
  - 3.4 La forme d'une fiche
  - 3.5 Lire une réponse d'erreur
  - 3.6 Le contrat details.code
  - 3.7 Notation des montants, dates et identifiants
  - 3.8 Hygiène de campagne
4. Scénarios transverses
  - 4.1 Santé et sonde publique
  - 4.2 Mode maintenance
  - 4.3 Limiteur de débit
  - 4.4 Identifiant de corrélation
  - 4.5 Langue et en-tête x-locale
  - 4.6 Format des erreurs et contrat details.code
  - 4.7 Sémantique 401 / 403 / 404
  - 4.8 Pagination par curseur
5. Un chapitre par service
  - 5.1 auth-service — comptes, sessions, profil, RGPD
  - 5.2 trip-service — recherche, trajets, cycle de vie, téléversement
  - 5.3 deal-service — le cœur transactionnel
  - 5.4 message-service — fil, rendez-vous, numéro, signalement
  - 5.5 notification-service — boîte aux lettres et webhook email
6. Sécurité
  - 6.1 Accès sans jeton, jeton expiré, jeton d'autrui
  - 6.2 La fenêtre sensible
  - 6.3 Les secrets qui ne doivent jamais sortir
  - 6.4 Signatures de webhooks
  - 6.5 Non-divulgateion
7. Idempotence et concurrence
  - 7.1 Rejeu séquentiel
  - 7.2 Concurrence réelle
8. Webhooks
  - 8.1 Stripe — un appel serveur à serveur, hors passerelle
  - 8.2 Email (Resend) — via la passerelle, signature Svix
9. Consignation
  - 9.1 Tableau de suivi

9.2 Fiche d'anomalie

9.3 Critères de sortie

**Ce cahier se déroule sans navigateur.** Un testeur muni d'un terminal, de `curl` et de `jq` (ou d'un client REST : Bruno, Insomnia, Postman, `httpie`) doit pouvoir le jouer de bout en bout et signer, ligne à ligne, que **le serveur tient ses promesses tout seul** : les bons codes HTTP, la bonne sémantique d'erreur, les bonnes autorisations, l'idempotence annoncée et les effets de bord attendus.

Version du cahier : 1.0 — branche `docs/recette`. Il décrit le comportement de l'API tel que le code de la branche courante l'implémente ; toute divergence constatée est **soit un défaut du code, soit un défaut de ce cahier**, jamais une interprétation du testeur.

## 1. Objet et périmètre

### 1.1 Ce que ce cahier prouve

Les trois autres cahiers de recette regardent la plateforme **par l'écran**. Celui-ci la regarde **par le fil** — la requête HTTP nue, sans le confort et sans le filtre de l'interface. C'est une différence de nature, pas de degré :

- une interface qui masque un bouton **n'a rien prouvé** : la règle peut n'exister que dans le JavaScript du navigateur, et n'importe qui sait rejouer un appel avec `curl` ;
- une interface qui affiche un message clair **n'a rien prouvé** non plus : le message peut être écrit côté client à partir d'un code HTTP générique.

Ce cahier fait donc systématiquement le geste que l'écran interdit, et il exige du serveur qu'il refuse tout seul, avec le bon statut et le bon code. Il vérifie en particulier :

| Famille          | Ce qu'on prouve                                                                                                                                                                                                                    |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Autorisation     | Une route protégée sans jeton répond 401. Le jeton d'un membre qui n'est pas partie au deal répond 403. Une session membre n'ouvre aucune route <code>/admin/*</code> . Un profil admin sans la permission reçoit 403, jamais 404. |
| Sémantique       | 401 ≠ 403 ≠ 404 ≠ 409. Un 404 ne révèle jamais qu'une ressource existe pour quelqu'un d'autre. Un 409 porte un code métier stable.                                                                                                 |
| Contrat d'erreur | Le champ <code>details.code</code> arrive bien au client (A146) : c'est lui que le front traduit, jamais le <code>message</code> anglais.                                                                                          |
| Gardes serveur   | Les limites métier (poids, capacité, fenêtres, plafonds, verrous) sont appliquées par le serveur, pas seulement reflétées par l'écran.                                                                                             |
| Secrets          | Le code de livraison n'apparaît dans aucune surface interdite. Le destinataire n'est jamais exposé au Voyageur ni à la page publique.                                                                                              |
| Idempotence      | Rejouer une transition, une acceptation, une remise, un marquage lu, un favori, un webhook ne produit ni doublon ni effet destructeur.                                                                                             |
| Concurrence      | Deux écritures simultanées : une gagne, l'autre reçoit un conflit typé. Jamais deux gagnantes.                                                                                                                                     |
| Exploitation     | Santé, sonde publique, maintenance, limiteur de débit, identifiant de corrélation : les surfaces que le moniteur et le support utilisent.                                                                                          |

### 1.2 Ce que ce cahier ne couvre pas

- **L'ergonomie, les libellés, la traduction affichée** : cahier n° 1 (parcours membre) et n° 2 (back-office). L'API répond en anglais et par des codes ; c'est normal, c'est le contrat.
- **Le rendu des emails** : ce cahier vérifie qu'un email *part* (trace `EmailDelivery`, boîte Mailpit) et qu'un webhook de retour est traité, pas la mise en page du message.

- **Les écrans d'administration** : le cahier n° 2 les déroule. Ici, seules les routes admin utilisées comme **preuve de garde** (permission refusée, journal écrit) sont jouées.
- **La performance et la charge** : hors périmètre ; seul le limiteur de débit est vérifié, et seulement dans sa fonction de garde.

## 1.3 Les quatre cahiers

| Cahier                           | Fichier                                     | Point de vue                                                        |
|----------------------------------|---------------------------------------------|---------------------------------------------------------------------|
| n° 1 — Parcours membre           | <code>docs/recette/RECETTE-01-*.md</code>   | L'Expéditeur et le Voyageur, par l'écran.                           |
| n° 2 — Back-office               | <code>docs/recette/RECETTE-02-*.md</code>   | Le support, la médiation, la finance, par l'écran d'administration. |
| n° 3 — API                       | <code>docs/recette/RECETTE-03-API.md</code> | <b>Le serveur seul, par le fil HTTP.</b>                            |
| n° 4 — Technique et exploitation | <code>docs/recette/RECETTE-04-*.md</code>   | Le démarrage, les crons, les événements, la supervision.            |

Quand une fiche de ce cahier dépend d'un état créé par un autre cahier (par exemple un litige décidé par un médiateur), elle le signale et propose une alternative jouable en `curl`.

## 1.4 Documents de référence

Le testeur n'a besoin d'aucun autre document pour dérouler ce cahier, mais trois sources font autorité en cas de doute, dans cet ordre :

1. **Le code et ses tests** — précedence absolue.
2. `docs/livrables/03-YAMBA-DOCUMENTATION-API.md` — la documentation d'usage de l'API (passerelle, conventions, guides pas à pas, catalogue des codes d'erreur, webhooks).
3. `docs/livrables/_api-reference.generated.md` — la référence générée des **173 opérations** (méthode, chemin, authentification, permission, paramètres, corps, réponses, schémas), produite depuis les cinq documents OpenAPI 3.1 des services.

Les documents OpenAPI vivants sont servis par les services eux-mêmes :

`http://localhost:6001/openapi.json`, `:6002`, `:6003`, `:6004`, `:6005` — avec une visionneuse Scalar sur `/docs` pour les quatre premiers.

## 2. Prérequis et outillage

### 2.1 Ce qu'il faut sur le poste

| Outil                                        | Rôle                                                                | Vérification                                           |
|----------------------------------------------|---------------------------------------------------------------------|--------------------------------------------------------|
| <code>curl</code>                            | Tous les appels du cahier.                                          | <code>curl --version</code>                            |
| <code>jq</code>                              | Lire les réponses JSON et extraire les identifiants du jeu d'essai. | <code>jq --version</code>                              |
| <code>node</code> (≥ 20) et <code>npx</code> | Démarrer les services, rejouer le jeu d'essai.                      | <code>node -v</code>                                   |
| MongoDB (Atlas ou local)                     | La base unique des cinq services.                                   | <code>DATABASE_URL</code> dans <code>.env</code>       |
| Redis                                        | OTP, sessions, fenêtres sudo, compteurs.                            | <code>REDIS_DATABASE_URI</code> dans <code>.env</code> |
| Mailpit (facultatif)                         | Lire les codes OTP envoyés par email en local.                      | <code>http://localhost:8025</code>                     |

| Outil                                | Rôle                                                                | Vérification                                    |
|--------------------------------------|---------------------------------------------------------------------|-------------------------------------------------|
| Redpanda (facultatif)                | Le broker d'événements. Les services démarrent sans lui.            | <code>bash scripts/redpanda-bootstrap.sh</code> |
| <code>stripe CLI</code> (facultatif) | Rejouer le webhook Stripe. Sans elle, les fiches concernées sont ➡. | <code>stripe --version</code>                   |

Le cahier suppose un environnement **de recette ou de développement** : `NODE_ENV` différent de `production`. Plusieurs fiches (fournisseur de paiement `FAKE`, exposition complète des `details`) ne sont jouables que là.

## 2.2 Démarrer la plateforme

Deux façons de faire, selon ce que l'on veut prouver.

### a) Le mode courant — tout tourne.

```
npm run dev
```

Cette commande sert les six processus : la passerelle (8080), l'auth-service (6001), le trip-service (6002), le deal-service (6003), le notification-service (6004), le message-service (6005), plus les deux fronts. Le cahier n'utilise que la passerelle.

### b) La vérification préalable — chaque bundle démarre-t-il vraiment ?

```
bash scripts/smoke-services.sh
```

Ce script construit et lance chaque service **sur un port décalé de +900** (donc sans conflit avec `npm run dev`), interroge son `/health`, puis l'arrête. Il coupe crons et relais pour ne rien écrire. Sortie attendue : six lignes . Une ligne  signifie qu'un bundle ne démarre pas — le cahier ne peut pas commencer.

```

 api-gateway - {"status":"ok","service":"api-gateway",...
 auth-service - {"status":"ok","service":"auth-service",...
 trip-service - ...
 deal-service - ...
 notification-service - ...
 message-service - ...

```

**Rappel qui a coûté cher** : un build vert n'est pas un service démarré. Chaque service exige un dossier `src/assets` versionné ; sans lui webpack échoue en `ENOENT`, `nx serve` est ignoré et la passerelle répond 500. C'est exactement ce que ce script attrape.

## 2.3 Rejouer le jeu d'essai

Le jeu d'essai est la **remise à zéro de la recette**. Il efface puis recrée les trajets et les deals des comptes de démonstration, en couvrant chaque statut du cycle de vie et six corridors internationaux.

```
npx tsx --env-file=.env packages/libs/prisma/scripts/seed-deals.ts
```

**Ne jamais sourcer `.env` dans `zsh`** avant de lancer un seed : le mot de passe Mongo contient des caractères que le shell interprète. On passe toujours par `--env-file=.env`.

Ce que le script garantit :

- **12 comptes** aux identifiants stables entre deux exécutions (upsert par email normalisé) ;
- **8 trajets publiés** et **24 deals** couvrant tous les statuts ( `PENDING` , `ACCEPTED` , `PICKED_UP` , `DELIVERED` , `DISPUTED` , `COMPLETED` , `CANCELLED` , `DECLINED` , `EXPIRED` ) ;
- des **identifiants de deals et de trajets neufs à chaque exécution** (effacement puis recréation) — d'où l'obligation de les relire dans le fichier de sortie, jamais de les recopier d'une session précédente ;
- l'invariant de capacité recalculé ( `reservedKg` = somme des poids des deals actifs ) ;
- une **page Voyageur complète et Stripe factice** pour chaque transporteur, ce qui fait passer la porte D31 à l'acceptation ;
- un **vrai code de livraison** — `742891` — haché et chiffré sur tous les deals passés par la récupération ;
- une **conversation vivante** sur le deal accepté du corridor 1 : deux messages, un rendez-vous proposé, un message signalé.

## 2.4 Les comptes du jeu d'essai

Mot de passe **commun à tous** : `Yamba-Dev-2026!`

| Clé         | Email                                           | Rôles                        |
|-------------|-------------------------------------------------|------------------------------|
| thomas      | <code>thomas.carrier@seed.yamba.dev</code>      | Expéditeur + <b>Voyageur</b> |
| marc        | <code>marc.carrier@seed.yamba.dev</code>        | Expéditeur + <b>Voyageur</b> |
| ines        | <code>ines.carrier@seed.yamba.dev</code>        | Expéditeur + <b>Voyageur</b> |
| adebayo     | <code>adebayo.carrier@seed.yamba.dev</code>     | Expéditeur + <b>Voyageur</b> |
| linh        | <code>linh.carrier@seed.yamba.dev</code>        | Expéditeur + <b>Voyageur</b> |
| josephine   | <code>josephine.carrier@seed.yamba.dev</code>   | Expéditeur + <b>Voyageur</b> |
| aminata     | <code>aminata.shipper@seed.yamba.dev</code>     | Expéditeur                   |
| joao        | <code>joao.shipper@seed.yamba.dev</code>        | Expéditeur                   |
| chinwe      | <code>chinwe.shipper@seed.yamba.dev</code>      | Expéditeur                   |
| marieclaire | <code>marieclaire.shipper@seed.yamba.dev</code> | Expéditeur                   |
| mai         | <code>mai.shipper@seed.yamba.dev</code>         | Expéditeur                   |
| pauline     | <code>pauline.shipper@seed.yamba.dev</code>     | Expéditeur                   |

Le mot de passe contient un `!`. En zsh interactif, on l'écrit **toujours entre apostrophes simples** ( `'Yamba-Dev-2026!'` ) ou dans un fichier ; entre guillemets doubles, l'historique du shell peut le mutiler.

Aucun de ces comptes n'est administrateur. Pour les fiches admin, il faut promouvoir un compte :

```
npx tsx --env-file=.env packages/libs/prisma/scripts/grant-admin.ts \  
aminata.shipper@seed.yamba.dev --role SUPPORT
```

## 2.5 Les deals du jeu d'essai, par statut

Les identifiants changent à chaque exécution ; la **clé** ( `bzv-accepted...` ) est stable. Le tableau sert à choisir la bonne cible pour chaque fiche.

| Clé           | Statut    | Expéditeur  | Voyageur | Utile pour                                              |
|---------------|-----------|-------------|----------|---------------------------------------------------------|
| bzv-pending   | PENDING   | aminata     | thomas   | Acceptation, refus, annulation, expiration              |
| gru-pending   | PENDING   | joao        | ines     | Deuxième cible PENDING (concurrency)                    |
| bzv-accepted  | ACCEPTED  | pauline     | thomas   | <b>Conversation vivante</b> , rendez-vous, récupération |
| yul-accepted  | ACCEPTED  | marieclaire | marc     | Annulation après acceptation (retenue)                  |
| bzv-picked    | PICKED_UP | aminata     | thomas   | Code de livraison, remise, jalons                       |
| bzv-tracking  | PICKED_UP | pauline     | thomas   | Séquence de jalons déjà complète                        |
| los-picked    | PICKED_UP | chinwe      | adebayo  | Jalons partiels (2 sur 3)                               |
| sgn-picked    | PICKED_UP | mai         | linh     | Jalons partiels (1 sur 3)                               |
| bzv-delivered | DELIVERED | joao        | thomas   | Confirmation, litige                                    |
| yul-delivered | DELIVERED | aminata     | marc     | Deuxième cible DELIVERED                                |
| bzv-disputed  | DISPUTED  | chinwe      | thomas   | Déclaration du Voyageur, lecture seule du fil           |
| los-disputed  | DISPUTED  | mai         | adebayo  | Deuxième litige                                         |
| bzv-completed | COMPLETED | mai         | thomas   | Notation double aveugle                                 |
| gru-completed | COMPLETED | ines        | ines     | Notation, portefeuille                                  |
| bzv-cancelled | CANCELLED | aminata     | thomas   | Transitions refusées depuis un état terminal            |
| bzv-declined  | DECLINED  | joao        | thomas   | Idem                                                    |
| bzv-expired   | EXPIRED   | chinwe      | thomas   | Idem                                                    |

Et les trajets, par clé : `bzv-upcoming` (Paris → Brazzaville, départ J+10, Thomas), `bzv-perkg` (Paris → Brazzaville, départ J+15, **11,50 €/kg**, 23 kg, électronique +20 %, alimentaire **refusé**), `yul`, `gru`, `los`, `sgn`, `fih`, `bzv-inflight`.

## 2.6 Préparer le terminal

Tous les appels du cahier supposent ces variables et ces fonctions. On les colle une fois dans le terminal de recette.

```
# La passerelle est le SEUL point d'entrée. On n'appelle jamais :6001...:6005 directement.
export BASE="http://localhost:8080/api"
export SEED="packages/libs/prisma/scripts/seed-output.json"

# Identifiants du jeu d'essai (relus à chaque exécution du seed)
bid () { jq -r --arg k "$1" '.bookings[] | select(.key==$k) | .id' "$SEED"; } # id d'un deal
tid () { jq -r --arg k "$1" '.trips[$k]' "$SEED"; } # id d'un trajet
uid () { jq -r --arg k "$1" '.users[$k]' "$SEED"; } # id d'un membre

# Ouvrir une session dans un pot de biscuits dédié : login <email> <fichier>
login () {
  curl -s -c "$2" -X POST "$BASE/auth/login" \
    -H 'Content-Type: application/json' -H 'x-locale: fr' \
    -d '{"email":"'${1}'',"password":"'Yamba-Dev-2026!',"rememberMe":true}'
}

# Un appel qui montre le code HTTP et le corps (le format de preuve du cahier)
show () { curl -s -o /tmp/body.json -w '%{http_code}\n' "$@"; jq . /tmp/body.json 2>/dev/null ||
cat /tmp/body.json; }
```

Exemple d'usage, à faire une fois avant les chapitres 5 et suivants :

```
login aminata.shipper@seed.yamba.dev shipper.txt
login thomas.carrier@seed.yamba.dev carrier.txt
login pauline.shipper@seed.yamba.dev shipper2.txt
login joao.shipper@seed.yamba.dev shipper3.txt

DEAL_PENDING=$(bid bzv-pending)
DEAL_ACCEPTED=$(bid bzv-accepted)
DEAL_PICKED=$(bid bzv-picked)
DEAL_DELIVERED=$(bid bzv-delivered)
DEAL_COMPLETED=$(bid bzv-completed)
TRIP_PERKG=$(tid bzv-perkg)
echo "$DEAL_PENDING $DEAL_ACCEPTED $DEAL_PICKED $TRIP_PERKG"
```

## 2.7 Tenir la session entre deux appels

L'API pose les jetons dans des **cookies** `httpOnly` — jamais dans le corps de la réponse. Avec `curl`, on tient donc un **pot de biscuits** (cookie jar) :

- `-c` fichier **écrit** les cookies reçus (Set-Cookie) dans le fichier ;
- `-b` fichier **renvoie** les cookies du fichier avec la requête ;
- les deux ensemble (`-c jar -b jar`) sont nécessaires dès qu'un appel **fait tourner** la session (POST `/auth/refresh`, qui remplace les deux jetons).

```
# Ouvrir la session
curl -s -c shipper.txt -X POST "$BASE/auth/login" -H 'Content-Type: application/json' \
  -d '{"email":"aminata.shipper@seed.yamba.dev","password":"Yamba-Dev-2026!"}'

# Utiliser la session
curl -s -b shipper.txt "$BASE/auth/me" | jq '.user.email, .roles'

# Faire tourner la session (les DEUX drapeaux)
curl -s -b shipper.txt -c shipper.txt -X POST "$BASE/auth/refresh"
```

Inspecter le pot pour comprendre ce que le serveur a posé :

```
grep -E 'access_token|refresh_token' shipper.txt | awk '{print $6, $5}'
```

**Deux sessions distinctes = deux fichiers distincts.** Ne jamais mélanger le pot d'un Expéditeur et celui d'un Voyageur : la moitié des fiches de sécurité de ce cahier repose sur cette séparation. De même, la session **administrateur** vit dans son propre pot (`admin.txt`) avec ses propres cookies (`admin_preauth`, `admin_access_token`, `admin_refresh_token`) : un compte administrateur connecté comme membre n'ouvre **aucune** route `/admin/*`.

Variante **Bearer** (utile pour un client mobile, D36) : les intergiciels acceptent aussi `Authorization: Bearer <access_token>`. Le jeton n'étant aujourd'hui lisible que dans le cookie, on l'extrait du pot pour les fiches qui en ont besoin :

```
TOKEN=$(awk '/access_token/ {print $7}' shipper.txt | head -1)
curl -s "$BASE/auth/me" -H "Authorization: Bearer $TOKEN" | jq .success
```

## 2.8 Une seule porte d'entrée

Tout passe par `http://localhost:8080/api`. C'est la passerelle, et elle seule applique le CORS, le limiteur de débit, le mode maintenance et l'identifiant de corrélation. Appeler un service par son port (`:6001` ...) contourne

ces quatre gardes : c'est une manipulation de diagnostic, jamais une fiche de recette — sauf **une exception assumée**, le webhook Stripe (chapitre 8), qui est un appel serveur à serveur sur `http://localhost:6003/webhooks/stripe`.

La table de routage, telle que la passerelle la déclare (l'ordre compte : le plus spécifique d'abord, l'auth-service en dernier recours) :

| Préfixe client                                                                                                                                            | Service                    | Chemin reçu                                       |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------|---------------------------------------------------|
| <code>/api/trips/*</code> , <code>/api/uploads/*</code>                                                                                                   | trip-service :6002         | <code>/trips/*</code> , <code>/uploads/*</code>   |
| <code>/api/deals/*</code> , <code>/api/me/bookings/*</code> , <code>/api/me/deals/*</code> ,<br><code>/api/me/wallet/*</code> , <code>/api/track/*</code> | deal-service :6003         | idem sans <code>/api</code>                       |
| <code>/api/messages/*</code>                                                                                                                              | message-service :6005      | <code>/messages/*</code>                          |
| <code>/api/me/notifications/*</code> , <code>/api/webhooks/email/*</code>                                                                                 | notification-service :6004 | idem sans <code>/api</code>                       |
| <code>/api/admin/disputes</code> , <code>/alerts</code> , <code>/finances</code> , <code>/deals</code>                                                    | deal-service :6003         | <code>/admin/*</code>                             |
| <code>/api/admin/trips</code> , <code>/api/admin/tickets</code>                                                                                           | trip-service :6002         | <code>/admin/*</code>                             |
| <code>/api/admin/conversations</code>                                                                                                                     | message-service :6005      | <code>/admin/conversations/*</code>               |
| <b>tout le reste</b>                                                                                                                                      | auth-service :6001         | <b>chemin inchangé, <code>/api</code> compris</b> |

Conséquence de lecture : dans la référence générée, les chemins sont écrits **du point de vue du service** (`/auth/login`, `/trips/search`, `/deals/{id}`). Pour obtenir l'URL client, on préfixe par `/api`.

## 2.9 Forcer le fournisseur de paiement factice

Plusieurs fiches du chapitre 5.3 créent un vrai deal de bout en bout. Avec Stripe, il faudrait un navigateur pour confirmer la carte. Avec le fournisseur **FAKE**, l'autorisation est immédiate (`status: AUTHORIZED`, `clientSecret: null`) et **tout le parcours devient jouable en curl**.

Le choix se fait à partir de l'environnement : `STRIPE_SECRET_KEY` présent → Stripe ; absent → Fake (et le service **refuse de démarrer** sans clé si `NODE_ENV=production`).

**Piège** : `nx serve` charge le `.env` racine et **écrase** les variables passées en ligne de commande. Pour forcer le mode factice, on lance le bundle construit :

```
bash npx nx build deal-service cd apps/deal-service && STRIPE_SECRET_KEY= node --env-file=../../.env dist/main.js
```

Node laisse alors l'environnement du processus l'emporter. Vérification : la réponse de `POST /deals/payment-intents` porte `"provider": "FAKE"` et `"clientSecret": null`.

Le fournisseur factice est **en mémoire du processus** : une intention créée puis le service redémarré, l'intention est perdue. Une exception a été prévue pour la recette : les identifiants du jeu d'essai (`pi_fake_seed_...`) sont adoptés comme « autorisés » à la première lecture, ce qui rend les acceptations et les refus des deals seedés jouables après un redémarrage.

Les fiches qui exigent Stripe (webhook signé, tableau de bord Express, rapprochement) sont marquées ►► quand la clé n'est pas configurée.

### 3. Conventions du cahier

#### 3.1 Numérotation

API-<SERVICE>-<n> , où <SERVICE> désigne le domaine de la fiche :

| Préfixe     | Domaine                                                                        |
|-------------|--------------------------------------------------------------------------------|
| API-GW-n    | Transverse : passerelle, santé, maintenance, limiteur, erreurs, pagination     |
| API-AUTH-n  | auth-service : comptes, sessions, sudo, profil, signalements, RGPD             |
| API-TRIP-n  | trip-service : recherche, trajets, cycle de vie, documents, téléversement      |
| API-DEAL-n  | deal-service : paiement, cycle de vie du deal, remise, litige, notation, suivi |
| API-MSG-n   | message-service : fil, rendez-vous, numéro, signalement de message             |
| API-NOTIF-n | notification-service : notifications, webhook email                            |
| API-SEC-n   | Sécurité transverse                                                            |
| API-IDEM-n  | Idempotence et concurrence                                                     |
| API-HOOK-n  | Webhooks entrants                                                              |

Les numéros ne sont **jamais réutilisés** : une fiche retirée laisse son numéro vide, une fiche ajoutée prend le suivant. C'est ce qui rend un rapport d'anomalie citable d'une campagne à l'autre.

#### 3.2 Verdicts

| Verdict | Sens                                                                                                                                    |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------|
| OK      | Le code HTTP, le corps attendu et l'effet de bord sont tous conformes.                                                                  |
| KO      | Au moins un des trois diverge. Une anomalie est ouverte (§9.2).                                                                         |
| PARTIEL | Le code HTTP est bon mais le corps ou l'effet de bord diverge sur un point mineur, sans conséquence fonctionnelle. À tracer quand même. |
| ►►      | Non joué : configuration absente (Stripe, Redpanda, SMTP), ou dépendance d'un autre cahier. La raison est obligatoire.                  |

Un verdict **OK exige les trois colonnes**. Un code 200 avec un corps qui ne contient pas le champ attendu n'est pas un OK : c'est un PARTIEL, au mieux.

#### 3.3 Gravité

| Gravité   | Définition                                                          | Exemple                                                                                                         |
|-----------|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Bloquante | Une garde de sécurité ou d'argent ne tient pas côté serveur.        | Le code de livraison apparaît dans une vue Voyageur ; un tiers lit un deal ; une session membre ouvre /admin/*. |
| Majeure   | Une règle métier n'est pas appliquée, ou un conflit n'est pas typé. | Une double acceptation passe ; un 409 arrive sans details.code.                                                 |
| Mineure   | Un écart de forme sans conséquence fonctionnelle.                   | Un champ optionnel absent ; un statut 400 là où 404 serait plus juste (trip-service, écart connu).              |

### 3.4 La forme d'une fiche

Chaque fiche donne cinq choses, et toujours dans cet ordre :

**API-XXX-nn** — **Titre de l'intention** **Gravité** · **Prérequis** (session, état du deal, configuration) **Appel** — la commande exacte, copiable **Attendu** — le **code HTTP** puis les **champs du corps qui comptent** (jamais tout le corps) **Effet de bord** — ce qu'il faut aller vérifier ailleurs (une relecture, un journal, un compteur) **Verdict** — à porter dans le tableau du chapitre 9

Les champs « qui comptent » sont ceux qu'un client lit pour décider : le statut, `details.code`, `allowedActions`, un compteur, une date d'ouverture. Le reste du corps est décrit par la référence générée ; ce cahier ne le recopie pas.

### 3.5 Lire une réponse d'erreur

La plateforme a **quatre formes** d'erreur. Le testeur doit savoir laquelle il regarde, sinon il cherche un `details.code` là où il n'y en a pas par construction.

**1. ErrorResponse** — toute erreur applicative passée par l'intergiciel d'erreurs :

```
{ "status": "error", "message": "...", "errors": { "champ": "message" },
  "details": { "type": "booking", "code": "QUOTE_DIVERGENCE" } }
```

**2. UnhandledError** — exception non prévue : { "status": "error", "error": "Something went wrong, please try again!" }. Noter le champ **error**, pas **message**. Toute apparition de cette forme est une anomalie.

**3. UnauthorizedResponse** — les 401 et 403 émis **directement** par les intergiciels d'authentification, hors intergiciel d'erreurs : { "message": "..." }, parfois avec **code** (`ACCOUNT_DELETED`, `ACCOUNT_SUSPENDED`, `ACCOUNT_RESTRICTED`, `ADMIN_PERMISSION_DENIED`). Corps **plat** : pas de **status**, pas de **details**.

**4. Réponses de la passerelle** — 429 { "error": "Too many requests, please try again later!" } et 503 { "code": "MAINTENANCE", "message", "messages": { "fr", "en" }, "retryAfterSeconds": 300 }.

**Règle de lecture du cahier** : on ne juge jamais un refus sur le **message** anglais. On juge sur le **statut HTTP** et sur le **code** (`details.code`, ou **code** à la racine pour les formes 3 et 4).

### 3.6 Le contrat `details.code`

C'est le point le plus important du chapitre 4 et il mérite d'être posé ici.

L'intergiciel d'erreurs expose `details` selon deux conditions, dont **une seule suffit** :

- `details.type` appartient à la liste sûre : `otp`, `booking`, `password`, `register`, `locale`, `favorite`, `oauth`, `trip` ;
- ou** `details.code` est une chaîne non vide — auquel cas `details` est exposé **quel que soit l'environnement**.

Cette seconde condition est la correction A146 : avant elle, un code posé sans `type` disparaissait en production, ce qui cassait `SUDO_REQUIRED`, `DELIVERY_CODE_IN_MESSAGE` et `T00_EARLY`. La fiche **API-GW-13** en fait la preuve, et c'est la fiche à rejouer en priorité sur un environnement où `NODE_ENV=production`.

Hors production, un `details` **sans** `type` sûr et **sans** `code` reste exposé pour le débogage ; en production il est masqué. Les traces de pile ne sortent jamais, dans aucun environnement.

### 3.7 Notation des montants, dates et identifiants

- Tout montant est un **entier en centimes** accompagné d'un `currencyCode` ISO 4217. Une valeur décimale dans un champ `...Cents` est une anomalie **bloquante**. Seule exception documentée : les **cartes de résultat de recherche**, qui exposent des prix en unités (`minPrice`, `pricePerKg`, `totalForWeight`) et un **symbole** (`currency`) — DTO d'affichage, pas donnée de calcul.
- Les dates sont des chaînes **ISO 8601 en UTC** (`2026-10-12T20:30:00.000Z`), sauf les champs explicitement locaux d'un trajet (`departureDateLocal`, `departureTimeLocal` + `originTimezone`) et les dates **formatées par le serveur** des cartes de recherche (`travelDate`, `departureTime`), pilotées par le paramètre `locale`.
- Les identifiants sont des `objectId` MongoDB (24 caractères hexadécimaux). Un identifiant malformé donne **400**. Les membres sont désignés publiquement par leur **slug** (`seed-aminata`...), jamais par leur identifiant.

### 3.8 Hygiène de campagne

1. **Rejouer le jeu d'essai avant chaque campagne** et relire les identifiants : ils changent.
2. **Noter l'`x-correlation-id`** de toute réponse d'erreur inattendue : c'est la clé partagée avec les journaux serveur et Sentry.
3. **Ne jamais recopier un identifiant d'une campagne précédente** : il pointe sur un document effacé, et l'API répondra 404 — un faux KO.
4. **Une fiche = un état de départ connu**. Si une fiche précédente a fait avancer un deal, on rejoue le jeu d'essai plutôt que d'improviser.
5. **Ne jamais versionner** les pots de biscuits, les exports de données personnelles ni les captures d'écran.

---

## 4. Scénarios transverses

Ce chapitre se joue **en premier**. Il vérifie les gardes que la passerelle applique à *toutes* les requêtes, et le contrat d'erreur que tous les services partagent. Si l'une de ces fiches est KO, les chapitres suivants héritent du défaut et leurs verdicts ne veulent plus rien dire.

### 4.1 Santé et sonde publique

Trois adresses « système » vivent directement à la passerelle, hors proxy.

---

**API-GW-01 — La sonde publique répond ok quand tout va bien** **Gravité** : majeure · **Prérequis** : les six processus démarrés.

```
curl -s -o /tmp/status.json -w 'HTTP %{http_code}\n' "$BASE/status"
jq '{status, n: (.services|length), services: [.services[] | {name, reachable, status}]}'
/tmp/status.json
```

**Attendu — HTTP 200**. Corps : `status: "ok"`, un tableau `services` de **cinq** entrées (`auth-service`, `trip-service`, `deal-service`, `notification-service`, `message-service`), chacune avec `reachable: true`, `status: "ok"` et un `ms` numérique, plus un horodatage `at`.

**Ce que le corps ne doit PAS contenir** : aucune URL interne, aucun message d'erreur brut, aucune trace de pile. Le corps public est volontairement pauvre.

**Effet de bord :** l'en-tête `Cache-Control: no-store` est posé ; la réponse est mise en cache **10 secondes** côté passerelle (deux appels rapprochés renvoient le même `at`).

---

**API-GW-02 — La sonde publique passe à `down` quand un service se tait Gravité :** majeure · **Prérequis :** pouvoir arrêter un service.

```
# 1. Arrêter le message-service (Ctrl-C sur son processus, ou :)
kill $(lsof -ti :6005)
# 2. Attendre la fin du cache de 10 s, puis :
curl -s -o /tmp/status.json -w 'HTTP %{http_code}\n' "$BASE/status"
jq '{status, services: [.services[] | select(.reachable==false) | .name]}' /tmp/status.json
```

**Attendu — HTTP 503.** Corps : `status: "down"`, l'entrée `message-service` avec `reachable: false` et `status: null`. Les quatre autres restent `reachable: true`.

**Variante degraded :** couper **Redis** (et non un service) puis rappeler la sonde. Les cinq services restent joignables mais leur `/health` répond `degraded` → la sonde répond **HTTP 503** avec `status: "degraded"`. C'est la nuance qui compte pour un moniteur : `down` = un processus muet, `degraded` = un processus vivant mais amputé d'une dépendance.

**Effet de bord :** aucun. La sonde ne modifie rien. Redémarrer le service et vérifier le retour à `ok` avant de continuer.

---

**API-GW-03 — La santé de la passerelle elle-même Gravité :** mineure.

```
curl -s -o /tmp/gw.json -w 'HTTP %{http_code}\n' "http://localhost:8080/gateway-health"
jq '{status, service, version, uptimeSeconds, checks}' /tmp/gw.json
```

**Attendu — HTTP 200.** `service: "api-gateway"`, `status: "ok"`, un `uptimeSeconds` entier, et un contrôle maintenance à `ok: true` (il passe à `false` quand le mode lecture seule est actif). Noter que cette route est **sans préfixe `/api`** : elle n'est pas proxifiée.

---

**API-GW-04 — Le `/health` de chaque service, toujours 200 Gravité :** mineure · **Prérequis :** appels directs assumés (fiche de diagnostic).

```
for p in 6001 6002 6003 6004 6005; do
  printf '%s → ' "$p"; curl -s -m 3 "http://localhost:$p/health" | jq -c '{service, status, checks: (.checks|keys)}'
done
```

**Attendu — HTTP 200 pour les cinq**, même en cas de panne : un service qui répond est vivant, c'est **le corps** qui dit s'il est en forme (`status: "ok"` ou `"degraded"`). Chaque corps porte les contrôles `mongo` et `redis`, chacun avec `ok`, `ms` et `error`. Chaque contrôle est borné à **2 secondes**.

**Piège à ne pas signaler comme anomalie :** un `/health` qui répond 200 avec `status: "degraded"` est le comportement voulu, pas un défaut.

---

## 4.2 Mode maintenance

Le mode lecture seule bloque **les écritures seulement**, et **jamais** sous `/api/auth/`, `/api/admin/` ni `/api/maintenance` — sinon personne ne pourrait se connecter pour le lever.

---

**API-GW-05 — L'état public de la maintenance est lisible sans session Gravit ** : mineure.

```
curl -s "$BASE/maintenance" | jq .
```

**Attendu — HTTP 200**, corps `{ "enabled": false, "message": { "fr": "", "en": "" }, "scheduledAt": null }` hors maintenance. L'en-t te `Cache-Control: no-store` est pos . Aucune session n'est requise : c'est la route qui alimente les bandeaux des deux fronts.

---

**API-GW-06 — Une  criture est refus e en maintenance Gravit ** : majeure · **Pr requis** : pouvoir red marrer la passerelle avec une variable.

Deux interrupteurs existent : le document de configuration en base (r gl  par l'admin via `PUT /api/admin/maintenance`) et la variable d'environnement, qui l'emporte — pour le jour o  c'est la base qui est en panne. Le plus simple en recette est la variable :

```
# Relancer la passerelle en mode lecture seule
MAINTENANCE_MODE=on \
MAINTENANCE_MESSAGE_FR="Maintenance planifi e, retour   14 h." \
MAINTENANCE_MESSAGE_EN="Scheduled maintenance, back at 2 pm." \
node --env-file=.env apps/api-gateway/dist/main.js
```

Puis, dans un autre terminal :

```
show -X POST "$BASE/trips/$TRIP_PERKG/favorite" -b shipper.txt
```

**Attendu — HTTP 503**. Corps :

```
{ "code": "MAINTENANCE",
  "message": "Maintenance planifi e, retour   14 h.",
  "messages": { "fr": "Maintenance planifi e, retour   14 h.", "en": "Scheduled maintenance,
back at 2 pm." },
  "retryAfterSeconds": 300 }
```

En-t te **Retry-After: 300** pr sent.

**Effet de bord** : aucun favori n'est cr  . Relire `GET /api/trips/favorites` apr s la lev e de la maintenance pour le confirmer.

---

**API-GW-07 — Les lectures et les exceptions passent en maintenance Gravit ** : majeure · **Pr requis** : maintenance active (fiche pr c dente).

```
# a) Une lecture – doit passer
show "$BASE/trips/search?from=Paris&limit=3"
# b) Une écriture sous /api/auth/ – doit passer (sinon personne ne se connecte)
show -X POST "$BASE/auth/login" -c tmp.txt -H 'Content-Type: application/json' \
  -d '{"email":"aminata.shipper@seed.yamba.dev","password":"Yamba-Dev-2026!"}'
# c) L'état de la maintenance – doit passer
show "$BASE/maintenance"
# d) Une écriture admin – doit passer (l'admin doit pouvoir lever la maintenance)
show -X PUT "$BASE/admin/maintenance" -b admin.txt -H 'Content-Type: application/json' \
  -d '{"enabled":false,"messageFr":"","messageEn":"","reason":"Fin de la fenêtre de maintenance planifiée"}'
```

**Attendu :** (a) 200, (b) 200 avec les cookies posés, (c) 200 avec `enabled: true`, (d) 200 ou 403 selon le profil admin — **jamais 503** pour aucune des quatre.

**Point de vigilance :** un `GET` qui répondrait 503 est une **anomalie majeure** : la plateforme resterait consultable en maintenance, c'est tout l'intérêt du mode lecture seule.

### 4.3 Limiteur de débit

Fenêtre de **15 minutes**. Le plafond dépend de la **signature du jeton de session**, pas de la simple présence d'un cookie : la passerelle vérifie l'empreinte HMAC, ce qui ne se contrefait pas.

| Appelant                                                                | Plafond                             |
|-------------------------------------------------------------------------|-------------------------------------|
| Anonyme, ou jeton absent                                                | <b>100</b> requêtes / 15 min / IP   |
| Jeton de session <b>valide</b> (cookie membre, cookie admin, ou Bearer) | <b>1 000</b> requêtes / 15 min / IP |
| Jeton <b>expiré ou forgé</b>                                            | retombe sur <b>100</b>              |

Les requêtes en échec ne sont pas comptées, et les en-têtes standards `RateLimit-Limit`, `RateLimit-Remaining`, `RateLimit-Reset` sont renvoyés.

**API-GW-08 — Le plafond anonyme et le plafond connecté sont différents** **Gravité :** majeure · **Prérequis :** la passerelle vient d'être redémarrée (compteurs à zéro).

```
# a) Le plafond annoncé pour un appelant anonyme
curl -s -D - -o /dev/null "$BASE/maintenance" | grep -i '^ratelimit'

# b) Le plafond annoncé pour un appelant dont le jeton est VALIDE
curl -s -D - -o /dev/null -b shipper.txt "$BASE/maintenance" | grep -i '^ratelimit'

# c) Le plafond avec un jeton FORGÉ (signature invalide) – doit retomber sur le plafond anonyme
curl -s -D - -o /dev/null -H 'Authorization: Bearer aaa.bbb.ccc' "$BASE/maintenance" | grep -i '^ratelimit'
```

**Attendu :** (a) `RateLimit-Limit: 100` · (b) `RateLimit-Limit: 1000` · (c) `RateLimit-Limit: 100`.

C'est la preuve que le plafond dépend d'une **vérification** et non d'une présence. La branche « membre connecté » était inerte avant la correction A147 (la passerelle ne decode aucun jeton pour le reste), et tout le monde subissait 100 requêtes — ce qu'une navigation normale atteint.

**Franchir le plafond (facultatif, à faire en dernier) :**

```
for i in $(seq 1 110); do curl -s -o /dev/null -w '%{http_code} ' "$BASE/maintenance"; done;
echo
```

**Attendu :** une série de 200 puis des 429. Le corps d'un 429 de la passerelle est `{ "error": "Too many requests, please try again later!" }` — **champ error**, forme n° 4 du §3.5.

**Effet de bord :** le compteur est par adresse IP et par fenêtre. Après cette fiche, attendre la fin de la fenêtre (`RateLimit-Reset`) ou redémarrer la passerelle avant de continuer le cahier.

---

## 4.4 Identifiant de corrélation

---

**API-GW-09 — L'identifiant de corrélation est posé et renvoyé** **Gravité :** majeure.

```
# a) Sans identifiant fourni : la passerelle en génère un
curl -s -D - -o /dev/null -b shipper.txt "$BASE/deals/$DEAL_PENDING" | grep -i 'x-correlation-id'

# b) Avec un identifiant fourni : il est CONSERVÉ de bout en bout
curl -s -D - -o /dev/null -b shipper.txt -H 'x-correlation-id: recette-api-0001' \
  "$BASE/deals/$DEAL_PENDING" | grep -i 'x-correlation-id'
```

**Attendu :** (a) un en-tête `x-correlation-id` porteur d'un UUID v4 · (b) le **même** `recette-api-0001` renvoyé.

**Périmètre exact — à ne pas signaler comme anomalie :** trois services renvoient l'en-tête dans la réponse (`deal-service`, `message-service`, `notification-service`). L'`auth-service` et le `trip-service` le **reçoivent** et le journalisent, mais ne le renvoient pas aujourd'hui. On teste donc l'aller-retour sur une route `deal`, `message` ou `notification`.

**Effet de bord :** l'identifiant est copié dans chaque événement de la boîte d'envoi (`correlationId`) et suit la transition jusqu'aux consommateurs et jusqu'à Sentry en cas de 5xx. Le vérifier après une transition (chapitre 5.3) en lisant la ligne `OutboxEvent`.

---

## 4.5 Langue et en-tête `x-locale`

---

La liste des langues est unique : `fr` (défaut) et `en`. La résolution est tolérante : `fr-FR`, `FR`, `en-US`, `en;q=0.9` sont ramenés à `fr` / `en` ; absent ou inconnu → `fr`.

**API-GW-10 — L'en-tête `x-locale` pilote le formatage serveur** **Gravité :** mineure.

```
curl -s "$BASE/trips/search?from=Paris&limit=1&locale=fr" | jq '.trips[0] | {travelDate, departureTime}'
curl -s "$BASE/trips/search?from=Paris&limit=1&locale=en" | jq '.trips[0] | {travelDate, departureTime}'
curl -s "$BASE/messages/quick-replies" -b shipper.txt -H 'x-locale: en' | jq '.items[0]'
curl -s "$BASE/messages/quick-replies" -b shipper.txt -H 'x-locale: fr' | jq '.items[0]'
```

**Attendu :** les dates formatées par le serveur diffèrent (`dim. 12 oct.` contre `Sun, 0ct 12`) ; les réponses rapides portent **la même clé** dans les deux langues et **un texte différent**.

---

#### API-GW-11 — Une langue non supportée est refusée proprement **Gravité** : mineure.

```
show -X PATCH "$BASE/auth/me/locale" -b shipper.txt -H 'Content-Type: application/json' \
-d '{"locale":"pt"}'
```

**Attendu** — HTTP 400, `details.type: "locale"` et `details.code: "LOCALE_UNSUPPORTED"`.

**Contre-épreuve** :

```
show -X PATCH "$BASE/auth/me/locale" -b shipper.txt -H 'Content-Type: application/json' -d
'{"locale":"en"}'
curl -s -b shipper.txt "$BASE/auth/me" | jq '.user.preferredLocale'
# → 200 { "success": true, "preferredLocale": "en" } puis "en"
```

**Effet de bord** : la langue des emails suit désormais le **destinataire**, pas l'appelant. Remettre `fr` avant de continuer.

#### API-GW-12 — Une valeur de `x-locale` exotique ne casse rien **Gravité** : mineure.

```
for L in "fr-FR" "FR" "en-US,en;q=0.9" "zz" ""; do
  printf '%-18s → ' "$L"
  curl -s -o /dev/null -w '%{http_code}\n' "$BASE/trips/search?limit=1" -H "x-locale: $L"
done
```

**Attendu** : 200 pour les cinq. Aucune valeur d'en-tête ne doit produire un 400 ni un 500 : la résolution est tolérante par contrat.

### 4.6 Format des erreurs et contrat `details.code`

**API-GW-13 — Un `details.code` arrive bien au client **Gravité** : bloquante** (c'est le contrat que tous les clients traduisent).

Trois refus de familles différentes, dont deux portent un **code sans type sûr** — exactement le cas que la correction A146 a rendu fiable.

```
# a) Un conflit métier typé (type "booking", liste sûre)
show -X POST "$BASE/deals/payment-intents" -b shipper.txt -H 'Content-Type: application/json' \
-d "
{\"tripId\": \"\$TRIP_PERKG\", \"product\": \"PARCEL\", \"family\": \"FOOD_DRY_SEALED\", \"sizeClass\":
\"M\", \"weightKg\": 2, \"protection\": \"BASIC\", \"expectedTotalCents\": 1}"

# b) Un code SANS type sûr : la fenêtre sensible fermée
show -X POST "$BASE/auth/me/data-export" -b shipper.txt

# c) Un code SANS type sûr : le code de livraison écrit dans un message
CONV=$(curl -s -b shipper2.txt "$BASE/messages/conversations/by-deal/$(bid bzv-accepted)" | jq -r
.conversation.id)
show -X POST "$BASE/messages/conversations/$CONV/messages" -b shipper2.txt \
-H 'Content-Type: application/json' -d '{"body": "Le code est 742891, garde-le."}'
```

**Attendu** :

|   | Statut | Corps                                                                   |
|---|--------|-------------------------------------------------------------------------|
| a | 409    | <code>details.type = "booking", details.code = "FAMILY_REFUSED"</code>  |
| b | 403    | <code>details.code = "SUDO_REQUIRED", details.windowMinutes = 15</code> |
| c | 400    | <code>details.code = "DELIVERY_CODE_IN_MESSAGE"</code>                  |

**Le point de la fiche : details** est présent dans les trois cas. Un `code` est **public par contrat** ; il est exposé même en production, indépendamment de la liste des `type` sûrs (`otp`, `booking`, `password`, `register`, `locale`, `favorite`, `oauth`, `trip`).

**Contre-épreuve en production (à jouer sur l'environnement de recette configuré en production) :** rejouer (b) et (c) avec `NODE_ENV=production`. Les codes doivent **rester présents**. Leur disparition est une régression **bloquante** : le front ne pourrait plus distinguer ce 403 d'un autre et la fenêtre sensible deviendrait inutilisable.

**API-GW-14 — Les erreurs de formulaire arrivent champ par champ Gravit  : mineure.**

```
show -X POST "$BASE/auth/register" -H 'Content-Type: application/json' -H 'x-locale: fr' \
-d '{"firstName":"A","lastName":"B","email":"pas-un-email","password":"123",
  "termsAccepted":true,"termsVersion":"2026-06-01","privacyVersion":"2026-06-01"}'
```

**Attendu — HTTP 400.** Corps de forme `ErrorResponse` avec un objet `errors` au premier niveau (`{"email": "...", "password": "..."}` ), destin     tre affich   sous chaque champ. Le `message` racine reste g  n  rique et anglais : il n'est pas fait pour l'utilisateur.

**API-GW-15 — Aucune erreur non g  r  e ne remonte au client Gravit  : bloquante** si observ  e.

Cette fiche n'a pas d'appel : c'est une **r  gle de surveillance** valable pendant toute la campagne.

**Attendu :** aucune r  ponse du cahier ne doit avoir la forme `{ "status": "error", "error": "Something went wrong, please try again!" }`  — la forme n   2 du   3.5, qui signe une exception non pr  vue. Chaque occurrence est une anomalie **bloquante**,    consigner avec son `x-correlation-id` (l'  v  nement est aussi remont      Sentry, tagu   du service et de cet identifiant).

```
# Grep de contr  le sur les corps collect  s pendant la campagne
grep -l 'Something went wrong' /tmp/*.json 2>/dev/null
```

## 4.7 S  mantique 401 / 403 / 404

C'est le c  ur de ce cahier : **un refus ne doit rien r  v  ler**.

**API-GW-16 — 401 : pas de session Gravit  : bloquante.**

```
show "$BASE/auth/me"                # aucun cookie
show "$BASE/deals/$DEAL_PENDING"    # aucun cookie
show "$BASE/messages/conversations" # aucun cookie
show "$BASE/me/notifications"       # aucun cookie
```

**Attendu — HTTP 401 pour les quatre, corps plat** `{ "message": "Unauthorized! Token missing." }`  (forme n   3 : ni `status`, ni `details`).

---

## API-GW-17 — 403 : authentifié mais pas partie prenante Gravité : bloquante.

bzv-pending appartient à Aminata (Expéditrice) et Thomas (Voyageur). João n'y est pour rien.

```
show "$BASE/deals/$DEAL_PENDING" -b shipper3.txt
```

**Attendu — HTTP 403**, forme `ErrorResponse`, message d'interdiction. **Surtout : le corps ne contient aucune donnée du deal** — ni statut, ni montant, ni nom, ni destinataire.

**Contre-épreuve indispensable :**

```
show "$BASE/deals/$DEAL_PENDING" -b shipper.txt    # l'Expéditrice → 200
show "$BASE/deals/$DEAL_PENDING" -b carrier.txt    # le Voyageur   → 200
```

---

## API-GW-18 — 404 : la ressource n'existe pas, ou n'est pas visible — sans distinction Gravité : bloquante.

```
# a) Un identifiant bien formé mais inexistant
show "$BASE/deals/0000000000000000000000000000" -b shipper.txt
# b) Un identifiant mal formé
show "$BASE/deals/pas-un-objectid" -b shipper.txt
# c) Un trajet public inexistant
show "$BASE/trips/0000000000000000000000000000/public"
# d) Un profil public inexistant
show "$BASE/users/ce-slug-nexiste-pas/public"
# e) Un lien de suivi inventé
show "$BASE/track/jeton-invente-par-le-testeur"
```

**Attendu :** (a) **404** · (b) **400** (identifiant malformé) · (c) **404** au format `PublicNotFound` · (d) **404** · (e) **404**.

**La règle à faire tenir :** un 404 ne dit **jamais** si la ressource existe pour quelqu'un d'autre. On rejoue donc (c) et (d) sur des cibles qui **existent mais sont invisibles** — un trajet masqué par Yamba, un profil dont `profilePublic` est faux, un lien de suivi révoqué — et on exige **le même 404, avec le même corps**. Toute différence observable entre « inexistant » et « invisible » (statut, message, temps de réponse notablement différent) est une anomalie **bloquante** : elle permet d'énumérer les ressources.

---

## API-GW-19 — 409 : un conflit métier, jamais un 400 ni un 500 Gravité : majeure.

```
show -X POST "$BASE/deals/$DEAL_COMPLETED/accept" -b carrier.txt \
-H 'Content-Type: application/json' -d '{"charterAccepted":true}'
```

**Attendu — HTTP 409**, `details.type: "booking"`, `details.code: "TRANSITION_NOT_ALLOWED"`, et un `details.reason` qui porte le motif de la machine à états.

**Réaction attendue d'un client :** relire `GET /deals/{id}` et ne proposer que ce que `allowedActions` autorise. Le cahier vérifie donc, juste après :

```
curl -s -b carrier.txt "$BASE/deals/$DEAL_COMPLETED" | jq '{status: .deal.status,
allowedActions: .deal.allowedActions}'
```

**Attendu :** `status: "COMPLETED"`, et `allowedActions` **ne contient pas** `accept`.

---

## 4.8 Pagination par curseur

Il n'y a **jamais** de pagination par numéro de page. Trois formes coexistent.

### API-GW-20 — La recherche pagine par curseur, sans doublon ni trou Gravité : majeure.

```
P1=$(curl -s "$BASE/trips/search?limit=2&sort=earliest")
echo "$P1" | jq '{ids: [.trips[].id], nextCursor, totalCount}'
C=$(echo "$P1" | jq -r .nextCursor)
P2=$(curl -s "$BASE/trips/search?limit=2&sort=earliest&cursor=$C")
echo "$P2" | jq '{ids: [.trips[].id], nextCursor}'
# Aucun identifiant commun entre les deux pages :
comm -12 <(echo "$P1" | jq -r '.trips[].id' | sort) <(echo "$P2" | jq -r '.trips[].id' | sort)
```

**Attendu** : deux pages disjointes (la dernière commande n'affiche **rien**), un `totalCount` stable, et un `nextCursor` qui vaut **null en dernière page**. L'enveloppe de la recherche est **sans champ success** — c'est voulu et documenté ; ne pas le signaler.

### API-GW-21 — Le fil de conversation pagine vers le passé Gravité : mineure · **Prérequis** : le fil seedé du deal `bzv-accepted`.

```
CONV=$(curl -s -b shipper2.txt "$BASE/messages/conversations/by-deal/$(bid bzv-accepted)" | jq -r .conversation.id)
curl -s -b shipper2.txt "$BASE/messages/conversations/$CONV" | jq '{n: (.messages|length), first: .messages[0].id, nextCursor}'
OLDEST=$(curl -s -b shipper2.txt "$BASE/messages/conversations/$CONV" | jq -r '.messages[0].id')
curl -s -b shipper2.txt "$BASE/messages/conversations/$CONV?cursor=$OLDEST" | jq '{n: (.messages|length)}'
```

**Attendu** : le fil est rendu **du plus ancien au plus récent** ; le curseur est l'identifiant du **plus ancien message déjà chargé** et charge les messages **antérieurs**. Sur le jeu d'essai (2 messages), la seconde page est vide — c'est le résultat correct.

### API-GW-22 — Les listes sans curseur sont bornées côté serveur Gravité : mineure.

```
curl -s -b shipper.txt "$BASE/me/notifications" | jq '{n: (.notifications|length), unreadCount}'
```

**Attendu** : au plus **50** éléments, plus un `unreadCount`. Aucune requête client ne doit pouvoir faire dépasser cette borne (essayer `?limit=1000` : le paramètre est ignoré).

## 5. Un chapitre par service

### 5.1 auth-service — comptes, sessions, profil, RGPD

L'auth-service est le seul service qui reçoit le **chemin complet, préfixe `/api` compris** : ses routeurs sont montés sous `/api`. C'est aussi la destination par défaut de la passerelle (tout ce qui n'a pas de préfixe plus spécifique).

### 5.1.1 Inscription et code à usage unique

---

**API-AUTH-01 — Démarrer une inscription Gravit  : majeure** · **Pr requis** : une adresse email neuve ; Mailpit ouvert si l'on veut lire le code.

```
show -X POST "$BASE/auth/register" -H 'Content-Type: application/json' -d '{
  "firstName":"Recette","lastName":"Api","email":"recette.api.001@seed.yamba.dev",
  "password":"Kinshasa-2026!","termsAccepted":true,
  "termsVersion":"2026-06-01","privacyVersion":"2026-06-01"}
```

**Attendu — HTTP 200**, corps { "message": "OTP sent to your email...", "verificationToken": "..." }.

**Effet de bord — le point   prouver : rien n'est  crit en base   cette  tape.** La demande vit **10 minutes en Redis**. Le v rifier : le compte n'appara t ni dans une connexion (POST /auth/login r pond 401), ni dans la liste admin des membres.

---

**API-AUTH-02 — Le consentement est obligatoire Gravit  : majeure.**

```
show -X POST "$BASE/auth/register" -H 'Content-Type: application/json' -d '{
  "firstName":"Recette","lastName":"Api","email":"recette.api.002@seed.yamba.dev",
  "password":"Kinshasa-2026!","termsAccepted":false,
  "termsVersion":"2026-06-01","privacyVersion":"2026-06-01"}
```

**Attendu — HTTP 400**. Le consentement (acceptation + versions des CGU et de la politique de confidentialit ) est requis par le contrat : sans lui, aucune inscription ne d marre. Rejouer en retirant compl tement `termsVersion` : **400**  galement.

---

**API-AUTH-03 — Un email d j  pris r pond 409 Gravit  : majeure.**

```
show -X POST "$BASE/auth/register" -H 'Content-Type: application/json' -d '{
  "firstName":"Doublon","lastName":"Test","email":"aminata.shipper@seed.yamba.dev",
  "password":"Kinshasa-2026!","termsAccepted":true,
  "termsVersion":"2026-06-01","privacyVersion":"2026-06-01"}
```

**Attendu — HTTP 409**. R action attendue d'un client : proposer la connexion ou le mot de passe oubli , jamais « r essayez ».

---

**API-AUTH-04 — Un mot de passe trop faible est refus  avec sa r gle Gravit  : mineure.**

```
show -X POST "$BASE/auth/register" -H 'Content-Type: application/json' -d '{
  "firstName":"Recette","lastName":"Api","email":"recette.api.003@seed.yamba.dev",
  "password":"recette","termsAccepted":true,
  "termsVersion":"2026-06-01","privacyVersion":"2026-06-01"}
```

**Attendu — HTTP 400**, `details.type: "password"` et un code de r gle ( `PASSWORD_CONTAINS_PERSONAL_INFO` quand le mot de passe reprend le nom ou l'email, sinon un code de force). Le client affiche la r gle sous le champ, pas le message anglais.

---

**API-AUTH-05 — Un code faux décrémente un compteur visible Gravit  : majeure · Pr requis : le verificationToken de la fiche API-AUTH-01.**

```
TOKEN='<le verificationToken>'
show -X POST "$BASE/auth/register/verify" -H 'Content-Type: application/json' \
-d '{"verificationToken":"$TOKEN","otp":"000000"}'
```

**Attendu — HTTP 401.** Corps :

```
{ "status": "error",
  "message": "Incorrect code. 4 attempt(s) left before this code is invalidated.",
  "details": { "type": "otp", "code": "OTP_INCORRECT", "attemptsLeft": 4, "locked": false } }
```

**Le point de la fiche :** le client construit son message depuis code, attemptsLeft, lockUntilSeconds et otpInvalidated — **jamais** depuis message. Rejouer quatre fois et observer la descente 4 → 3 → 2 → 1, puis le code invalid  (OTP\_INVALIDATED) et le verrou (OTP\_LOCKED, **429**, avec lockUntilSeconds). Les paliers sont **1 minute, puis 30 minutes, puis 24 heures** ; un email d'alerte de s curit  part d s le deuxi me palier.

**Effet de bord :** le compte n'est toujours pas cr  . Redemander un code : POST /auth/register/resent { verificationToken } (6 envois par heure au maximum) ; abandonner : POST /auth/register/cancel { verificationToken }.

---

**API-AUTH-06 — Le bon code cr   le compte, sans ouvrir de session Gravit  : majeure · Pr requis : le code lu dans Mailpit.**

```
show -X POST "$BASE/auth/register/verify" -c neuf.txt -H 'Content-Type: application/json' \
-d '{"verificationToken":"$TOKEN","otp":"482913"}'
grep -c access_token neuf.txt
```

**Attendu — HTTP 201,** corps { "success": true, "message": "Account created successfully" }.

**Le point   prouver :** grep -c access\_token neuf.txt renvoie **0**. **Aucune session n'est ouverte   la v rification :** le membre se connecte ensuite. Un client qui supposerait le contraire encha nerait sur un 401.

**Effet de bord :** le document User et le ConsentLog sont  crits ; l'email de bienvenue part.

---

### 5.1.2 Connexion, rotation, sessions

---

**API-AUTH-07 — Connexion : les jetons sont dans les cookies, pas dans le corps Gravit  : bloquante.**

```
curl -s -D /tmp/h.txt -c shipper.txt -X POST "$BASE/auth/login" \
-H 'Content-Type: application/json' -H 'x-locale: fr' \
-d '{"email":"aminata.shipper@seed.yamba.dev","password":"Yamba-Dev-2026!","rememberMe":true}'
| jq .
grep -i 'set-cookie' /tmp/h.txt
```

**Attendu — HTTP 200.** Corps : { "message": "Login successful!", "user": { id, email, firstName, lastName, roles } }.

**Ce que le corps ne doit PAS contenir :** ni accessToken, ni refreshToken, ni passwordHash. Les jetons voyagent **uniquement** dans deux en-t tes Set-Cookie, tous deux HttpOnly : access\_token (15 minutes) et

refresh\_token (cookie de session, ou **30 jours** avec rememberMe: true).

---

#### API-AUTH-08 — Identifiants invalides et compte suspendu : le même 401 Gravité : bloquante.

```
show -X POST "$BASE/auth/login" -H 'Content-Type: application/json' \
  -d '{"email":"aminata.shipper@seed.yamba.dev","password":"MauvaisMotDePasse-1!"}'
show -X POST "$BASE/auth/login" -H 'Content-Type: application/json' \
  -d '{"email":"personne.inexistante@seed.yamba.dev","password":"Yamba-Dev-2026!"}'
```

**Attendu — HTTP 401 pour les deux, avec le même corps.** Aucune différence observable entre « mot de passe faux », « compte inconnu » et « compte suspendu » : la connexion ne dit jamais lequel des trois. Toute divergence (statut, message, champ supplémentaire) est **bloquante** : elle permet d'énumérer les comptes.

---

#### API-AUTH-09 — Le membre courant, sans mot de passe Gravité : bloquante.

```
curl -s -b shipper.txt "$BASE/auth/me" | jq '{success, email: .user.email, roles,
  fuites: [(.user|keys[]) | select(test("passwordHash|totpSecret|deliveryCode|Encrypted"))]}'
```

**Attendu — HTTP 200**, success: true, l'email attendu, roles: ["SHIPPER"] ou équivalent, et surtout **fuites: []**. Le document User est renvoyé **sans passwordHash** ni aucun secret.

---

#### API-AUTH-10 — La rotation de session révoque l'ancien jeton Gravité : majeure.

```
cp shipper.txt avant.txt # photographie de la session
show -X POST "$BASE/auth/refresh" -b shipper.txt -c shipper.txt
show "$BASE/auth/me" -b shipper.txt # la NOUVELLE session → 200
show -X POST "$BASE/auth/refresh" -b avant.txt # l'ANCIENNE → doit échouer
```

**Attendu :** le premier rafraîchissement répond **200** { success, message } — **et rien d'autre : les nouveaux jetons sont dans les cookies de la réponse.** GET /auth/me avec le pot mis à jour répond **200**. Le rafraîchissement rejoué avec l'ancien pot répond **401** (« Session expired or invalid ») : le jti précédent a été révoqué.

**Le point à prouver :** un jeton de rafraîchissement ne sert **qu'une fois**. Si l'ancien pot fonctionne encore, c'est une anomalie **bloquante** (rejeu de session).

---

#### API-AUTH-11 — Mes appareils : lister et couper Gravité : majeure.

```
login aminata.shipper@seed.yamba.dev autre-appareil.txt # une seconde session du même membre
curl -s -b shipper.txt "$BASE/auth/me/sessions" | jq '[.sessions[] | {jti, device, ip,
current}]'
JTI=$(curl -s -b shipper.txt "$BASE/auth/me/sessions" | jq -r '.sessions[] |
select(.current==false) | .jti' | head -1)
show -X DELETE "$BASE/auth/me/sessions/$JTI" -b shipper.txt
show "$BASE/auth/me" -b autre-appareil.txt
```

**Attendu :** la liste porte **deux** sessions, dont **une seule** avec current: true, chacune avec son device (navigateur + système, ou « Appareil inconnu »), son ip et ses dates. La suppression répond **200**, et la session coupée répond ensuite **401** — la révocation est immédiate et côté serveur.

**Variante :** DELETE /api/auth/me/sessions (sans jti) coupe **toutes les autres** sessions et laisse la courante active.

---

**API-AUTH-12 — Changer de mot de passe révoque les autres sessions** Gravit  : majeure · **Pr requis :** fen tre sensible ouverte (§5.1.5).

```
login aminata.shipper@seed.yamba.dev autre-appareil.txt
show -X POST "$BASE/auth/me/password" -b shipper.txt -H 'Content-Type: application/json' \
  -d '{"currentPassword":"Yamba-Dev-2026!","newPassword":"Yamba-Recette-2026!"}'
show "$BASE/auth/me" -b autre-appareil.txt
```

**Attendu :** 200 sur le changement (une fois la fen tre sensible ouverte), puis 401 sur l'autre appareil. Le changement d'email produit le m me effet.

**Effet de bord :** remettre le mot de passe d'origine, ou rejouer le jeu d'essai avant la suite du cahier — sinon toutes les fiches suivantes  chouent   la connexion.

---

**API-AUTH-13 — Se d connecter efface les cookies et r voque la session** Gravit  : majeure.

```
show -X POST "$BASE/auth/logout" -b jetable.txt -c jetable.txt
show "$BASE/auth/me" -b jetable.txt
```

**Attendu :** 200 puis 401. Les cookies sont effac s c t  client (le pot ne porte plus de valeur utile) **et** le jti est supprim  c t  serveur — les deux, pas seulement le premier.

---

### 5.1.3 Mot de passe oubli 

---

**API-AUTH-14 — « Mot de passe oubli  » ne r v le jamais l'existence du compte** Gravit  : bloquante.

```
show -X POST "$BASE/auth/password/forgot" -H 'Content-Type: application/json' \
  -d '{"email":"aminata.shipper@seed.yamba.dev"}'
show -X POST "$BASE/auth/password/forgot" -H 'Content-Type: application/json' \
  -d '{"email":"personne.inexistante@seed.yamba.dev"}'
```

**Attendu — HTTP 200 pour les deux, avec le m me corps.** Un email ne part que dans le premier cas, mais la r ponse HTTP ne le dit pas. Toute diff rence est **bloquante**.

**Suite du parcours (m mes r gles d'OTP que l'inscription) :** POST /auth/password/verify { email, otp } → jeton de r initialisation, puis POST /auth/password/reset { ... } ; POST /auth/password/resent renvoie un code.

---

### 5.1.4 Profil, avatar, pr f rences, profils publics

---

**API-AUTH-15 — Lire et modifier son profil  ditable** Gravit  : mineure.

```
curl -s -b shipper.txt "$BASE/auth/me/profile" | jq '{firstName, lastName, publicSlug, profilePublic, showCity}'
show -X PATCH "$BASE/auth/me/profile" -b shipper.txt -H 'Content-Type: application/json' \
  -d '{"displayName":"Aminata D.,"bio":"Paris ≠ Brazzaville chaque mois.,"profilePublic":true,"showCity":false}'
```

**Attendu — 200** pour les deux, et **publicSlug** identique avant et après. Le slug public **ne change jamais** : c'est l'identifiant stable d'un membre dans les URL publiques et dans les signalements.

---

**API-AUTH-16 — Une date de naissance invalide est refusée champ par champ Gravité** : mineure.

```
show -X PATCH "$BASE/auth/me/profile" -b shipper.txt -H 'Content-Type: application/json' \
  -d '{"birthDate":"2030-01-01"}'
show -X PATCH "$BASE/auth/me/profile" -b shipper.txt -H 'Content-Type: application/json' \
  -d '{"birthDate":"2020-01-01"}'
```

**Attendu — HTTP 400** avec un code de règle dans `details.errors` : **IN\_THE\_FUTURE** pour la première, **T00\_YOUNG** pour la seconde (les autres codes possibles étant **INVALID\_DATE** et **NO\_CARRIER\_PAGE**).

---

**API-AUTH-17 — L'avatar : aucun octet ne passe par l'API Gravité** : majeure.

```
# 1. La signature (trip-service, ~30 min de validité)
curl -s -b shipper.txt "$BASE/uploads/imagekit-auth" | jq '{token: (.token|length), expire, signature: (.signature|length), publicKey, urlEndpoint}'

# 2. Le téléversement direct chez ImageKit — HORS API Yamba
curl -s -X POST "https://upload.imagekit.io/api/v1/files/upload" \
  -F "file=@portrait.jpg" -F "fileName=portrait.jpg" -F "folder=/avatars" \
  -F "publicKey=..." -F "signature=..." -F "expire=..." -F "token=..." | jq '{fileId, url}'

# 3. Déclarer l'avatar
show -X POST "$BASE/auth/me/avatar" -b shipper.txt -H 'Content-Type: application/json' \
  -d '{"fileId":"66g6...", "url":"https://ik.imagekit.io/yamba/avatars/portrait_abc.jpg"}'
```

**Attendu** : (1) **200** avec `token`, `expire`, `signature` non vides ; `publicKey` et `urlEndpoint` **absents si l'environnement ImageKit n'est pas configuré** — dans ce cas les étapes 2 et 3 sont ➡. (3) **200** avec le profil à jour.

**La garde à prouver — une URL étrangère est refusée** :

```
show -X POST "$BASE/auth/me/avatar" -b shipper.txt -H 'Content-Type: application/json' \
  -d '{"fileId":"x","url":"https://exemple-malveillant.test/photo.jpg"}'
```

**Attendu — HTTP 400**. L'URL doit appartenir au domaine ImageKit configuré. Un 200 ici serait une anomalie **majeure** : la plateforme afficherait des images arbitraires.

**Effet de bord** : déclarer un nouvel avatar **supprime l'ancien fichier** chez ImageKit. `DELETE /api/auth/me/avatar` le retire.

---

**API-AUTH-18 — Le profil public : visible, masqué, effacé Gravité** : bloquante.

```
SLUG=$(curl -s -b shipper.txt "$BASE/auth/me/profile" | jq -r .publicSlug)
show "$BASE/users/$SLUG/public" # sans session
show "$BASE/users/$SLUG/public" -b shipper3.txt # avec la session d'un tiers
show "$BASE/users/$SLUG/public" -b shipper.txt # avec sa PROPRE session
curl -s "$BASE/users/$SLUG/public/reviews" | jq '{n: (.reviews|length)}'
curl -s "$BASE/users/$SLUG/public/trips" | jq '{n: (.trips|length)}'
```

**Attendu :** la route passe par l'authentification **optionnelle** — donc **jamais 401**, ni sans jeton ni avec un jeton expiré. Réponse **200** quand le profil est public.

**La règle à faire tenir :** après `PATCH /auth/me/profile {"profilePublic": false}`, un tiers reçoit **404** (pas 403), tandis que le **propriétaire** reçoit **200 avec hidden: true**. C'est la seule asymétrie autorisée, et elle est réservée au propriétaire.

---

#### API-AUTH-19 — S'abonner à un membre Gravité : mineure.

```
show -X POST "$BASE/users/$SLUG/follow" -b shipper3.txt
curl -s -b shipper3.txt "$BASE/me/following" | jq '[.following[].publicSlug]'
show -X DELETE "$BASE/users/$SLUG/follow" -b shipper3.txt
show -X POST "$BASE/users/$SLUG/follow" -b shipper.txt # s'abonner à SOI-MÊME
```

**Attendu :** abonnement **200/201**, présence dans la liste, désabonnement **200**, et un **refus** sur soi-même. Le geste est idempotent : rejouer l'abonnement ne crée pas de doublon dans `/me/following`.

---

#### API-AUTH-20 — Les préférences et le consentement de mesure d'audience Gravité : mineure.

```
show -X PATCH "$BASE/auth/me/preferences" -b shipper.txt -H 'Content-Type: application/json' \
-d '{"analyticsOptIn":true}'
show -X PATCH "$BASE/auth/me/preferences" -b shipper.txt -H 'Content-Type: application/json' \
-d '{"messagingReminderEmails":false}'
```

**Attendu — 200** pour les deux. **Effet de bord à vérifier :** un changement d'`analyticsOptIn` écrit une ligne `ConsentLog` de type `COOKIES` — le consentement est **tracé**, pas seulement stocké. Avec `messagingReminderEmails: false`, la relance des messages non lus ne part plus.

---

#### 5.1.5 La fenêtre sensible (sudo)

Cinq gestes l'exigent : changer son mot de passe, changer son email, ouvrir le tableau de bord Stripe, exporter ses données, effacer son compte.

---

#### API-AUTH-21 — Un geste sensible sans fenêtre est refusé, avec son code Gravité : bloquante.

```
show "$BASE/auth/me/sudo" -b shipper.txt
show -X POST "$BASE/auth/me/data-export" -b shipper.txt
show -X POST "$BASE/auth/me/erasure" -b shipper.txt -H 'Content-Type: application/json' \
-d '{"confirmation":"SUPPRIMER"}'
show -X POST "$BASE/carrier/stripe/dashboard-link" -b carrier.txt
```

**Attendu :** l'état répond **200** `{ "active": false, "expiresAt": null }`, et les trois gestes répondent **403** avec

```
{ "status": "error", "message": "Sudo required: confirm with the code sent by email.",  
  "details": { "code": "SUDO_REQUIRED", "windowMinutes": 15 } }
```

**Cas particulier à connaître :** la fenêtre est liée au `jti` lu dans le cookie `refresh_token`. Un appel fait **avec le seul cookie d'accès** (sans cookie de rafraîchissement) répond aussi **403**, mais **sans `details.code`** — le serveur n'a pas de session à laquelle attacher la fenêtre. Ce n'est pas une anomalie ; c'est à connaître pour ne pas conclure trop vite en observant un 403 nu.

---

**API-AUTH-22 — Ouvrir la fenêtre puis rejouer exactement le même appel Gravité : bloquante · Prérequis :** Mailpit pour lire le code.

```
show -X POST "$BASE/auth/me/sudo/request" -b shipper.txt  
# ... lire le code à six chiffres dans Mailpit ...  
show -X POST "$BASE/auth/me/sudo/verify" -b shipper.txt -H 'Content-Type: application/json' \  
  -d '{"code":"391045"}'  
show "$BASE/auth/me/sudo" -b shipper.txt  
show -X POST "$BASE/auth/me/data-export" -b shipper.txt -o /tmp/export.json -D /tmp/exp-  
headers.txt
```

**Attendu :** la demande **200**, la vérification **200** { "active": true, "expiresAt": "...+15 min" }, l'état **200** active: true, puis l'export **200**.

**Effet de bord :** l'en-tête `Content-Disposition: attachment` est posé sur l'export, et une ligne `DataRequest` de type `EXPORT` est journalisée.

---

**API-AUTH-23 — La fenêtre est liée à UNE session, pas au membre Gravité : bloquante · Prérequis :** fenêtre ouverte sur `shipper.txt` (fiche précédente).

```
login aminata.shipper@seed.yamba.dev autre-appareil.txt  
show "$BASE/auth/me/sudo" -b autre-appareil.txt  
show -X POST "$BASE/auth/me/data-export" -b autre-appareil.txt
```

**Attendu :** l'autre session voit `active: false` et reçoit **403 SUDO\_REQUIRED** sur l'export. Une fenêtre ouverte sur un appareil **n'ouvre rien ailleurs**. Le contraire serait une anomalie **bloquante** : un appareil compromis hériterait des gestes sensibles d'un appareil de confiance.

---

**API-AUTH-24 — L'export ne contient pas les données des autres Gravité : bloquante · Prérequis :** un export obtenu (API-AUTH-22).

```
jq 'keys' /tmp/export.json  
jq '[paths(scalars) as $p | {p: ($p|join(".")), v: getpath($p)}]  
  | map(select(.v|tostring|test("742891|@seed\\.yamba\\.dev";"i")))' /tmp/export.json | head  
-40
```

**Attendu :** l'export porte les rubriques du membre (`profile`, `preferences`, `consents`, `trips`, `bookings`, `reviewsGiven`, `reviewsReceived`, `messages`, `meetups`, `phoneReveals`, `savedRoutes`, `favorites`, `following`, `notifications`, `reportsMade`, `dataRequests`).

**Ce qu'il ne doit JAMAIS contenir :** le **code de livraison** d'un deal, les **coordonnées de l'autre partie** (email, téléphone), les **signalements visant** le membre, les **dossiers de médiation**. La seconde commande ne doit remonter que la **propre adresse** du membre exportant. Toute autre occurrence est une anomalie **bloquante**.

---

**API-AUTH-25 — Un export trop fréquent est refusé avec sa date de réouverture Gravité : mineure.**

```
show -X POST "$BASE/auth/me/data-export" -b shipper.txt
```

**Attendu — HTTP 400** avec `details.code: "EXPORT_RATE_LIMITED"` et `details.nextAt`, quand un export a déjà été produit dans l'intervalle minimal. Le client propose de réutiliser le fichier déjà téléchargé et affiche `nextAt`.

---

### 5.1.6 Effacement du compte

---

**API-AUTH-26 — Ce qui bloque l'effacement se lit à l'avance Gravité : majeure.**

```
curl -s -b shipper.txt "$BASE/auth/me/erasure/blockers" | jq .
```

**Attendu — HTTP 200**, corps `{ "blockers": [...], "counts": {...} }`. Aminata porte des deals vivants dans le jeu d'essai : on attend au moins `ACTIVE_DEAL`. La liste est **fermée** : `ACTIVE_DEAL`, `PENDING_REQUEST`, `PAYOUT_PENDING`, `RETENTION_HELD`, `PUBLISHED_TRIP`, `ADMIN_ACCOUNT`. Un client traduit chacun et propose l'action qui le lève.

---

**API-AUTH-27 — L'effacement bloqué répond 409 avec ses bloqueurs Gravité : majeure · Prérequis : fenêtre sensible ouverte.**

```
show -X POST "$BASE/auth/me/erasure" -b shipper.txt -H 'Content-Type: application/json' \
-d '{"confirmation":"SUPPRIMER"}
```

**Attendu — HTTP 409**, corps `EraseurBlockedResponse : { "code": "ERASURE_BLOCKED", "blockers": ["ACTIVE_DEAL"], "counts": { "ACTIVE_DEAL": 1 } }`.

Noter que **code est ici à la racine**, pas sous `details` : c'est une réponse typée renvoyée hors intergiciel d'erreurs, comme le `PublicNotFound` du trip-service.

**Le mot de confirmation** : rejouer avec `{"confirmation":"oui"}` doit répondre **400**. Le geste irréversible exige le mot exact.

---

**API-AUTH-28 — L'effacement effectif anonymise sans supprimer l'histoire Gravité : bloquante · Prérequis : un compte sans deal vivant (créé par API-AUTH-06), fenêtre sensible ouverte.**

```
show -X POST "$BASE/auth/me/erasure" -b neuf.txt -H 'Content-Type: application/json' \
-d '{"confirmation":"SUPPRIMER"}'
show "$BASE/auth/me" -b neuf.txt
```

**Attendu : 200** `{ "success": true, "erased": true }`, puis **401** avec `code: "ACCOUNT_DELETED"` — la session est terminée immédiatement.

**Effet de bord à vérifier en base** : le document `User` existe toujours mais est anonymisé champ par champ ; les champs uniques deviennent `erased+<id>@anonymised.invalid` et `deleted-<id>` — **jamais null** (deux `null` collisionnent sur un index unique Mongo) ; `isDeleted` est vrai ; les deals, litiges, avis et messages sont **conservés** (obligations comptables et de médiation) ; une ligne `DataRequest` est écrite.

---

### 5.1.7 Alertes de route

**API-AUTH-29 — Créer, lister, modifier, prolonger, supprimer une alerte Gravité : mineure.**

```
show -X POST "$BASE/saved-routes" -b shipper.txt -H 'Content-Type: application/json' -d '{
  "originCity":"Paris","originCountry":"France","originCountryCode":"FR",
  "originPlaceId":"ChIJD7fiBh9u5kcRYJSMaMOCCwQ","originLat":48.8566,"originLng":2.3522,
  "destinationCity":"Kinshasa","destinationCountry":"RD Congo","destinationCountryCode":"CD",
  "destinationLat":-4.4419,"destinationLng":15.2663,
  "earliestDate":"2026-10-01","latestDate":"2026-12-31",
  "emailEnabled":true,"inAppEnabled":true,"includeNearby":true}'
SR=$(curl -s -b shipper.txt "$BASE/saved-routes" | jq -r '.savedRoutes[0].id')
show -X PATCH "$BASE/saved-routes/$SR" -b shipper.txt -H 'Content-Type: application/json' -d
'{"emailEnabled":false}'
show -X POST "$BASE/saved-routes/$SR/extend" -b shipper.txt
show -X DELETE "$BASE/saved-routes/$SR" -b shipper.txt
```

**Attendu :** création **201** avec `expiresAt` à **six mois** et `isActive`: `true` ; lecture **200** ; modification **200** ; prolongation **200** avec un `expiresAt` repoussé ; suppression **200**.

**API-AUTH-30 — Les gardes des alertes de route tiennent côté serveur Gravité : majeure.**

```
# a) Même ville et même pays des deux côtés
show -X POST "$BASE/saved-routes" -b shipper.txt -H 'Content-Type: application/json' -d '{
  "originCity":"Paris","originCountry":"France","originCountryCode":"FR",
  "destinationCity":"Paris","destinationCountry":"France","destinationCountryCode":"FR"}'
# b) La vingt-et-unième alerte
for i in $(seq 1 21); do
  curl -s -o /dev/null -w '%{http_code} ' -X POST "$BASE/saved-routes" -b shipper.txt \
    -H 'Content-Type: application/json' -d "
    {\"originCity\": \"Ville$i\", \"originCountry\": \"France\", \"originCountryCode\": \"FR\", \"destinat
    ionCity\": \"Kinshasa\", \"destinationCountry\": \"RD Congo\", \"destinationCountryCode\": \"CD\"}"
done; echo
```

**Attendu :** (a) **400** — un corridor à origine et destination identiques n'a pas de sens ; (b) une série de **201** puis un **409** au dépassement de **20** alertes.

**Effet de bord :** nettoyer les alertes créées avant de continuer (`DELETE /api/saved-routes/{id}` sur chacune), ou rejouer le jeu d'essai.

### 5.1.8 Signalements

**API-AUTH-31 — Signaler un trajet Gravité : majeure.**

```
show -X POST "$BASE/reports" -b shipper.txt -H 'Content-Type: application/json' -d "{
  \"targetType\": \"TRIP\", \"targetRef\": \"$TRIP_PERKG\", \"reason\": \"SCAM\",
  \"details\": \"Le prix affiché change après contact et un paiement hors plateforme est
  demandé.\"}"
```

**Attendu — HTTP 201**, corps `{ "reportId": "...", "createdAt": "..." }`.

**Effet de bord** : un accusé de réception part au signaleur. **La cible n'apprend jamais qui l'a signalée, ni même qu'elle l'a été.** Le dossier apparaît dans la file du support (GET /api/admin/reports), sans sanction automatique.

---

### API-AUTH-32 — Les gardes du signalement Gravité : majeure.

```
# a) Signaler SON PROPRE trajet (Thomas est le Voyageur de bzv-perkg)
show -X POST "$BASE/reports" -b carrier.txt -H 'Content-Type: application/json' \
  -d '{"targetType":"TRIP","targetRef":"$TRIP_PERKG","reason":"SCAM","details":""}'
# b) Un motif hors de la liste de CE type de cible
show -X POST "$BASE/reports" -b shipper.txt -H 'Content-Type: application/json' \
  -d '{"targetType":"TRIP","targetRef":"$TRIP_PERKG","reason":"IMPERSONATION","details":""}'
# c) Une cible invisible
show -X POST "$BASE/reports" -b shipper.txt -H 'Content-Type: application/json' \
  -d '{"targetType":"USER","targetRef":"slug-inexistant","reason":"SCAM","details":""}'
# d) Le même signalement une seconde fois
show -X POST "$BASE/reports" -b shipper.txt -H 'Content-Type: application/json' \
  -d '{"targetType":"TRIP","targetRef":"$TRIP_PERKG","reason":"SCAM","details":"Doublon volontaire de recette."}'
```

**Attendu** : (a) 400 OWN\_TARGET · (b) 400 REASON\_NOT\_ALLOWED · (c) 404 · (d) 409 (doublon ouvert du même auteur).

**Les listes de motifs sont fermées par cible** : trajet → ILLEGAL\_CONTENT, SCAM, INAPPROPRIATE, OTHER ;  
membre → SCAM, INAPPROPRIATE, IMPERSONATION, OTHER. Un client ne propose que les motifs de la cible.

**Une garde à ne pas oublier** : targetRef d'un membre est son slug public, jamais son identifiant — le DTO public d'un membre ne porte pas d'identifiant.

---

### 5.1.9 Onboarding Voyageur

---

**API-AUTH-33 — Les trois étapes de l'onboarding Gravité** : majeure · **Prérequis** : Stripe configuré, sinon ►► pour les étapes 2 et 3.

```
show -X POST "$BASE/carrier/onboarding/profile" -b shipper.txt -H 'Content-Type: application/json' -d '{
  "name":"Aminata D.,"bio":"Paris ≠ Brazzaville chaque mois.,"phoneE164":"+33612345611",
  "address":{"formattedAddress":"12 rue de la Paix, 75002 Paris, France","city":"Paris",
    "country":"France","countryCode":"FR","lat":48.8687,"lng":2.3312}}'
show -X POST "$BASE/carrier/onboarding/stripe" -b shipper.txt
show "$BASE/carrier/onboarding/stripe/status" -b shipper.txt
show -X POST "$BASE/carrier/onboarding/complete" -b shipper.txt
```

**Attendu** : (1) 200 avec la page Voyageur et stripeOnboardingComplete: false ; (2) 200 avec une url Stripe à usage unique, jamais stockée ; (3) 200 avec status ∈ not\_started | pending | complete et les drapeaux chargesEnabled, payoutsEnabled, detailsSubmitted ; (4) 200 — le rôle CARRIER est accordé.

**Effet de bord** : après (4), GET /auth/me porte CARRIER dans roles. Les drapeaux Stripe sont ensuite maintenus par le webhook account.updated sans repasser par l'onboarding.

---

**API-AUTH-34 — Le tableau de bord Stripe sans compte Connect Gravit  : mineure · Pr requis :** fen tre sensible ouverte, membre sans compte Stripe.

```
show -X POST "$BASE/carrier/stripe/dashboard-link" -b shipper3.txt
```

**Attendu — HTTP 409** avec `details.code: "STRIPE_ACCOUNT_MISSING"` (une fois la fen tre sensible ouverte ; sinon **403 SUDO\_REQUIRED** , ce qui est l'ordre correct : la fen tre d'abord).

## 5.2 trip-service — recherche, trajets, cycle de vie, t l versement

**Note s mantique   lire avant de commencer.** Les contr leurs historiques du trip-service l vent une erreur de validation (**400**) pour presque tout, y compris « Trip not found. » et « Unauthorized. » (non-propri taire). Ce n'est **pas** une anomalie de recette : c'est un  cart connu, catalogu , dont la correction est une porte ouverte ( `fix/error-semantics` ). Le testeur attend donc **400** l  o  403 ou 404 seraient plus justes, et le consigne en **mineur** s'il veut le rappeler. Les exceptions d j  corrig es sont not es fiche par fiche : le favori d'un trajet inexistant est un vrai **404**, celui de son propre trajet un vrai **403**, l'annulation d'un trajet portant un deal vivant un vrai **409**.

### 5.2.1 Recherche et facettes

**API-TRIP-01 — La recherche est publique et personnalis e si connect  Gravit  : majeure.**

```
show "$BASE/trips/search?from=Paris&to=Brazzaville&sort=earliest&locale=fr&limit=5"
curl -s -b shipper.txt "$BASE/trips/search?from=Paris&limit=5" | jq '[.trips[] | {id,
isFavorite}]'
```

**Attendu — HTTP 200 dans les deux cas** (jamais 401 : la route passe par l'authentification optionnelle). L'enveloppe est `{ trips, nextCursor, totalCount }`, **sans champ success** — c'est voulu. Sans session, `isFavorite` vaut `false` partout ; avec session, il refl te les favoris r els du membre.

**Champs d'une carte de r sultat :** `id`, `fromCity`, `toCity`, `travelDate` (format e serveur), `departureTime`, `pricePerKg`, `remainingKg`, `minPrice`, `currency` (**un symbole**), `transportMode`, `allowedCategories`, `travelerFirstName`, `travelerLastName` (initiale), `rating`, `reviewCount`, `isFavorite`, `viewsCount`.

**API-TRIP-02 — Les filtres durs ne sont pas n gociables Gravit  : bloquante.**

```
curl -s "$BASE/trips/search?limit=50" | jq '{
  statutsNonPublies: [.trips[] | select(.status? and .status!="PUBLISHED") | .id],
  departsPasses: [.trips[] | select(.travelDate==null) | .id],
  n: (.trips|length)}'
curl -s "$BASE/trips/search?dateFrom=2000-01-01&limit=50" | jq '.totalCount'
```

**Attendu :** aucun trajet non publi , aucun trajet **d j  parti**, aucun trajet **masqu  par Yamba** ne sort de la recherche — m me avec une borne basse dans le pass  (la borne effective est `max(maintenant, dateFrom)`). C'est la garde qui emp che de r server un trajet retir  : un client ne peut pas la contourner par un param tre.

**API-TRIP-03 — Les valeurs de filtre invalides sont ignor es, pas rejet es Gravit  : mineure.**

```
show "$BASE/trips/search?
categories=clothes,inventee,documents&mode=teleportation&departureBuckets=matin,morning&limit=5"
```

**Attendu — HTTP 200.** Les catégories et modes sont exprimés en **convention d'interface** (clothes, plane) et non en énumération de base (CLOTHES, PLANE) ; les valeurs inconnues sont filtrées **silencieusement**. Un 400 ici serait une régression de compatibilité.

**API-TRIP-04 — Le devis affiché par la recherche et les paramètres publics de prix Gravité : majeure.**

```
curl -s "$BASE/trips/pricing/params" | jq '{commissionPct, version} + (. |
with_entries(select(.key|test("min|coef|Coef|floor"))))'
curl -s "$BASE/trips/search?from=Paris&weightKg=3&limit=3" | jq ' [.trips[] | {id, pricePerKg,
transportForWeight, totalForWeight, currency}]'
```

**Attendu :** les paramètres publics (commission, planchers, coefficients de taille, garantie étendue, kilo de référence) sont lisibles **sans session** ; avec weightKg, chaque carte porte en plus transportForWeight et totalForWeight.

**Le point à comprendre :** ces prix sont en unités (euros), pas en centimes — DTO d'affichage assumé. Le devis qui fait foi est celui que le serveur recalcule à POST /api/deals/payment-intents, en centimes. Un client qui réserve à partir des valeurs de la carte doit s'attendre à QUOTE\_DIVERGENCE s'il arrondit mal : c'est le sujet de la fiche API-DEAL-03.

**API-TRIP-05 — Les facettes comptent avec les mêmes filtres durs Gravité : mineure.**

```
curl -s "$BASE/trips/search/facets?from=Paris" | jq .
```

**Attendu — HTTP 200,** neuf compteurs. Les compteurs **par mode** sont calculés **sans** le filtre mode courant (sinon un mode non sélectionné afficherait zéro) ; les compteurs des bascules douces sont calculés **avec**. Enveloppe **sans success**, comme la recherche.

## 5.2.2 Trajet public et favoris

**API-TRIP-06 — La vue publique d'un trajet Gravité : majeure.**

```
show "$BASE/trips/$TRIP_PERKG/public"
show "$BASE/trips/000000000000000000000000/public"
show "$BASE/trips/pas-un-id/public"
```

**Attendu :** (1) **200** avec un DTO structuré (origine, destination, dates, tripper, prix, lieux) — le nom du Voyageur est réduit à **prénom + initiale** ; (2) **404** au format PublicNotFound ; (3) **400** (identifiant malformé).

**Ce que la vue publique ne doit PAS contenir :** ni l'email ni le téléphone du Voyageur, ni son identifiant technique, ni la liste de ses deals.

**API-TRIP-07 — Le favori est idempotent et gardé Gravité : majeure.**

```

show -X POST "$BASE/trips/$TRIP_PERKG/favorite" -b shipper.txt
show -X POST "$BASE/trips/$TRIP_PERKG/favorite" -b shipper.txt # rejeu
curl -s -b shipper.txt "$BASE/trips/favorites" | jq ' [.trips[].id] '
show -X DELETE "$BASE/trips/$TRIP_PERKG/favorite" -b shipper.txt
show -X DELETE "$BASE/trips/$TRIP_PERKG/favorite" -b shipper.txt # rejeu
show -X POST "$BASE/trips/$TRIP_PERKG/favorite" -b carrier.txt # SON PROPRE trajet
show -X POST "$BASE/trips/00000000000000000000000000000000/favorite" -b shipper.txt

```

**Attendu :** les deux ajouts répondent **200** { tripId, isFavorite: true } — **une seule entrée** dans la liste ; les deux retraits **200** { isFavorite: false } ; le favori sur son propre trajet **403 OWN\_TRIP** (details.type: "favorite") ; le favori d'un trajet inexistant **404** — et **jamais 403** : ne pas révéler l'existence. Un trajet non publié donne **409 TRIP\_NOT\_FAVORITABLE**.

**Effet de bord :** le favori est un **signet privé**. Le Voyageur n'en est jamais informé.

### 5.2.3 Créer et faire vivre un trajet

**API-TRIP-08 — Créer un brouillon puis le publier Gravité :** majeure · **Prérequis :** une session Voyageur avec onboarding complet (carrier.txt).

```

show -X POST "$BASE/trips" -b carrier.txt -H 'Content-Type: application/json' -d '{
  "transportMode": "PLANE", "tripType": "ONE_WAY",
  "originLabel": "Paris-Charles de Gaulle (CDG)", "originCity": "Paris", "originCountry": "France",
  "originCountryCode": "FR", "originLat": 49.0097, "originLng": 2.5479, "originTimezone": "Europe/Paris",
  "destinationLabel": "Brazzaville Maya-Maya (BZV)", "destinationCity": "Brazzaville",
  "destinationCountry": "Congo", "destinationCountryCode": "CG", "destinationLat": -4.2517,
  "destinationLng": 15.2530, "destinationTimezone": "Africa/Brazzaville",
  "departureAt": "2026-12-12T20:30:00.000Z", "arrivalAt": "2026-12-13T03:30:00.000Z",
  "flightType": "DIRECT", "acceptedCategories": ["CLOTHES", "DOCUMENTS", "BOOKS"],
  "pricePerKgCents": 1200, "capacityKg": 15,
  "pickupLocations": [{"kind": "AIRPORT", "details": "Terminal 2E", "flexibility": "EXACT"}],
  "deliveryLocations": [{"kind": "AIRPORT", "details": "Hall arrivées", "flexibility": "EXACT"}],
  "instantBooking": false, "publish": false}'
NEW_TRIP=$(jq -r '.trip.id' /tmp/body.json)
show -X POST "$BASE/trips/$NEW_TRIP/publish" -b carrier.txt

```

**Attendu :** création **201** avec status: "DRAFT", pricingModel: "PER\_KG" et un minPriceCents recalculé côté serveur (ainsi que departureHourLocal) ; publication **200** { "success": true, "message": "Trip published" }.

**Le point à prouver :** le client n'impose pas minPriceCents ni departureHourLocal — les envoyer n'a aucun effet, le serveur recalcule.

**API-TRIP-09 — Les portes de publication tiennent côté serveur Gravité :** majeure.

```
# Un brouillon volontairement incomplet : pas de lieu de récupération
show -X POST "$BASE/trips" -b carrier.txt -H 'Content-Type: application/json' -d '{
  "transportMode":"PLANE","originCity":"Paris","originCountryCode":"FR",
  "destinationCity":"Brazzaville","destinationCountryCode":"CG",
  "departureAt":"2026-12-20T20:30:00.000Z","acceptedCategories":["CLOTHES"],
  "pricePerKgCents":1200,"publish":false}'
INCOMPLET=$(jq -r '.trip.id' /tmp/body.json)
show -X POST "$BASE/trips/$INCOMPLET/publish" -b carrier.txt
```

**Attendu — HTTP 400**, avec le message de la machine à états. Les portes vérifiées sont : onboarding Voyageur terminé, Stripe avec les encaissements actifs, champs requis (mode, villes, départ **futur**, au moins une catégorie), **au moins un** lieu de récupération et **au moins un** lieu de remise, et `capacityKg` obligatoire dès que `pricePerKgCents` est posé.

**Contre-épreuve avec un compte restreint** : un compte sanctionné `RESTRICTED` reçoit **403** avec `code: "ACCOUNT_RESTRICTED"` — et **avant** le contrôleur, par l'intergiciel : la porte n'est même pas atteinte.

---

**API-TRIP-10 — `allowedActions` est la seule source des boutons Gravité** : majeure.

```
curl -s -b carrier.txt "$BASE/trips/my?status=PUBLISHED" | jq ' [.trips[] | {id, status,
allowedActions}] '
curl -s -b carrier.txt "$BASE/trips/$NEW_TRIP" | jq '.trip.allowedActions'
```

**Attendu — 200**. Chaque trajet embarque sa liste d'actions permises, calculée par la machine à états. Le client **reflète** cette liste ; il ne décide jamais.

**La contre-épreuve qui compte** : appeler une transition **absente** de `allowedActions` doit être refusée par le serveur.

```
show -X POST "$BASE/trips/$NEW_TRIP/resume" -b carrier.txt # reprendre un trajet non mis en
pause
```

**Attendu — HTTP 400** avec le motif de la machine. Un 200 ici serait une anomalie **majeure** : la règle ne vivrait que dans l'écran.

---

**API-TRIP-11 — Le cycle de vie complet d'un trajet Gravité** : majeure.

```
for A in pause resume unpublish publish cancel restore archive; do
  printf '%-10s → ' "$A"
  curl -s -o /dev/null -w '%{http_code}\n' -X POST "$BASE/trips/$NEW_TRIP/$A" -b carrier.txt
done
```

**Attendu** : chaque transition **autorisée par la machine** répond **200** ; chaque transition interdite depuis l'état courant répond **400** avec son motif. La séquence ci-dessus est volontairement mêlée : on lit la colonne des codes et on la confronte à `allowedActions` relu entre chaque appel. La règle : **jamais de 500, jamais de 200 sur une transition absente de la liste**.

---

**API-TRIP-12 — Annuler un trajet qui porte un deal vivant est REFUSÉ Gravité** : bloquante · Prérequis : `bzv-upcoming` porte `bzv-pending` (`PENDING`) et `bzv-accepted` (`ACCEPTED`).

C'est la fiche la plus importante du chapitre : elle prouve que le serveur protège l'argent et les engagements pris, indépendamment de l'écran.

```
TRIP_AVEC_DEALS=$(tid bzv-upcoming)
show -X POST "$BASE/trips/$TRIP_AVEC_DEALS/cancel" -b carrier.txt
```

**Attendu — HTTP 409** (et non 400 : le refus est un **conflit métier typé**) :

```
{ "status": "error", "message": "...",
  "details": { "type": "trip", "code": "TRIP_HAS_ACTIVE_DEALS", "activeDeals": 2 } }
```

Le champ **activeDeals** porte le nombre exact de deals vivants, pour que le client puisse écrire « annule d'abord tes 2 deals » plutôt qu'une erreur de saisie.

**Effet de bord — à vérifier absolument** : le trajet n'a pas changé de statut.

```
curl -s -b carrier.txt "$BASE/trips/$TRIP_AVEC_DEALS" | jq '{status: .trip.status,
allowedActions: .trip.allowedActions}'
```

**Attendu** : `status: "PUBLISHED"` — inchangé. Un trajet annulé alors qu'il portait un deal accepté laisserait un Expéditeur payé sans transport : c'est le scénario que cette garde interdit.

**La sortie légitime** : le Voyageur annule **chaque deal** (`POST /api/deals/{id}/cancel`, remboursement intégral), puis annule le trajet. Rejouer la fiche après avoir vidé le trajet doit donner **200**.

---

**API-TRIP-13 — Modifier un trajet, et ce qui devient immuable Gravité** : majeure.

```
show -X PUT "$BASE/trips/$NEW_TRIP" -b carrier.txt -H 'Content-Type: application/json' \
-d '{"pricePerKgCents":1300}'
show -X PUT "$BASE/trips/$NEW_TRIP" -b carrier.txt -H 'Content-Type: application/json' \
-d '{"capacityKg":50}'
```

**Attendu** : le corps est **partiel** — seuls les champs envoyés sont écrits. La modification du prix passe (**200**) tant que la machine l'autorise ; **la capacité est immuable après publication** et le second appel doit être refusé (**400**). Une édition sur un trajet `COMPLETED`, `ARCHIVED` ou `CANCELLED` est refusée par la machine.

---

**API-TRIP-14 — Le trajet d'autrui n'est ni lisible ni modifiable Gravité** : bloquante.

```
show "$BASE/trips/$TRIP_PERKG" -b shipper.txt # vue propriétaire d'un trajet d'autrui
show -X PUT "$BASE/trips/$TRIP_PERKG" -b shipper.txt -H 'Content-Type: application/json' -d
'{"pricePerKgCents":1}'
show -X POST "$BASE/trips/$TRIP_PERKG/cancel" -b shipper.txt
```

**Attendu** : **400 « Unauthorized. »** pour les trois (écart de sémantique connu — le juste code serait 403 ou 404). Ce qui compte pour la recette : **aucune de ces trois requêtes ne réussit**, et le corps ne divulgue rien du trajet. Le prix reste inchangé — le vérifier avec `GET /api/trips/{id}/public`.

---

**API-TRIP-15 — Supprimer un brouillon, annuler un trajet publié Gravité** : mineure.

```
show -X DELETE "$BASE/trips/$INCOMPLET?hard=true" -b carrier.txt # brouillon → suppression douce
show -X DELETE "$BASE/trips/$NEW_TRIP" -b carrier.txt # sans ?hard → alias d'annulation
show "$BASE/trips/$INCOMPLET/public"
```

**Attendu : 200 « Draft deleted. »** pour le premier, **200 « Trip cancelled. »** pour le second, **404** pour la lecture publique du brouillon supprimé (invisible partout).

## 5.2.4 Documents de trajet et téléversement signé

**API-TRIP-16 — Déclarer un billet et suivre sa vérification Gravit  : majeure · Pr requis : ImageKit configur , sinon ►►.**

```
show -X POST "$BASE/trips/$NEW_TRIP/documents" -b carrier.txt -H 'Content-Type: application/json' -d '{
  "documents": [{"type": "TICKET_PROOF", "fileId": "recette-billet-001",
    "url": "https://ik.imagekit.io/yamba/tickets/billet.jpg",
    "originalName": "billet.jpg", "mimeType": "image/jpeg", "title": "Billet"}]}'
show -X POST "$BASE/trips/$NEW_TRIP/documents" -b carrier.txt -H 'Content-Type: application/json' -d '{
  "documents": [{"type": "TICKET_PROOF", "fileId": "recette-billet-001",
    "url": "https://ik.imagekit.io/yamba/tickets/billet.jpg",
    "originalName": "billet.jpg", "mimeType": "image/jpeg", "title": "Billet"}]}'
curl -s -b carrier.txt "$BASE/trips/$NEW_TRIP" | jq '{ticketVerificationStatus: .trip.ticketVerificationStatus, docs: [.trip.documents[] | {id, type, status}]}'
```

**Attendu : premier appel 201** avec le trajet complet ; **second appel 200** avec « No new documents to add. » — la d duplication se fait par `fileId`, elle est **serveur**. `ticketVerificationStatus` passe de `NOT_SUBMITTED`   **PENDING** d s qu'un `TICKET_PROOF` est d pos . Les limites (nombre de documents, taille) sont appliqu es c t  serveur.

**Suppression :**

```
DOC=$(curl -s -b carrier.txt "$BASE/trips/$NEW_TRIP" | jq -r '.trip.documents[0].id')
show -X DELETE "$BASE/trips/$NEW_TRIP/documents/$DOC" -b carrier.txt
curl -s -b carrier.txt "$BASE/trips/$NEW_TRIP" | jq '.trip.ticketVerificationStatus'
```

**Attendu : 200**, et le statut de v rification **re passe   NOT\_SUBMITTED** si c t t le dernier billet.

**API-TRIP-17 — La signature de t l versement exige une session Gravit  : majeure.**

```
show "$BASE/uploads/imagekit-auth" # sans session
show "$BASE/uploads/imagekit-auth" -b shipper.txt # avec session
```

**Attendu : 401** sans session, **200** avec. La signature vaut environ **30 minutes**. Sans configuration ImageKit, `publicKey` et `urlEndpoint` sont absents du corps (et la fiche est ►► pour la suite).

**Le principe   rappeler : aucun octet d'image ne transite par l'API Yamba.** Le client demande une signature, t l verse **directement** chez ImageKit, puis **d clare l'URL obtenue**. Le m me m canisme sert aux photos de colis, de r cup ration, de remise, de litige, de message et aux documents de trajet.

**API-TRIP-18 — Supprimer un fichier est idempotent** Gravit  : mineure · Pr requis : ImageKit configur .

```
show -X DELETE "$BASE/uploads/imagekit/UN_FILE_ID" -b shipper.txt
show -X DELETE "$BASE/uploads/imagekit/UN_FILE_ID" -b shipper.txt
```

**Attendu : 200** aux deux appels ; le second r pond « File was already deleted. ». Un 500 sur un fichier d j  absent serait une anomalie majeure : la suppression doit tol rer l'absence.

## 5.3 deal-service — le c ur transactionnel

C'est le chapitre le plus dense, et le seul dans lequel de l'argent bouge. Deux acteurs, deux pots de biscuits : l'**Exp diteur** ( `shipper.txt` ) et le **Voyageur** propri taire du trajet ( `carrier.txt` ). Le deal traverse `PENDING` → `ACCEPTED` → `PICKED_UP` → `DELIVERED` → `COMPLETED`, avec les fins alternatives `DECLINED`, `EXPIRED`, `CANCELLED`, `DISPUTED`.

**R gle absolue du chapitre** : *le client refl te `allowedActions`, il ne d cide jamais*. Toute fiche qui appelle une action absente de `allowedActions` doit recevoir un refus typ .

**Pr requis global** : le fournisseur de paiement **FAKE** (§2.9) rend les fiches 5.3.1   5.3.3 jouables sans navigateur. Avec Stripe, `POST /deals/payment-intents` r pond bien mais `POST /deals`  chouera en `PAYMENT_NOT_AUTHORIZED` faute de confirmation client : ces fiches passent alors en    et l'on se rabat sur les deals du jeu d'essai.

### 5.3.1 L'intention de paiement ( tape 1 sur 2)

**API-DEAL-01 — Autoriser le montant d'un devis** Gravit  : majeure.

```
show -X POST "$BASE/deals/payment-intents" -b shipper.txt -H 'Content-Type: application/json' -d '{
  "tripId": "$TRIP_PERKG", "product": "PARCEL", "family": "CLOTHES_TEXTILE",
  "sizeClass": "M", "weightKg": 3, "protection": "BASIC", "expectedTotalCents": 0}'
```

Le devis n' tant pas connu   l'avance, on lit d'abord le refus pour apprendre le bon total, puis on rejoue avec la bonne valeur :

```
TOTAL=$(jq -r '.details.serverTotalCents // .details.totalShipperCents // empty' /tmp/body.json)
show -X POST "$BASE/deals/payment-intents" -b shipper.txt -H 'Content-Type: application/json' -d '{
  "tripId": "$TRIP_PERKG", "product": "PARCEL", "family": "CLOTHES_TEXTILE",
  "sizeClass": "M", "weightKg": 3, "protection": "BASIC", "expectedTotalCents": $TOTAL}'
PI=$(jq -r .paymentIntentId /tmp/body.json)
```

**Attendu — HTTP 201**. Corps :

```
{ "provider": "FAKE", "paymentIntentId": "pi_fake_...", "clientSecret": null,
  "amountCents": 4032, "currencyCode": "EUR",
  "quote": { "pricingModel": "PER_KG", "weightKg": 3, "pricePerKgCents": 1150, "sizeClass": "M",
    "sizeCoef": 1.1, "transportCents": ..., "commissionPct": ..., "commissionCents": ...,
    "premiumCents": 0, "serviceCents": ..., "totalShipperCents": ..., "currencyCode": "EUR"
  } }
```

**Le point   prouver — rien n'est persist **. Aucun `Booking` n'est cr  , aucun kilo n'est r serv . Le v rifier :

```
curl -s -b shipper.txt "$BASE/me/bookings" | jq '[.bookings[].id] | length'
curl -s "$BASE/trips/$TRIP_PERKG/public" | jq '.trip.pricing.remainingKg // .trip.remainingKg'
```

**Attendu** : compteurs inchangés avant et après. Une intention abandonnée expire simplement chez le fournisseur.

**Tous les montants sont des entiers en centimes.** Une valeur décimale dans un champ `...Cents` est une anomalie **bloquante**.

**API-DEAL-02 — Les huit refus typés du devis et de la demande Gravité : bloquante** (c'est le catalogue que tout client doit savoir traduire).

Chaque appel ci-dessous doit répondre **409** avec `details.type: "booking"` et le code annoncé.

```
Q () { # Q <famille> <poids> <total attendu> <pot> → un devis paramétrable
  curl -s -o /tmp/body.json -w '%{http_code}' -X POST "$BASE/deals/payment-intents" -b "$4" \
    -H 'Content-Type: application/json' \
    -d "
    {\"tripId\": \"$TRIP_PERKG\", \"product\": \"PARCEL\", \"family\": \"$1\", \"sizeClass\": \"M\", \"weigh
    tKg\": $2, \"protection\": \"BASIC\", \"expectedTotalCents\": $3}
    jq -c '{code: .details.code, cap: .details.cap, limit: .details.limit}' /tmp/body.json
  }
```

Q CLOTHES\_TEXTILE 3 1 shipper.txt # 1 – total faux  
 Q CLOTHES\_TEXTILE 999 100000 shipper.txt # 2 – plus de capacité  
 Q FOOD\_DRY\_SEALED 2 5000 shipper.txt # 3 – famille refusée par le Voyageur  
 Q CLOTHES\_TEXTILE 3 4032 carrier.txt # 5 – l'Expéditeur est le propriétaire du trajet

| # | Code attendu      | Cause                                                                                                                                                                                                     | Remède côté client                                                                                                                              |
|---|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | QUOTE_DIVERGENCE  | Le total recalculé par le serveur diffère de <code>expectedTotalCents</code> .                                                                                                                            | Rafraîchir le trajet et les paramètres de prix, recalculer, <b>ré-afficher</b> , faire reconfirmer. <b>Ne jamais débiter un montant non vu.</b> |
| 2 | CAPACITY_EXCEEDED | Plus assez de kilos disponibles sur le trajet.                                                                                                                                                            | Proposer un poids inférieur ou un autre trajet.                                                                                                 |
| 3 | FAMILY_REFUSED    | Le Voyageur refuse cette famille ( <code>familyConditions</code> , mode REFUSE).                                                                                                                          | Griser la famille dans l'assistant — les conditions sont dans le DTO public du trajet.                                                          |
| 4 | TRIP_NOT_BOOKABLE | Trajet non publié, parti, en pause, masqué ou supprimé.                                                                                                                                                   | Retour à la recherche. À jouer sur un trajet mis en pause ( <code>POST /api/trips/{id}/pause</code> ).                                          |
| 5 | OWN_TRIP          | L'Expéditeur est le propriétaire du trajet.                                                                                                                                                               | Masquer « Réserver » sur ses propres trajets.                                                                                                   |
| 6 | NEW_ACCOUNT_CAP   | Plafond progressif d'un compte récent : <code>details.cap</code> ∈ <code>DECLARED_VALUE</code> , <code>WEIGHT</code> , <code>SHIPMENTS_PER_MONTH</code> , avec <code>limit</code> et <code>value</code> . | Expliquer le plafond ; proposer de réduire ou d'attendre. À jouer avec un compte créé le jour même (API-AUTH-06).                               |

Les deux derniers codes concernent l'étape 2 :

| # | Code                   | Cause                                                                 | Remède                                                                        |
|---|------------------------|-----------------------------------------------------------------------|-------------------------------------------------------------------------------|
| 7 | PAYMENT_NOT_AUTHORIZED | L'intention n'est pas au statut « autorisée » chez le fournisseur.    | Relancer la confirmation du paiement, puis rejouer <code>POST /deals</code> . |
| 8 | PAYMENT_MISMATCH       | Le montant ou la devise de l'intention ne correspondent pas au devis. | Recréer une intention, puis rejouer.                                          |

Et le neuvième, qui est le sujet de la fiche API-IDEM-01 :

| # | Code                 | Cause                                     | Remède                                                                                    |
|---|----------------------|-------------------------------------------|-------------------------------------------------------------------------------------------|
| 9 | PAYMENT_ALREADY_USED | L'intention a déjà servi à créer un deal. | <b>Relire</b> <code>GET /api/me/bookings</code> — le deal existe déjà. Ne jamais recréer. |

Un compte restreint reçoit **403 ACCOUNT\_RESTRICTED** avant d'atteindre ces contrôles.

### API-DEAL-03 — La divergence de devis protège l'Expéditeur Gravité : bloquante.

```
show -X POST "$BASE/deals/payment-intents" -b shipper.txt -H 'Content-Type: application/json' -d '{
  "tripId": "$TRIP_PERKG", "product": "PARCEL", "family": "CLOTHES_TEXTILE",
  "sizeClass": "M", "weightKg": 3, "protection": "BASIC", "expectedTotalCents": 1}'
```

**Attendu — HTTP 409 QUOTE\_DIVERGENCE . Aucune autorisation n'a été posée** chez le fournisseur : le montant qui n'a pas été vu par l'Expéditeur n'est jamais réservé sur sa carte. C'est la garde du devis figé : le prix est recalculé côté serveur avec le moteur unique, puis figé dans le deal — jamais recalculé depuis le trajet ensuite.

### 5.3.2 Créer la demande (étape 2 sur 2)

**API-DEAL-04 — Créer un deal PENDING Gravité : majeure · Prérequis : une intention PI valide (API-DEAL-01).**

```
show -X POST "$BASE/deals" -b shipper.txt -H 'Content-Type: application/json' -d '{
  "tripId": "$TRIP_PERKG", "paymentIntentId": "$PI", "expectedTotalCents": $TOTAL,
  "product": "PARCEL", "family": "CLOTHES_TEXTILE", "sizeClass": "M", "weightKg": 3,
  "protection": "BASIC",
  "description": "Deux pagnes et un boubou brodé, emballés sous vide.",
  "declaredValueCents": 15000,
  "photoUrls": ["https://ik.imagekit.io/yamba/parcels/pagne-1.jpg"],
  "recipient": {
    "firstName": "Marie", "lastName": "Kabeya",
    "phoneE164": "+243812345678", "email": "marie.k@example.com"
  },
  "pickupPlace": {
    "kind": "AIRPORT", "details": "Terminal 2E"
  },
  "deliveryPlace": {
    "kind": "AIRPORT", "details": "Hall arrivées"
  },
  "charterAccepted": true, "termsAccepted": true
}'
NEW_DEAL=$(jq -r .bookingId /tmp/body.json)
```

**Attendu — HTTP 201 :**

```
{ "bookingId": "...", "status": "PENDING", "expiresAt": "...+24 h",
  "totalShipperCents": 4032, "currencyCode": "EUR" }
```

**Effets de bord à vérifier — c'est le cœur de la fiche.** Tout se fait en **une seule transaction Mongo** :

```
# a) Les kilos sont réservés sur le trajet
curl -s "$BASE/trips/$TRIP_PERKG/public" | jq '.trip.remainingKg'
# b) Le deal apparaît dans « mes envois »
curl -s -b shipper.txt "$BASE/me/bookings?status=PENDING" | jq '[.bookings[].id]'
# c) Le Voyageur le voit dans ses deals reçus
curl -s -b carrier.txt "$BASE/me/deals" | jq '[.deals[] | {id, status}]'
```

**Attendu :** (a) `remainingKg` diminué de 3 · (b) et (c) le nouveau deal présent. Deux événements sont écrits dans la boîte d'envoi (`booking.requested`, `booking.payment_authorized`) — le chapitre 4 du cahier n° 4 en fait la

preuve côté événements.

**La charte est obligatoire** : rejouer avec `"charterAccepted": false` doit répondre **400**.

#### API-DEAL-05 — Cinq instantanés sont figés dans le deal Gravité : bloquante.

```
curl -s -b shipper.txt "$BASE/deals/$NEW_DEAL" | jq '.deal | {trip, pricing, parcel, recipient, places: {pickupPlace, deliveryPlace}}'
# Modifier le prix du trajet APRÈS la création
curl -s -b carrier.txt -X PUT "$BASE/trips/$TRIP_PERKG" -H 'Content-Type: application/json' -d '{"pricePerKgCents":9999}'
curl -s -b shipper.txt "$BASE/deals/$NEW_DEAL" | jq '.deal.pricing'
```

**Attendu** : le devis du deal est **identique avant et après** la modification du trajet. Le prix d'un deal est un **instantané immuable** ; il n'est **jamais** recalculé depuis le trajet. Un changement observé ici est une anomalie **bloquante** (l'Expéditeur serait débité d'un montant qu'il n'a pas vu).

**Remettre le prix d'origine** avant de continuer.

#### API-DEAL-06 — Les deux vues d'un deal sont des listes blanches strictes Gravité : bloquante.

```
curl -s -b shipper.txt "$BASE/deals/$DEAL_PICKED" | jq '{viewerRole, statut: .deal.status, allowedActions: .deal.allowedActions, code: .deal.deliveryCode, recipient: .deal.recipient, total: .deal.pricing.totalShipperCents}'
curl -s -b carrier.txt "$BASE/deals/$DEAL_PICKED" | jq '{viewerRole, statut: .deal.status, allowedActions: .deal.allowedActions,
  fuites: [(.deal|paths(scalars) as $p | select((getpath($p)|tostring)=="742891")) | ($p|join("."))],
  champsInterdits: [(.deal|keys[]) | select(test("deliveryCode|recipient|codeHash|totalShipper";"i"))]}'
```

**Attendu** :

| Vue     | Ce qu'elle porte                                                                                                                                                                                    | Ce qu'elle ne porte JAMAIS                                                                            |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| SHIPPER | Le devis complet ( <code>totalShipperCents</code> ...), le destinataire, <code>cancellationPreview</code> , <b><code>deliveryCode</code> en clair — uniquement au statut <code>PICKED_UP</code></b> | —                                                                                                     |
| CARRIER | Sa rémunération ( <code>transportCents</code> ) et son <code>currencyCode</code>                                                                                                                    | <b><code>deliveryCode</code></b> , l'empreinte du code, le destinataire, <b>les totaux Expéditeur</b> |

La commande `fuites` doit renvoyer `[]` et `champsInterdits` `[]` dans la vue Voyageur. Toute occurrence est une anomalie **bloquante** — c'est le secret qui garantit la remise.

**Sur un deal qui n'est pas `PICKED_UP`**, `deliveryCode` vaut `null` **même** dans la vue Expéditeur. Et le code n'apparaît **dans aucune liste** :

```
curl -s -b shipper.txt "$BASE/me/bookings" | grep -c 742891 # attendu : 0
curl -s -b carrier.txt "$BASE/me/deals" | grep -c 742891 # attendu : 0
```

### 5.3.3 Acceptation, refus, annulation

**API-DEAL-07 — Le Voyageur accepte : le paiement est capturé** Gravité : majeure · Prérequis : un deal PENDING (\$DEAL\_PENDING ou \$NEW\_DEAL).

```
show -X POST "$BASE/deals/$DEAL_PENDING/accept" -b carrier.txt \
-H 'Content-Type: application/json' -d '{"charterAccepted":true}'
```

**Attendu — HTTP 200 :** { "bookingId": "...", "status": "ACCEPTED", "refundAmountCents": null, "currencyCode": "EUR" }.

**Effets de bord :**

```
curl -s -b shipper.txt "$BASE/deals/$DEAL_PENDING" | jq '{status: .deal.status, allowedActions:
.deal.allowedActions, capturedAt: .deal.capturedAt}'
curl -s -b shipper.txt "$BASE/messages/conversations/by-deal/$DEAL_PENDING" | jq
'.conversation.access'
```

**Attendu :** status: "ACCEPTED", allowedActions mis à jour, une date de capture posée, et surtout la conversation du deal devient disponible (canWrite: true) — elle n'existait pas avant.

**La charte du transporteur est obligatoire :** rejouer avec {"charterAccepted": false} doit répondre 400.

---

**API-DEAL-08 — Les trois refus de l'acceptation** Gravité : majeure.

```
# a) Un deal déjà accepté (rejeu)
show -X POST "$BASE/deals/$DEAL_PENDING/accept" -b carrier.txt -H 'Content-Type:
application/json' -d '{"charterAccepted":true}'
# b) Un deal terminal
show -X POST "$BASE/deals/$(bid bzv-expired)/accept" -b carrier.txt -H 'Content-Type:
application/json' -d '{"charterAccepted":true}'
# c) L'Expéditeur qui tente d'accepter à la place du Voyageur
show -X POST "$BASE/deals/$(bid gru-pending)/accept" -b shipper3.txt -H 'Content-Type:
application/json' -d '{"charterAccepted":true}'
```

**Attendu :** (a) 409 TRANSITION\_NOT\_ALLOWED avec details.reason · (b) 409 TRANSITION\_NOT\_ALLOWED · (c) 403 (João est l'Expéditeur de gru-pending, pas le Voyageur — la transition est liée au rôle).

Les deux autres codes possibles ici : CARRIER\_ONBOARDING\_REQUIRED (porte D31 — profil Voyageur incomplet ou Stripe non prêt ; à jouer avec un compte dont l'onboarding n'est pas fini) et PAYMENT\_STATE\_CONFLICT (l'autorisation n'est plus capturable ; l'Expéditeur doit refaire une demande).

---

**API-DEAL-09 — Le Voyageur refuse : remboursement intégral** Gravité : majeure.

```
show -X POST "$BASE/deals/$(bid gru-pending)/decline" -b ines.txt \
-H 'Content-Type: application/json' -d '{"reason":"TIMING"}'
```

**Attendu — HTTP 200** avec status: "DECLINED" et un refundAmountCents égal au total payé. Le motif est facultatif, parmi cinq : CATEGORY\_NOT\_CARRIED, TOO\_HEAVY, PLACES\_INCOMPATIBLE, TIMING, OTHER. Un motif hors liste répond 400.

**Effet de bord :** les kilos sont libérés sur le trajet — le vérifier avec GET /api/trips/{id}/public. L'autorisation n'est jamais capturée dans ce chemin.

---

## API-DEAL-10 — L'Expéditeur annonce puis annule, et la retenue est celle annoncée Gravité : bloquante.

```
# 1. Ce que le serveur annonce AVANT le geste
curl -s -b shipper.txt "$BASE/deals/$DEAL_PENDING" | jq '.deal.cancellationPreview'
# 2. Le geste
show -X POST "$BASE/deals/$DEAL_PENDING/cancel" -b shipper.txt \
  -H 'Content-Type: application/json' -d '{"reason":"Changement de plan côté destinataire"}
```

**Attendu :** l'aperçu porte `refundCents`, `retentionCents`, `retentionPct` et `fullRefundUntil` (= départ – 48 h). La réponse de l'annulation porte `refundAmountCents`, et il doit **correspondre à l'aperçu** lu juste avant.

**La règle :** remboursement **intégral** tant que le deal est `PENDING`, et jusqu'à 48 h avant le départ une fois accepté ; **retenue de 50 %** au-delà (valeur réglable par l'administration).

**Le point de recette :** l'aperçu est un **instantané informatif** ; le remboursement est **recalculé au moment réel de l'annulation**. Un écart entre les deux, sur un intervalle court, est une anomalie **bloquante** — l'Expéditeur aurait consenti à un montant différent de celui appliqué.

**Après la récupération, l'annulation est impossible :**

```
show -X POST "$BASE/deals/$DEAL_PICKED/cancel" -b shipper.txt -H 'Content-Type:
application/json' -d '{"reason":"trop tard"}'
```

**Attendu — 409 `TRANSITION_NOT_ALLOWED`.** Le seul chemin restant est le litige.

---

### 5.3.4 Transport : récupération, jalons, remise

---

**API-DEAL-11 — La récupération exige les cinq points d'inspection Gravité : bloquante · Prérequis :** un deal `ACCEPTED` (`$DEAL_ACCEPTED`, Voyageur Thomas).

```
# a) Une inspection PARTIELLE doit être refusée
show -X POST "$BASE/deals/$DEAL_ACCEPTED/pickup" -b carrier.txt -H 'Content-Type:
application/json' -d '{
  "checklist":["CONTENT_MATCHES","WEIGHT_OK"],
  "photoUrls":["https://ik.imagekit.io/yamba/pickups/1.jpg"],"notes":null}'
# b) Sans photo
show -X POST "$BASE/deals/$DEAL_ACCEPTED/pickup" -b carrier.txt -H 'Content-Type:
application/json' -d '{
  "checklist":["CONTENT_MATCHES","WEIGHT_OK","NO_FORBIDDEN","PACKAGING_OK","ITEMS_IDENTIFIED"],
  "photoUrls":[],"notes":null}'
# c) L'inspection complète
show -X POST "$BASE/deals/$DEAL_ACCEPTED/pickup" -b carrier.txt -H 'Content-Type:
application/json' -d '{
  "checklist":["CONTENT_MATCHES","WEIGHT_OK","NO_FORBIDDEN","PACKAGING_OK","ITEMS_IDENTIFIED"],
  "photoUrls":
["https://ik.imagekit.io/yamba/pickups/1.jpg","https://ik.imagekit.io/yamba/pickups/2.jpg"],
  "notes":"Colis conforme, 2,9 kg à la pesée."}'
```

**Attendu :** (a) **400** — les **cinq** points sont obligatoires, le serveur refuse une inspection partielle ; (b) **400** — entre **1 et 5** photos, déjà téléversées chez ImageKit ; (c) **200** avec `status: "PICKED_UP"`.

---

**API-DEAL-12 — Le code de livraison n'est JAMAIS dans la réponse de la récupération Gravité : bloquante.**

```
grep -c -E '[0-9]{6}' /tmp/body.json      # le corps de l'appel (c) ci-dessus
jq . /tmp/body.json
```

**Attendu :** le corps est { bookingId, status: "PICKED\_UP", refundAmountCents: null, currencyCode } — **et rien d'autre**. Le code à six chiffres généré par le serveur n'y figure pas.

**Où le code vit :** stocké **deux fois** (empreinte bcrypt pour la validation, chiffrement AES pour le ré-affichage à l'Expéditeur). Il est lisible **uniquement** par l'Expéditeur, **uniquement** sur GET /api/deals/{id}, **uniquement** au statut PICKED\_UP.

```
curl -s -b shipper2.txt "$BASE/deals/$DEAL_ACCEPTED" | jq '{status: .deal.status, deliveryCode:
.deal.deliveryCode}'
curl -s -b carrier.txt "$BASE/deals/$DEAL_ACCEPTED" | jq '.deal.deliveryCode'
```

**Attendu :** un code à six chiffres côté Expéditeur ; **null** ou **champ absent** côté Voyageur.

**Les quatre surfaces interdites, à contrôler pendant toute la campagne :** le code ne voyage **jamais** dans un événement, **jamais** dans un email, **jamais** dans une vue Voyageur, **jamais** dans une liste. Il se transmet **de vive voix** de l'Expéditeur au destinataire.

---

**API-DEAL-13 — Refuser le colis à la récupération Gravité :** majeure.

```
show -X POST "$BASE/deals/$(bid yul-accepted)/pickup/refuse" -b marc.txt \
-H 'Content-Type: application/json' -d '{"reason":"OVERWEIGHT"}'
```

**Attendu — HTTP 200 :** le deal passe en CANCELLED, le paiement capturé est **remboursé intégralement**, les kilos sont libérés. **Aucune pénalité de réputation** pour le Voyageur : c'est le comportement voulu (refuser un colis non conforme doit être sans coût).

---

**API-DEAL-14 — Les jalons suivent une séquence stricte Gravité :** majeure · **Prérequis :** sgn-picked a déjà AT\_AIRPORT.

```
DEAL_SGN=$(bid sgn-picked)
# a) Sauter une étape
show -X POST "$BASE/deals/$DEAL_SGN/events" -b linh.txt -H 'Content-Type: application/json' -d
'{"step":"FLIGHT_ARRIVED"}'
# b) Rejouer une étape déjà confirmée
show -X POST "$BASE/deals/$DEAL_SGN/events" -b linh.txt -H 'Content-Type: application/json' -d
'{"step":"AT_AIRPORT"}'
# c) L'étape suivante, dans l'ordre
show -X POST "$BASE/deals/$DEAL_SGN/events" -b linh.txt -H 'Content-Type: application/json' -d
'{"step":"FLIGHT_DEPARTED"}'
# d) Un jalon sur un deal qui n'est pas PICKED_UP
show -X POST "$BASE/deals/$DEAL_DELIVERED/events" -b carrier.txt -H 'Content-Type:
application/json' -d '{"step":"AT_AIRPORT"}'
```

**Attendu :** (a) **409 TRACKING\_STEP\_NOT\_ALLOWED** — pas de saut ; (b) **409** — pas de doublon ; (c) **200** avec { bookingId, step, confirmedAt, trackingEvents: [...] } (la **séquence complète** est renvoyée) ; (d) **409** — le deal doit être PICKED\_UP.

**Deux points à ne pas confondre :** un jalon **ne change pas le statut** du deal, et l'annulation « 5 secondes » est **entièrement côté client** — il n'existe aucune annulation serveur. Le client appelle la route **après** sa fenêtre

d'annulation locale.

**API-DEAL-15 — La remise valide le code, et le verrou tient Gravité : bloquante · Prérequis : un deal PICKED\_UP du jeu d'essai (code 742891).**

```
D=$(bid bzv-picked)
# Trois codes faux
for c in 111111 222222 333333; do
    printf '%s → ' "$c"
    curl -s -o /tmp/body.json -w '%{http_code} ' -X POST "$BASE/deals/$D/deliver" -b carrier.txt \
        -H 'Content-Type: application/json' -d '{"code":"$c","photoUrls":[]}'
    jq -c '{code: .details.code, attemptsLeft: .details.attemptsLeft, lockedUntil:
.details.lockedUntil}' /tmp/body.json
done
# Le bon code PENDANT le verrou
show -X POST "$BASE/deals/$D/deliver" -b carrier.txt -H 'Content-Type: application/json' \
    -d '{"code":"742891","photoUrls":[]}'
```

**Attendu :**

| Essai                      | Statut | details                                                                   |
|----------------------------|--------|---------------------------------------------------------------------------|
| 1                          | 409    | code: "DELIVERY_CODE_INVALID", attemptsLeft: 2                            |
| 2                          | 409    | code: "DELIVERY_CODE_INVALID", attemptsLeft: 1                            |
| 3                          | 409    | code: "DELIVERY_LOCKED", lockedUntil à +15 minutes                        |
| bon code pendant le verrou | 409    | DELIVERY_LOCKED — le serveur refuse la comparaison avant même de la faire |

C'est le point de la fiche : le verrou est vérifié **par la machine à états, avant toute comparaison**. Un client qui insisterait ne gagnerait rien. Après expiration du verrou, le bon code répond **200** :

```
{ "bookingId": "...", "status": "DELIVERED", "deliveredAt": "...", "payoutDueAt": "...+4 jours" }
```

**Effet de bord :** payoutDueAt = remise + **4 jours** — la fenêtre de vérification de l'Expéditeur.

**Le cinquième code possible :** DELIVERY\_CODE\_UNAVAILABLE , pour un enregistrement antérieur à la mise en place du code (contacter le support).

**API-DEAL-16 — Régénérer le code, cinq fois au plus Gravité : majeure.**

```
D=$(bid bzv-tracking)
for i in 1 2 3 4 5 6; do
    printf 'régénération %s → ' "$i"
    curl -s -o /tmp/body.json -w '%{http_code} ' -X POST "$BASE/deals/$D/code/regenerate" -b
shipper2.txt
    jq -c '{code: (.deliveryCode|type), left: .codeRegenerationsLeft, err: .details.code}'
/tmp/body.json
done
show -X POST "$BASE/deals/$D/code/regenerate" -b carrier.txt # le VOYAGEUR tente
```

**Attendu :** cinq **200** avec un **nouveau code** et un codeRegenerationsLeft décroissant (4, 3, 2, 1, 0), puis un **409 CODE\_REGENERATION\_LIMIT**. La tentative du Voyageur répond **403** : seule l'Expéditrice régénère.

**Deux points remarquables :**

- Cette réponse est la **seule surface d'écriture** de toute l'API qui renvoie un code de livraison.
- L'ancien code devient **immédiatement invalide**, et les tentatives ratées ainsi que le verrou sont remis à zéro. Le vérifier : après régénération, 742891 répond `DELIVERY_CODE_INVALID`.
- L'événement émis ne porte que des **compteurs**, jamais le code.

### 5.3.5 Règlement : confirmation, litige, notation

**API-DEAL-17 — L'Expéditeur confirme : le versement part Gravité : majeure · Prérequis : un deal DELIVERED.**

```
show -X POST "$BASE/deals/$DEAL_DELIVERED/confirm" -b shipper3.txt
```

**Attendu — HTTP 200 :**

```
{ "bookingId": "...", "status": "COMPLETED", "completedAt": "...",
  "payoutStatus": "SENT", "payoutAmountCents": 3600, "currencyCode": "EUR" }
```

`payoutStatus` vaut **SENT** si le transfert a réussi en ligne, **FAILED** si le fournisseur a refusé (une tâche planifiée réessaie toutes les 5 minutes, jusqu'à 10 fois ; l'administration voit la file d'exception). `payoutAmountCents` doit être égal au `transportCents` du devis — le net Voyageur, jamais le total Expéditeur.

**Le geste est irréversible :** rejouer la confirmation, ou ouvrir un litige après, doit répondre **409**

**TRANSITION\_NOT\_ALLOWED**. Le droit de contester disparaît avec la confirmation ; c'est une règle métier, et elle doit tenir côté serveur.

```
show -X POST "$BASE/deals/$DEAL_DELIVERED/confirm" -b shipper3.txt
show -X POST "$BASE/deals/$DEAL_DELIVERED/dispute" -b shipper3.txt -H 'Content-Type:
application/json' \
  -d '{"category":"DAMAGED","pledgeAccepted":true,"description":"Description de plus de
cinquante caractères pour passer la garde de longueur.","photoUrls":[]}'
```

**API-DEAL-18 — Ouvrir un litige gèle le versement Gravité : majeure · Prérequis : un deal DELIVERED non confirmé (yul-delivered).**

```
D=$(bid yul-delivered)
# a) Une description trop courte
show -X POST "$BASE/deals/$D/dispute" -b shipper.txt -H 'Content-Type: application/json' \
  -d '{"category":"DAMAGED","pledgeAccepted":true,"description":"Cassé.","photoUrls":[]}'
# b) Sans engagement d'honnêteté
show -X POST "$BASE/deals/$D/dispute" -b shipper.txt -H 'Content-Type: application/json' \
  -d '{"category":"DAMAGED","pledgeAccepted":false,"description":"Le colis est arrivé avec l
emballage déchiré et le contenu abîmé sur le côté gauche.","photoUrls":[]}'
# c) Le litige complet
show -X POST "$BASE/deals/$D/dispute" -b shipper.txt -H 'Content-Type: application/json' -d '{
  "category":"DAMAGED","pledgeAccepted":true,"desiredOutcome":"PARTIAL_REFUND",
  "description":"Le colis est arrivé avec l emballage déchiré et le boubou taché sur la manche
gauche ; photos jointes.",
  "photoUrls":["https://ik.imagekit.io/yamba/disputes/1.jpg"]}'
```

**Attendu :** (a) **400** — la description fait **au moins 50 caractères** ; (b) **400** — `pledgeAccepted` doit valoir `true` ; (c) **200** avec `{ bookingId, status: "DISPUTED", ticketNumber: "YAM-XXXX", disputedAt }`.

## Effets de bord :

```
curl -s -b shipper.txt "$BASE/deals/$D" | jq '{status: .deal.status, payoutStatus:
.deal.payoutStatus}'
curl -s -b marc.txt "$BASE/messages/conversations/by-deal/$D" | jq '.conversation.access'
```

**Attendu :** payoutStatus: "FROZEN" — le versement est **gelé** ; et la conversation passe en **lecture seule** avec access.reason: "DISPUTE\_OPEN" (les échanges passent désormais par la médiation).

**Les six catégories :** NOT\_DELIVERED, CONTENT\_MISSING, DAMAGED, SIGNIFICANT\_DELAY, RECIPIENT\_ISSUE, OTHER. Depuis PICKED\_UP (colis non remis, départ passé de plus de 48 h), la catégorie est **obligatoirement** NOT\_DELIVERED — toute autre répond **400**.

---

**API-DEAL-19 — Le Voyageur donne sa version, une seule fois Gravité :** majeure · **Prérequis :** un deal DISPUTED .

```
D=$(bid bzv-disputed)
show -X POST "$BASE/deals/$D/dispute/statement" -b carrier.txt -H 'Content-Type:
application/json' -d '{
  "statement":"Le colis m a été remis fermé et je l ai transporté en bagage cabine sans incident
; photos de la remise jointes.",
  "photoUrls":["https://ik.imagekit.io/yamba/disputes/carrier-1.jpg"]}'
show -X POST "$BASE/deals/$D/dispute/statement" -b carrier.txt -H 'Content-Type:
application/json' -d '{
  "statement":"Le colis m a été remis fermé et je l ai transporté en bagage cabine sans incident
; photos de la remise jointes.",
  "photoUrls":[]}'
show -X POST "$BASE/deals/$D/dispute/statement" -b shipper.txt -H 'Content-Type:
application/json' -d '{
  "statement":"Une déclaration de plus de cinquante caractères écrite par l Expéditrice, qui n a
pas le droit.",
  "photoUrls":[]}'
```

**Attendu :** premier appel **201** { bookingId, ticketNumber, respondedAt } ; second **409** (une seule déclaration) ; troisième **403** (le Voyageur seul). Longueur minimale : **50 caractères**, **5 photos** au plus.

---

**API-DEAL-20 — La notation en double aveugle Gravité :** majeure · **Prérequis :** un deal COMPLETED de moins de 14 jours.

```
D=$(bid bzv-completed)
curl -s -b mai.txt "$BASE/deals/$D/rating" | jq '{canRate, ratedRole, windowEndsAt, myRating,
counterpartHasRated, counterpartRating, revealedAt}'
show -X POST "$BASE/deals/$D/rating" -b mai.txt -H 'Content-Type: application/json' -d '{
  "rating":5,"criteria":
{"PUNCTUALITY":"UP","COMMUNICATION":"UP","PARCEL_CARE":"UP","DECLARATION_CLARITY":"UP"},
  "comment":"Ponctuel et soigneux, je recommande."}'
curl -s -b mai.txt "$BASE/deals/$D/rating" | jq '{myRating, counterpartRating, revealedAt}'
```

**Attendu :** le contexte répond **200** avec canRate: true et counterpartRating: null ; la notation répond **201** { bookingId, submittedAt, revealed: false, revealedAt: null } ; la relecture montre **sa propre note** et counterpartRating: null tant que l'autre n'a pas noté.

**Le point du double aveugle :** un avis reste caché jusqu'à ce que **les deux** aient noté, ou jusqu'à la fin de la fenêtre de **14 jours**. Voir la note de l'autre partie avant cette condition serait une anomalie **majeure**.

**Les critères hors du rôle noté sont ignorés silencieusement** : ci-dessus, `DECLARATION_CLARITY` (critère d'un Expéditeur) est écarté puisque la cible est un Voyageur. Le relire pour le confirmer. Critères valides : pour noter un Voyageur `PUNCTUALITY`, `COMMUNICATION`, `PARCEL_CARE` ; pour noter un Expéditeur `DECLARATION_CLARITY`, `RESPONSIVENESS`, `PUNCTUALITY`.

**Les refus :**

```
show -X POST "$BASE/deals/$D/rating" -b mai.txt -H 'Content-Type: application/json' -d '{"rating":4}'
show -X POST "$BASE/deals/$DEAL_PICKED/rating" -b shipper.txt -H 'Content-Type: application/json' -d '{"rating":5}'
```

**Attendu : 409 `TRANSITION_NOT_ALLOWED`** pour les deux (déjà noté ; deal non `COMPLETED`).

**Effet de bord** : seuls les avis **révélés** nourrissent la réputation publique. Le vérifier sur `GET /api/users/{slug}/public/reviews`.

---

### 5.3.6 Lien de suivi du destinataire

---

**API-DEAL-21 — Un seul lien par deal, créé par l'Expéditeur Gravité : majeure.**

```
show -X POST "$BASE/deals/$DEAL_PICKED/tracking-link" -b shipper.txt
TOK=$(jq -r .token /tmp/body.json)
show -X POST "$BASE/deals/$DEAL_PICKED/tracking-link" -b shipper.txt      # rejeu
show -X POST "$BASE/deals/$DEAL_PICKED/tracking-link" -b carrier.txt      # le Voyageur
show -X POST "$BASE/deals/$(bid bzv-pending)/tracking-link" -b shipper.txt # avant acceptation
```

**Attendu** : premier appel **200** avec `{ token, path: "/track/<token>", recipientFirstName, recipientPhoneE164 }` ; le **rejeu renvoie le même token** (un seul lien par deal) ; le Voyageur reçoit **403** ; avant l'acceptation, **409 `TRACKING_NOT_AVAILABLE`**.

**path est un chemin côté front**, pas une route d'API : la page publique du front l'affiche et appelle ensuite la route publique.

---

**API-DEAL-22 — La page destinataire est publique et minimale Gravité : bloquante.**

```
curl -s "$BASE/track/$TOK" | jq .
curl -s "$BASE/track/$TOK" | jq '[paths(scalars) as $p | {p: ($p|join(".")), v: getpath($p)}]
| map(select((.v|tostring) | test("742891|@|\\+[0-9]{8,}|Cents";"i")))'
```

**Attendu — HTTP 200, sans aucune session :**

```
{ "milestone": "IN_TRANSIT",
  "steps": [ { "key": "ACCEPTED", "at": "..." }, { "key": "PICKED_UP", "at": "..." } ],
  "recipientFirstName": "Clarisse", "shipperFirstName": "Aminata",
  "carrier": { "firstName": "Thomas", "lastInitial": "N" },
  "corridor": { "originCity": "Paris", "destinationCity": "Brazzaville" },
  "departureAt": "...", "arrivalAt": "..." }
```

**Ce que la page ne doit JAMAIS contenir** — et la seconde commande doit renvoyer `[]` : aucune adresse, aucun numéro de téléphone, aucun email, **aucun code de livraison**, aucune photo, aucun montant. Les jalons possibles sont `ACCEPTED`, `PICKED_UP`, `IN_TRANSIT`, `ARRIVED`, `DELIVERED`, `CLOSED`.

**Le jeton est un secret :**

```
show "$BASE/track/${TOK}x"
show "$BASE/track/aaaaaaaaaaaaaaaaaaaaaa"
```

**Attendu — 404 pour les deux.** Le jeton fait 32 octets tirés au sort ; il n'est ni devinable ni énumérable. Un 404 est aussi la réponse attendue une fois le destinataire anonymisé (rétention), le lien révoqué ou le deal supprimé.

**Rappel :** cette route est publique, donc soumise au **plafond anonyme** du limiteur (100 requêtes par quart d'heure et par adresse IP).

---

### 5.3.7 Listes et portefeuille

---

**API-DEAL-23 — Chaque liste est bornée à son propriétaire Gravité : bloquante.**

```
curl -s -b shipper.txt "$BASE/me/bookings" | jq '[.bookings[] | {id, status}] | length'
curl -s -b shipper.txt "$BASE/me/bookings?status=PENDING" | jq '[.bookings[].status] | unique'
curl -s -b carrier.txt "$BASE/me/deals" | jq '[.deals[].id] | length'
curl -s -b carrier.txt "$BASE/deals?tripId=$TRIP_PERKG" | jq '[.deals[].id] | length'
curl -s -b shipper.txt "$BASE/deals?tripId=$TRIP_PERKG" -o /tmp/body.json -w '%{http_code}\n'
curl -s -b carrier.txt "$BASE/me/wallet" | jq 'keys'
```

**Attendu :** chaque liste ne contient que **les deals de l'appelant**, dans son rôle. Le filtre par statut est un **filtre exact**. La liste des deals **d'un trajet** est réservée au **propriétaire du trajet** : l'appel de l'Expéditrice répond **403** (le trajet appartient à Thomas).

**Le contrôle qui compte :** croiser les identifiants renvoyés à un membre avec ceux d'un autre. Un deal qui apparaît dans deux listes de membres non parties est une anomalie **bloquante**.

```
comm -12 <(curl -s -b shipper.txt "$BASE/me/bookings" | jq -r '.bookings[].id' | sort) \
      <(curl -s -b shipper3.txt "$BASE/me/bookings" | jq -r '.bookings[].id' | sort)
```

**Attendu :** aucune ligne.

---

## 5.4 message-service — fil, rendez-vous, numéro, signalement

Le jeu d'essai fournit une conversation vivante sur le deal `bzv-accepted` : deux messages, un rendez-vous proposé, un message déjà signalé. C'est la cible par défaut de ce chapitre.

```
DEAL_CONV=$(bid bzv-accepted)
CONV=$(curl -s -b shipper2.txt "$BASE/messages/conversations/by-deal/$DEAL_CONV" | jq -r
.conversation.id)
```

---

**API-MSG-01 — Le fil d'un deal est créé au premier accès Gravité : majeure.**

```
show "$BASE/messages/conversations/by-deal/$DEAL_CONV" -b shipper2.txt -H 'x-locale: fr'
show "$BASE/messages/conversations/by-deal/$DEAL_CONV" -b carrier.txt
show "$BASE/messages/conversations" -b shipper2.txt
```

**Attendu — HTTP 200.** Le corps porte :

```
{ "conversation": { "id": "...", "bookingId": "...", "role": "SHIPPER",
  "counterpart": { "id": "...", "firstName": "Thomas", "avatarUrl": null },
  "corridor": { "originCity": "Paris", "destinationCity": "Brazzaville", "departureAt": "..." },
  "bookingStatus": "ACCEPTED", "lastMessage": {...}, "unreadCount": 1, "nextMeetup": {...},
  "access": { "canRead": true, "canWrite": true, "reason": null, "writeClosesAt": null } },
  "messages": [ ... ], "meetups": [ ... ], "nextCursor": null,
  "phone": { "revealed": false, "phoneE164": null, "opensAt": "..." } }
```

**Le point à prouver — l'appel est idempotent et créateur :** appeler « by-deal » **crée** la conversation si elle n'existe pas, et **renvoie la même** ensuite. Rejouer trois fois et vérifier que `conversation.id` ne change pas.

**Le contact ne porte que le prénom :** `counterpart` n'expose ni email, ni téléphone, ni nom complet.

**API-MSG-02 — Aucun fil avant l'acceptation, aucun fil pour un tiers Gravit  : bloquante.**

```
show "$BASE/messages/conversations/by-deal/${bid gru-pending}" -b shipper3.txt # deal PENDING
show "$BASE/messages/conversations/by-deal/$DEAL_CONV" -b shipper3.txt # un tiers
show "$BASE/messages/conversations/$CONV" -b shipper3.txt # un tiers, par id
```

**Attendu — HTTP 403 pour les trois.** La conversation d'un deal **existe   partir de l'acceptation** : avant, la demande porte d j  une description, et ouvrir un fil inviterait   sortir de la plateforme. Un tiers n'acc de ni par le deal, ni par l'identifiant de conversation.

**API-MSG-03 — Poster un message, et les deux gardes Gravit  : bloquante.**

```
# a) Un message ordinaire
show -X POST "$BASE/messages/conversations/$CONV/messages" -b shipper2.txt \
  -H 'Content-Type: application/json' -d '{"body":"Bonjour Thomas, je serai au terminal 2E vers 19 h avec le colis."}'
# b) LE CODE DE LIVRAISON dans le texte – doit  tre REFUS 
show -X POST "$BASE/messages/conversations/$CONV/messages" -b shipper2.txt \
  -H 'Content-Type: application/json' -d '{"body":"Je te donne le code tout de suite : 742891"}'
# c) Des coordonn es – doivent  tre MARQU ES, jamais bloqu es
show -X POST "$BASE/messages/conversations/$CONV/messages" -b shipper2.txt \
  -H 'Content-Type: application/json' -d '{"body":"Mon num ro est le +33612345611, appelle-moi."}'
# d) Un texte vide
show -X POST "$BASE/messages/conversations/$CONV/messages" -b shipper2.txt \
  -H 'Content-Type: application/json' -d '{"body":""}'
```

**Attendu :**

|   | Statut | Corps                                                                                                                            |
|---|--------|----------------------------------------------------------------------------------------------------------------------------------|
| a | 201    | Le message post , kind: "TEXT", flaggedContact: false                                                                            |
| b | 400    | <b>details.code: "DELIVERY_CODE_IN_MESSAGE"</b> — le serveur compare chaque groupe de six chiffres   l'empreinte du code du deal |

|   | Statut | Corps                                                                        |
|---|--------|------------------------------------------------------------------------------|
| c | 201    | <code>flaggedContact: true</code> — détecté et <b>marqué</b> , jamais bloqué |
| d | 400    | Message vide                                                                 |

**La distinction est le cœur de la fiche.** Le code de livraison est **refusé** (il se donne de vive voix) ; les coordonnées sont **marquées** (le client peut afficher un rappel discret, mais la plateforme ne censure pas un échange légitime).

**Les limites :** texte de **2 000 caractères** au plus, **5 photos** ImageKit au plus. Les dépassements répondent **400**.

**Effet de bord :** l'écriture du message, l'horodatage de la conversation et l'événement de la boîte d'envoi se font en **une seule transaction**.

**API-MSG-04 — La fenêtre d'écriture se ferme, avec son motif Gravité :** majeure.

```
for D in $(bid bzv-disputed) $(bid bzv-completed) $(bid bzv-cancelled); do
  printf '%s → ' "$D"
  curl -s -b carrier.txt "$BASE/messages/conversations/by-deal/$D" | jq -c
  '.conversation.access'
done
```

**Attendu :** `access.reason` porte une **clé stable** que le client traduit :

| Clé               | Sens                                                                        |
|-------------------|-----------------------------------------------------------------------------|
| NOT_ACCEPTED_YET  | Le deal n'est pas encore accepté (fil inexistant).                          |
| DISPUTE_OPEN      | Lecture seule pendant un litige — les échanges passent par la médiation.    |
| DEAL_CLOSED       | Le deal est terminé sans suite.                                             |
| WRITE_WINDOW_OVER | 14 jours après la fin du deal ; <code>writeClosesAt</code> annonce la date. |

**La contre-épreuve serveur :** tenter d'écrire malgré la fermeture.

```
CONV_D=$(curl -s -b carrier.txt "$BASE/messages/conversations/by-deal/$(bid bzv-disputed)" | jq
-r .conversation.id)
show -X POST "$BASE/messages/conversations/$CONV_D/messages" -b carrier.txt \
-H 'Content-Type: application/json' -d '{"body":"Je réponds quand même."}'
```

**Attendu — HTTP 400** « This conversation is read-only (DISPUTE\_OPEN). ». Le client désactive la saisie **avant** l'envoi, mais c'est le serveur qui tient la règle.

**API-MSG-05 — Marquer lu, et le compteur de non-lus Gravité :** mineure.

```
curl -s -b carrier.txt "$BASE/messages/conversations" | jq '{totalUnread, items: [.items[] |
{id, unreadCount}]}'
show -X POST "$BASE/messages/conversations/$CONV/read" -b carrier.txt
curl -s -b carrier.txt "$BASE/messages/conversations" | jq '{totalUnread}'
show -X POST "$BASE/messages/conversations/$CONV/read" -b carrier.txt      # rejeu
```

**Attendu :** **200** { `readAt` }, le compteur du fil retombe à zéro, et le **rejeu est idempotent** (200, aucun effet). Le compteur est **par lecteur** : marquer lu côté Voyageur ne touche pas le compteur de l'Expéditrice.

## API-MSG-06 — Le rendez-vous est un objet, pas un message **Gravité** : majeure.

```
# Une proposition valide (≥ 30 min à l'avance, ≤ 90 jours, créneau ≤ 12 h)
show -X POST "$BASE/messages/conversations/$CONV/meetups" -b carrier.txt -H 'Content-Type:
application/json' -d '{
  "kind":"PICKUP","placeLabel":"CDG Terminal 2E, porte 8","placeDetails":"Devant le comptoir Air
France",
  "startAt":"2026-12-10T17:30:00.000Z","endAt":"2026-12-10T18:30:00.000Z"}'
MEET=$(jq -r .id /tmp/body.json)
```

**Attendu — HTTP 201** : { id, kind: "PICKUP", status: "PROPOSED", proposedByRole: "CARRIER", ... }.

**Les cinq refus de créneau, tous en 400** avec « Invalid meeting slot (...) » :

| Code             | Règle enfreinte                       |
|------------------|---------------------------------------|
| T00_SOON         | Moins de <b>30 minutes</b> à l'avance |
| T00_FAR          | Plus de <b>90 jours</b> à l'avance    |
| WINDOW_T00_LONG  | Créneau de plus de <b>12 heures</b>   |
| END_BEFORE_START | Fin antérieure au début               |
| INVALID_DATES    | Dates non analysables                 |

```
for W in '"startAt":"2026-12-10T17:30:00.000Z","endAt":"2026-12-11T20:00:00.000Z"' \
  '"startAt":"2026-12-10T18:30:00.000Z","endAt":"2026-12-10T17:30:00.000Z"' \
  '"startAt":"pas-une-date","endAt":"pas-une-date"'; do
  curl -s -o /tmp/body.json -w '%{http_code} ' -X POST
"$BASE/messages/conversations/$CONV/meetups" \
  -b carrier.txt -H 'Content-Type: application/json' \
  -d '{"kind":"PICKUP","placeLabel":"Test",$W}'
  jq -c '.message' /tmp/body.json
done
```

## API-MSG-07 — Une contre-proposition remplace la précédente **Gravité** : majeure.

```
show -X POST "$BASE/messages/conversations/$CONV/meetups" -b shipper2.txt -H 'Content-Type:
application/json' -d '{
  "kind":"PICKUP","placeLabel":"CDG Terminal 2E, comptoir 4",
  "startAt":"2026-12-10T16:00:00.000Z","endAt":"2026-12-10T17:00:00.000Z"}'
curl -s -b shipper2.txt "$BASE/messages/conversations/$CONV" | jq '[.meetups[] | {id, kind,
status, proposedByRole}]'
```

**Attendu : 201**, et la lecture du fil montre que la proposition précédente **du même type** n'est plus **PROPOSED** : une nouvelle proposition **remplace** l'ouverte. C'est la contre-proposition, et c'est ce qui évite un fil encombré de créneaux morts.

## API-MSG-08 — On n'accepte que la proposition de l'autre **Gravité** : majeure.

```
MEET2=$(curl -s -b shipper2.txt "$BASE/messages/conversations/$CONV" | jq -r ' [.meetups[] |
select(.status=="PROPOSED")][0].id')
show -X POST "$BASE/messages/conversations/$CONV/meetups/$MEET2/accept" -b shipper2.txt # sa
PROPRE proposition
show -X POST "$BASE/messages/conversations/$CONV/meetups/$MEET2/accept" -b carrier.txt #
l'autre partie
show -X POST "$BASE/messages/conversations/$CONV/meetups/$MEET2/accept" -b carrier.txt #
rejeu
```

**Attendu :** (1) **400** « This meeting cannot be accepted (OWN\_PROPOSAL) » ; (2) **200** avec `status: "ACCEPTED"` et `acceptedAt` ; (3) **400** « ... (NOT\_PROPOSED) » — la garde optimiste répond plutôt que d'écraser.

## API-MSG-09 — Le numéro ne s'ouvre pas avant l'heure Gravit  : bloquante.

```
curl -s -b shipper2.txt "$BASE/messages/conversations/$CONV" | jq '.phone'
show -X POST "$BASE/messages/conversations/$CONV/phone" -b shipper2.txt
```

**Attendu :** le fil porte `phone: { revealed: false, phoneE164: null, opensAt: "..." }` — **le num ro est absent tant qu'il n'est pas ouvert**, seule l'**heure d'ouverture** est publi e. La demande avant l'heure r pond **HTTP 400** :

```
{ "message": "The phone number opens 2026-12-10T14:00:00.000Z.",
  "details": { "code": "T00_EARLY" } }
```

Et sans aucun point d'ancrage (ni rendez-vous accept , ni date de d part) : **400** avec `details.code: "NO_ANCHOR"` .

**La r gle :** le num ro s'ouvre **au plus t t 2 heures avant le rendez-vous de r cup ration accept ** ;   d faut de rendez-vous, avant le d part du trajet.

**Apr s l'ouverture** (  jouer quand l'heure est atteinte, sinon ►►) :

```
show -X POST "$BASE/messages/conversations/$CONV/phone" -b shipper2.txt
curl -s -b shipper2.txt "$BASE/messages/conversations/$CONV" | jq ' [.messages[] |
select(.kind=="SYSTEM") | .systemKey] '
```

**Attendu — 200** { `phoneE164`, `firstName`, `revealedAt` } . **Effets de bord :** la r v lation est **trac e** (une fois par lecteur) et **annonc e dans le fil** par un message syst me. L'ouverture n'est donc jamais silencieuse : l'autre partie sait que son num ro a  t  vu.

**Ce que le client ne doit jamais faire :** exposer un lien `tel:` direct avant l'ouverture. Le bouton « Appeler » m ne au fil avec `?focus=phone` , jamais au composeur.

## API-MSG-10 — Signaler un message de l'autre partie Gravit  : majeure.

```
MSG_AUTRE=$(curl -s -b shipper2.txt "$BASE/messages/conversations/$CONV" | jq -r ' [.messages[] |
select(.authorRole=="CARRIER" and .kind=="TEXT")][0].id')
MSG_MIEN=$(curl -s -b shipper2.txt "$BASE/messages/conversations/$CONV" | jq -r ' [.messages[] |
select(.authorRole=="SHIPPER" and .kind=="TEXT")][0].id')
MSG_SYS=$(curl -s -b shipper2.txt "$BASE/messages/conversations/$CONV" | jq -r ' [.messages[] |
select(.kind=="SYSTEM")][0].id')

show -X POST "$BASE/messages/conversations/$CONV/messages/$MSG_MIEN/report" -b shipper2.txt \
-H 'Content-Type: application/json' -d '{"reason":"OFF_PLATFORM","details":"Test"}'
show -X POST "$BASE/messages/conversations/$CONV/messages/$MSG_SYS/report" -b shipper2.txt \
-H 'Content-Type: application/json' -d '{"reason":"OFF_PLATFORM","details":"Test"}'
show -X POST "$BASE/messages/conversations/$CONV/messages/$MSG_AUTRE/report" -b shipper2.txt \
-H 'Content-Type: application/json' -d '{"reason":"OFF_PLATFORM","details":"Il propose de
régler hors de Yamba."}'
show -X POST "$BASE/messages/conversations/$CONV/messages/$MSG_AUTRE/report" -b shipper2.txt \
-H 'Content-Type: application/json' -d '{"reason":"SCAM","details":"Deuxième signalement du
même message."}'
```

**Attendu :** (1) **400 OWN\_MESSAGE** · (2) **400 NOT\_A\_TEXT** · (3) **201** { reportId, createdAt } · (4) **409** (déjà signalé par cet utilisateur).

**Motifs autorisés :** OFF\_PLATFORM, SCAM, HARASSMENT, OTHER.

**Effet de bord — le point qui distingue ce geste des transitions :** un signalement est un **dossier de modération**, pas une transition. **Aucun événement n'est émis.** Le support le voit dans sa file (GET /api/admin/conversations/reports) et sur son accueil.

**API-MSG-11 — Les réponses rapides sont traduites, à clés stables Gravité :** mineure.

```
curl -s -b shipper2.txt "$BASE/messages/quick-replies" -H 'x-locale: fr' | jq ' [.items[] | {key,
kind, text}]'
curl -s -b shipper2.txt "$BASE/messages/quick-replies" -H 'x-locale: en' | jq ' [.items[] | {key,
text}]'
show "$BASE/messages/quick-replies"
```

**Attendu :** **200** avec session, les **mêmes clés** dans les deux langues et des textes différents ; **401** sans session. Le client envoie le texte comme un message ordinaire.

**API-MSG-12 — La lecture admin d'un fil est journalisée Gravité :** majeure · **Prérequis :** une session administrateur (admin.txt) avec la permission de lecture des conversations.

```
show "$BASE/admin/conversations/by-deal/$DEAL_CONV" -b admin.txt
show "$BASE/admin/conversations/by-deal/$DEAL_CONV" -b shipper2.txt # session MEMBRE
show "$BASE/admin/conversations/reports?status=OPEN" -b admin.txt
```

**Attendu :** (1) **200** avec les deux parties, tous les messages et leurs signalements, les rendez-vous et les traces de révélation de numéro — **qui a vu le numéro et quand, jamais le numéro** ; (2) **401** — une session membre n'ouvre **aucune** route /admin/\* ; (3) **200** avec la file, ou **403 ADMIN\_PERMISSION\_DENIED** si le profil n'a pas reports.review.

**Effet de bord — c'est le point de la fiche :** la lecture (1) écrit une ligne de journal CONVERSATION\_VIEWED. Le vérifier :

```
curl -s -b admin.txt "$BASE/admin/audit" | jq -r '.items[0] | "\(.at) \(.admin) \(.action) \(.targetType) \(.targetId)'"
```

**Attendu :** la ligne la plus récente correspond à la lecture qu'on vient de faire. Une lecture sensible **non journalisée** est une anomalie **majeure**.

---

## 5.5 notification-service — boîte aux lettres et webhook email

---

**API-NOTIF-01 — Lire ses notifications** Gravit  : mineure.

```
show "$BASE/me/notifications" -b shipper.txt
curl -s -b shipper.txt "$BASE/me/notifications" | jq '{n: (.notifications|length), unreadCount,
  types: [.notifications[].type] | unique}'
```

**Attendu — HTTP 200** avec au plus **50** notifications (les plus r centes d'abord) et un `unreadCount`. Les types correspondent aux cl s d' v nements (`booking.accepted`, `booking.picked_up`, `conversation.message_posted`...).

**Un client doit pr voir un cas par d faut :** le type est une  num ration **ouverte** — les listes s'allongent, et une valeur inconnue ne doit pas faire tomber le client.

---

**API-NOTIF-02 — Marquer lu, une par une puis toutes** Gravit  : mineure.

```
N=$(curl -s -b shipper.txt "$BASE/me/notifications" | jq -r '[.notifications[] |
  select(.readAt==null)][0].id')
show -X PATCH "$BASE/me/notifications/$N/read" -b shipper.txt
show -X PATCH "$BASE/me/notifications/$N/read" -b shipper.txt          # rejeu
show -X PATCH "$BASE/me/notifications/read-all" -b shipper.txt
show -X PATCH "$BASE/me/notifications/read-all" -b shipper.txt        # rejeu
curl -s -b shipper.txt "$BASE/me/notifications" | jq '.unreadCount'
```

**Attendu : 200** partout, y compris aux rejeux — les deux marquages sont **idempotents**. Le « tout lire » renvoie **combien**  taient non lues (donc **0** au rejeu), et `unreadCount` finit   **0**.

---

**API-NOTIF-03 — La notification d'un autre membre est inaccessible** Gravit  : bloquante.

```
N_AUTRE=$(curl -s -b shipper3.txt "$BASE/me/notifications" | jq -r '.notifications[0].id')
show -X PATCH "$BASE/me/notifications/$N_AUTRE/read" -b shipper.txt
show -X PATCH "$BASE/me/notifications/00000000000000000000000000000000/read" -b shipper.txt
show -X PATCH "$BASE/me/notifications/pas-un-id/read" -b shipper.txt
show -X PATCH "$BASE/me/notifications/$N_AUTRE/read"
```

**Attendu :** (1) **403** — authentifi  mais pas destinataire ; (2) **404** ; (3) **400** ; (4) **401** sans session. Les quatre statuts sont distincts et corrects : c'est le tableau complet de la s mantique sur une seule route, et un bon test de non-r gression.

---

**API-NOTIF-04 — Les notifications refl tent une transition r elle** Gravit  : majeure · **Pr requis :** Redpanda d marr  et le consommateur actif ; sinon ►►.

```

AVANT=$(curl -s -b carrier.txt "$BASE/me/notifications" | jq '.unreadCount')
curl -s -o /dev/null -X POST "$BASE/deals" -b shipper.txt -H 'Content-Type: application/json' -d
'...' # cf. API-DEAL-04
sleep 5
APRES=$(curl -s -b carrier.txt "$BASE/me/notifications" | jq '.unreadCount')
echo "$AVANT → $APRES"
curl -s -b carrier.txt "$BASE/me/notifications" | jq '.notifications[0] | {type, createdAt}'

```

**Attendu :** le compteur du Voyageur **augmente**, et la notification la plus récente porte `booking.requested`. Le chemin complet est : transition → événement écrit **dans la même transaction** → relais vers le broker → consommateur → notification matérialisée.

**Sans broker :** les événements **s'accumulent** dans la boîte d'envoi et partent à son retour ; le compteur ne bouge pas tout de suite. Ce n'est pas une anomalie — c'est le comportement « vie sans broker » voulu. La fiche se joue alors en ►► avec cette mention.

## 6. Sécurité

Ce chapitre reprend, en un seul bloc et sous un angle d'attaquant, les gardes déjà touchées par les chapitres précédents. Il se joue **intégralement** à chaque campagne : c'est le seul dont **toutes** les fiches sont bloquantes ou majeures, et le seul qu'un rapport de recette doit reproduire ligne à ligne.

### 6.1 Accès sans jeton, jeton expiré, jeton d'autrui

**API-SEC-01 — Une route protégée sans jeton répond 401, partout Gravité : bloquante.**

```

for U in "auth/me" "auth/me/profile" "auth/me/sessions" "auth/me/sudo" "saved-routes" \
    "trips/my" "trips/favorites" "uploads/imagekit-auth" \
    "me/bookings" "me/deals" "me/wallet" "deals/$DEAL_PENDING" \
    "messages/conversations" "messages/quick-replies" "me/notifications"; do
    printf '%-28s → %s\n' "$U" "$(curl -s -o /dev/null -w '%{http_code}' "$BASE/$U")"
done

```

**Attendu : 401** pour les quinze. Un **200** sur l'une d'elles est une anomalie **bloquante** : une ressource privée serait publique. Un **500** est une anomalie **majeure** : l'intergiciel d'authentification doit refuser proprement, pas planter.

**API-SEC-02 — Un jeton expiré, invalide ou tronqué répond 401 Gravité : bloquante.**

```

for T in "aaa.bbb.ccc" "" "Bearer" "${awk '/access_token/ {print $7}' shipper.txt | head -1 |
cut -c1-20}"; do
    printf '[%.20s] → %s\n' "$T" "$(curl -s -o /dev/null -w '%{http_code}' "$BASE/auth/me" -H
"Authorization: Bearer $T")"
done

```

**Attendu : 401** pour tous, corps plat `{ "message": "Unauthorized! Token expired or invalid." }`.

**Le jeton expiré** se teste en attendant 15 minutes après une connexion, ou en réduisant la durée de vie sur un environnement de recette. Le comportement attendu d'un client est alors : **un seul** rafraîchissement, puis rejeu ; en cas d'échec du rafraîchissement, passage à l'état déconnecté sans boucler (le front porte un disjoncteur de 30 secondes exactement pour cela).

**Cas particulier des routes à authentification optionnelle** (recherche, trajet public, profil public) : un jeton expiré ne donne **jamais 401** — la route répond en mode « non connecté ». Le vérifier :

```
curl -s -o /dev/null -w '%{http_code}\n' "$BASE/trips/search?limit=1" -H 'Authorization: Bearer aaa.bbb.ccc'
curl -s -o /dev/null -w '%{http_code}\n' "$BASE/users/$SLUG/public" -H 'Authorization: Bearer aaa.bbb.ccc'
```

**Attendu : 200** pour les deux.

### API-SEC-03 — Le jeton d'un autre membre n'ouvre que ce qui lui appartient Gravit  : bloquante.

C'est le test de **travers e horizontale** : la m me route, le m me verbe, un identifiant qui appartient   quelqu'un d'autre.

```
for D in "$DEAL_PENDING" "$DEAL_PICKED" "$DEAL_DELIVERED"; do
  printf 'GET deals/%s (tiers) → %s\n' "$D" "$(curl -s -o /dev/null -w '%{http_code}' "$BASE/deals/$D" -b shipper3.txt)"
done
printf 'POST accept (tiers) → %s\n' "$(curl -s -o /dev/null -w '%{http_code}' -X POST "$BASE/deals/$DEAL_PENDING/accept" -b shipper3.txt -H 'Content-Type: application/json' -d '{"charterAccepted":true}')"
printf 'POST cancel (tiers) → %s\n' "$(curl -s -o /dev/null -w '%{http_code}' -X POST "$BASE/deals/$DEAL_PENDING/cancel" -b shipper3.txt -H 'Content-Type: application/json' -d '{}')"
printf 'POST deliver (tiers) → %s\n' "$(curl -s -o /dev/null -w '%{http_code}' -X POST "$BASE/deals/$DEAL_PICKED/deliver" -b shipper3.txt -H 'Content-Type: application/json' -d '{"code":"742891","photoUrls":[]}')"
printf 'POST link (tiers) → %s\n' "$(curl -s -o /dev/null -w '%{http_code}' -X POST "$BASE/deals/$DEAL_PICKED/tracking-link" -b shipper3.txt)"
printf 'GET conv (tiers) → %s\n' "$(curl -s -o /dev/null -w '%{http_code}' "$BASE/messages/conversations/$CONV" -b shipper3.txt)"
printf 'PUT trip (tiers) → %s\n' "$(curl -s -o /dev/null -w '%{http_code}' -X PUT "$BASE/trips/$TRIP_PERKG" -b shipper3.txt -H 'Content-Type: application/json' -d '{"pricePerKgCents":1}')"

```

**Attendu : 403** partout (400 pour la route trip,  cart de s mantique connu). **Aucun 200**. Et surtout : **aucun corps ne contient de donn e de la ressource vis e**.

**Le contr le d cisif** : v rifier qu'aucune de ces r ponses ne diff re selon que le deal existe ou non. Comparer avec un identifiant invent  de m me forme.

### API-SEC-04 — Une session membre n'ouvre aucune route d'administration Gravit  : bloquante.

```
for U in "admin/me" "admin/kpis" "admin/audit" "admin/users" "admin/reports" "admin/settings" \
  "admin/disputes" "admin/finances/queue" "admin/trips" "admin/tickets" \
  "admin/conversations/reports" "admin/alerts" "admin/status" "admin/maintenance"; do
  printf '%-30s membre=%s admin=%s\n' "$U" \
    "$(curl -s -o /dev/null -w '%{http_code}' "$BASE/$U" -b shipper.txt)" \
    "$(curl -s -o /dev/null -w '%{http_code}' "$BASE/$U" -b admin.txt)"
done
```

**Attendu** : colonne « membre » = **401** partout, **sans exception**. C'est la garde structurelle du back-office : l'intergiciel d'administration lit **uniquement** le cookie `admin_access_token` (ou un Bearer), **jamais** `access_token`. Un compte administrateur connect  comme membre ordinaire n'ouvre **rien**.

La colonne « admin » vaut **200** ou **403** selon le profil — jamais 401 si la session est valide.

**Deux exigences supplémentaires du jeton admin**, à vérifier avec un jeton membre présenté dans le cookie admin : le JWT doit porter `adm: true` et `"totp"` dans `amr`. Un jeton membre recopié dans `admin_access_token` répond donc **401** « Unauthorized! Admin session required. » — et non 403, car la session n'est pas seulement insuffisante : elle n'est pas une session admin.

---

**API-SEC-05 — Un profil admin sans la permission reçoit 403, jamais 404** Gravité : bloquante · Prérequis : une session admin de profil `SUPPORT`.

```
curl -s -b admin.txt "$BASE/admin/me" | jq '{adminRole, adminRoles}'
show "$BASE/admin/finances/report" -b admin.txt
show "$BASE/admin/users/export?reason=Audit%20de%20recette%20du%20cahier%20numero%20trois" -b
admin.txt
```

**Attendu** : un profil `SUPPORT` reçoit **403** sur les routes financières et sur l'export de données personnelles, avec

```
{ "message": "Your admin profile does not allow this action.",
  "code": "ADMIN_PERMISSION_DENIED", "permission": "finances.read" }
```

**Le point de la fiche** : c'est un **403 explicite**, avec le nom de la permission — **jamais un 404**. La route **existe** ; c'est le profil qui manque. Un 404 ici serait une régression de diagnostic : le back-office ne pourrait plus expliquer à un opérateur pourquoi son action est refusée.

`SUPER_ADMIN` passe partout. Les profils cumulés donnent l'**union** des permissions — le vérifier en ajoutant un second profil à un compte et en jouant la même route.

---

**API-SEC-06 — Compte restreint et compte suspendu** Gravité : bloquante · Prérequis : un administrateur habilité pour poser la sanction.

```
# Restreindre le compte (depuis l'administration)
curl -s -b admin.txt -X POST "$BASE/admin/users/${uid aminata}/suspension" -H 'Content-Type:
application/json' \
  -d '{"level":"RESTRICTED","reason":"Vérification de la garde de compte restreint, recette API
du cahier trois."}'

# Les routes de CRÉATION sont fermées
show -X POST "$BASE/trips" -b shipper.txt -H 'Content-Type: application/json' -d
'{"transportMode":"PLANE"}'
show -X POST "$BASE/deals/payment-intents" -b shipper.txt -H 'Content-Type: application/json' -d
"
{"tripId\":"$TRIP_PERKG","\product\":"$PARCEL","\family\":"$CLOTHES_TEXTILE","\sizeClass\":"
"M","\weightKg\":"1,"\protection\":"$BASIC","\expectedTotalCents\":"1}"
# Les deals EN COURS continuent
show "$BASE/deals/$DEAL_PICKED" -b shipper.txt
show "$BASE/me/bookings" -b shipper.txt
```

**Attendu** : les deux créations répondent **403** `{ "code": "ACCOUNT_RESTRICTED" }`, et les lectures répondent **200**. La sanction est **exactement** cela : plus de publication ni de réservation, mais les engagements en cours continuent.

**Suspension** :

```
curl -s -b admin.txt -X POST "$BASE/admin/users/${uid aminata}/suspension" -H 'Content-Type: application/json' \
  -d '{"level":"SUSPENDED","reason":"Vérification de la garde de compte suspendu, recette API du cahier trois."}'
show "$BASE/auth/me" -b shipper.txt
show -X POST "$BASE/auth/login" -H 'Content-Type: application/json' \
  -d '{"email":"aminata.shipper@seed.yamba.dev","password":"Yamba-Dev-2026!"}'
```

**Attendu : 401** avec code: "ACCOUNT\_SUSPENDED" sur toute lecture, et **401 indistinguable d'un mot de passe faux** à la connexion (cf. API-AUTH-08).

**Le point d'architecture à vérifier :** les sanctions n'agissent **que par les lectures** — aucune écriture inter-services n'est déclenchée. Les deals du membre ne sont pas modifiés en base ; c'est la lecture qui refuse.

**Effet de bord : lever la sanction** avant de continuer (DELETE /api/admin/users/{id}/suspension).

---

## 6.2 La fenêtre sensible

---

**API-SEC-07 — Les cinq gestes sensibles sont tous gardés Gravité : bloquante.**

```
show -X POST "$BASE/auth/me/password" -b shipper.txt -H 'Content-Type: application/json' \
  -d '{"currentPassword":"Yamba-Dev-2026!","newPassword":"Autre-Mot-2026!"}'
show -X POST "$BASE/auth/me/email/request" -b shipper.txt -H 'Content-Type: application/json' \
  -d '{"newEmail":"nouvelle.adresse@seed.yamba.dev"}'
show -X POST "$BASE/carrier/stripe/dashboard-link" -b carrier.txt
show -X POST "$BASE/auth/me/data-export" -b shipper.txt
show -X POST "$BASE/auth/me/erasure" -b shipper.txt -H 'Content-Type: application/json' -d
  '{"confirmation":"SUPPRIMER"}'
```

**Attendu : 403 SUDO\_REQUIRED** pour les cinq, fenêtre fermée. Aucun de ces gestes ne doit passer sans la fenêtre : ce sont exactement les cinq points par lesquels un attaquant disposant d'un poste déverrouillé prendrait le contrôle d'un compte ou en exfiltrerait les données.

**La fenêtre expire :** ouvrir la fenêtre, attendre **15 minutes**, rejouer un geste. **Attendu :** retour du 403. Une fenêtre qui ne se referme pas est une anomalie **bloquante**.

---

## 6.3 Les secrets qui ne doivent jamais sortir

---

**API-SEC-08 — Le code de livraison, balayage complet Gravité : bloquante.**

Un seul balayage, sur **toutes** les surfaces du cahier, avec le code connu du jeu d'essai.

```
D=$(bid bzv-picked)
{
  curl -s -b carrier.txt "$BASE/deals/$D"
  curl -s -b carrier.txt "$BASE/me/deals"
  curl -s -b carrier.txt "$BASE/deals?tripId=$(tid bzv-inflight)"
  curl -s -b shipper.txt "$BASE/me/bookings"
  curl -s -b carrier.txt "$BASE/me/wallet"
  curl -s -b shipper.txt "$BASE/messages/conversations"
  curl -s -b carrier.txt "$BASE/messages/conversations/by-deal/$DEAL_CONV"
  curl -s -b shipper.txt "$BASE/me/notifications"
  curl -s "$BASE/track/$TOK"
  curl -s -b admin.txt "$BASE/admin/conversations/by-deal/$DEAL_CONV"
  curl -s -b admin.txt "$BASE/admin/deals/$D/money"
  curl -s -b shipper.txt -X POST "$BASE/auth/me/data-export"
} | grep -c 742891
```

**Attendu : 0 .**

**La seule surface autorisée**, à vérifier séparément :

```
curl -s -b shipper.txt "$BASE/deals/$D" | jq '.deal.deliveryCode' # → "742891"
```

C'est-à-dire : vue Expéditeur, détail d'un deal, statut PICKED\_UP . Plus la réponse de POST /deals/{id}/code/regenerate , qui rend le **nouveau** code à l'Expéditeur. **Toute autre occurrence est une anomalie bloquante**, y compris dans un email et dans un événement.

Compléter par le balayage des emails, si Mailpit est en place :

```
curl -s "http://localhost:8025/api/v1/messages" | grep -c 742891 # attendu : 0
```

## API-SEC-09 — Le destinataire n'est jamais exposé au Voyageur ni au public Gravité : bloquante.

Le destinataire est un **tiers sans compte**, qui n'a rien consenti. Ses coordonnées ne circulent qu'entre l'Expéditeur et la plateforme.

```
D=$(bid bzv-picked)
# a) Ce que le VOYAGEUR voit
curl -s -b carrier.txt "$BASE/deals/$D" | jq '{recipient: .deal.recipient,
  fuites: [(.deal|paths(scalars) as $p |
select((getpath($p)|tostring)|test("\\+242|clarisse@";"i")) | ($p|join(".")))]}'
# b) Ce que la PAGE PUBLIQUE montre
curl -s "$BASE/track/$TOK" | jq '{recipientFirstName,
  fuites: [paths(scalars) as $p | select((getpath($p)|tostring)|test("\\+242|@Mabiala;";"i")) |
($p|join(".")))}'
# c) Ce que l'EXPÉDITEUR voit (légitime)
curl -s -b shipper.txt "$BASE/deals/$D" | jq '.deal.recipient'
```

**Attendu : (a) fuites: []** — la vue Voyageur ne porte **ni téléphone, ni email, ni nom de famille** du destinataire ; (b) **fuites: []** — la page publique ne porte que le **prénom** ; (c) l'instantané complet, légitime pour l'Expéditeur qui l'a saisi.

**Le seul canal d'exposition prévu** est le lien de suivi que l'Expéditeur crée et **partage lui-même** — et son contenu est minimal par construction (API-DEAL-22).

**Rétention** : après le délai de conservation, l'instantané du destinataire est **rédigé** par une tâche planifiée, et le lien de suivi répond alors **404**. À vérifier dans le cahier n° 4.

---

#### API-SEC-10 — Aucun secret technique ne sort d'une réponse Gravité : bloquante.

```
{
  curl -s -b shipper.txt "$BASE/auth/me"
  curl -s -b shipper.txt "$BASE/auth/me/profile"
  curl -s -b shipper.txt "$BASE/auth/me/sessions"
  curl -s          "$BASE/users/$SLUG/public"
  curl -s -b admin.txt "$BASE/admin/me"
  curl -s -b admin.txt "$BASE/admin/users/$(uid aminata)"
} | grep -o -iE
'passwordHash|totpSecret|totpEncrypted|backupCodes|deliveryCodeHash|deliveryCodeEncrypted|client
Secret|sk_live|sk_test|STRIPE_[A-Z_]+|refreshToken' | sort -u
```

**Attendu : aucune ligne.** Aucune empreinte de mot de passe, aucun secret TOTP, aucun code de secours, aucune empreinte de code de livraison, aucune clé de fournisseur ne doit apparaître dans une réponse d'API.

**Le cas de `clientSecret`** : il apparaît **légitimement** dans la réponse de `POST /deals/payment-intents` avec le fournisseur Stripe — c'est son rôle, il sert une seule fois côté client. Il ne doit apparaître **nulle part ailleurs**, et **jamais** dans une lecture de deal.

---

#### API-SEC-11 — Les traces de pile ne sortent jamais Gravité : bloquante.

```
grep -o -iE '"stack"|at Object\.|node_modules/|/Users/|/home/|\.ts:[0-9]+' /tmp/*.json
2>/dev/null | sort -u
```

**Attendu : aucune ligne** sur l'ensemble des corps collectés pendant la campagne. L'intergiciel d'erreurs ne sérialise **jamais** de trace, dans aucun environnement. Une trace visible est une anomalie **bloquante** : elle divulgue l'arborescence du serveur et les versions de bibliothèques.

---

## 6.4 Signatures de webhooks

---

#### API-SEC-12 — Une signature de webhook email invalide est rejetée Gravité : bloquante.

```
# a) Sans en-tête de signature
show -X POST "$BASE/webhooks/email/resend" -H 'Content-Type: application/json' \
  -d '{"type":"email.bounced","data":{"email_id":"x","to":
["aminata.shipper@seed.yamba.dev"],"bounce":{"type":"hard"}}}'
# b) Avec une signature bidon
show -X POST "$BASE/webhooks/email/resend" -H 'Content-Type: application/json' \
  -H 'svix-id: msg_recette' -H 'svix-timestamp: 1760000000' -H 'svix-signature: v1,ZmFleA==' \
  -d '{"type":"email.bounced","data":{"email_id":"x","to":
["aminata.shipper@seed.yamba.dev"],"bounce":{"type":"hard"}}}'
```

**Attendu : 401** { "message": "Invalid signature.", "reason": "..." } pour les deux — ou **503** si `RESEND_WEBHOOK_SECRET` n'est pas configuré (l'endpoint refuse plutôt que d'accepter sans signature ; la fiche est alors ➡ pour la partie signature valide).

**Effet de bord** — c'est le point de la fiche : l'adresse visée n'est **PAS** mise sur la liste de suppression.

```
curl -s -b admin.txt "$BASE/admin/users/${uid aminata}" | jq '{emailSuppressedAt, emailSuppressedReason}'
```

**Attendu :** `null` pour les deux. Un webhook non signé qui produirait un effet permettrait à n'importe qui de **couper les emails** d'un membre arbitraire — c'est un déni de service, et il serait **bloquant**.

---

#### API-SEC-13 — Le webhook Stripe porte sur les octets bruts Gravité : bloquante.

```
# a) Sans en-tête de signature (appel DIRECT au deal-service – hors passerelle, par conception)
show -X POST "http://localhost:6003/webhooks/stripe" -H 'Content-Type: application/json' \
  -d '{"type":"payment_intent.canceled","data":{"object":{"id":"pi_test"}}}'
# b) Avec une signature bidon
show -X POST "http://localhost:6003/webhooks/stripe" -H 'Content-Type: application/json' \
  -H 'stripe-signature: t=1760000000,v1=faux' \
  -d '{"type":"payment_intent.canceled","data":{"object":{"id":"pi_test"}}}'
```

**Attendu :** **400** pour les deux (en-tête absent ou signature invalide — inutile de réessayer), ou **501** si `STRIPE_WEBHOOK_SECRET` n'est pas configuré. **Jamais 200**.

**Pourquoi cette route ne passe pas par la passerelle :** la signature Stripe porte sur les **octets bruts** du corps, que la passerelle analyse et re-séréalise — ce qui casserait la signature. La route est donc montée **avant** l'analyseur JSON du deal-service, sur un lecteur de corps brut. Cette exception est **la seule** de tout le cahier, et elle est assumée : c'est un appel serveur à serveur, il n'y a pas de navigateur pour l'émettre.

**Le point à vérifier après une signature valide :** rejouer **exactement** le même corps une seconde fois ne doit produire **aucun second effet** (cf. API-IDEM-05).

---

## 6.5 Non-divulgateion

#### API-SEC-14 — Aucune énumération possible Gravité : bloquante.

```
for I in 00000000000000000000000000000000 11111111111111111111111111111111 ffffffffffffffffffffffffff; do
  printf '%s deal=%s trip=%s user=%s\n' "$I" \
    "$(curl -s -o /dev/null -w '%{http_code}' "$BASE/deals/$I" -b shipper.txt)" \
    "$(curl -s -o /dev/null -w '%{http_code}' "$BASE/trips/$I/public)" \
    "$(curl -s -o /dev/null -w '%{http_code}' "$BASE/users/slug-$I/public)"
done
# Puis, avec des identifiants qui EXISTENT mais ne sont pas visibles pour l'appelant
printf 'deal existant, tiers → %s\n' "$(curl -s -o /dev/null -w '%{http_code}'
"$BASE/deals/$DEAL_PICKED" -b shipper3.txt)"
```

**Attendu :** les identifiants inexistantes donnent **404** (403 sur un deal existant pour un tiers, puisque la relation est la garde et que l'existence du deal n'est pas un secret vis-à-vis d'un membre authentifié). **Le contrôle décisif** est ailleurs : sur les routes **publiques** (trajet, profil, suivi), « inexistant » et « invisible » doivent donner **exactement le même 404** — même statut, même corps.

**Un temps de réponse notablement différent** entre les deux cas est aussi une fuite, à consigner en **majeur** (mesurer avec `curl -w '%{time_total}'` sur une vingtaine d'appels de chaque type).

---

#### API-SEC-15 — Le CORS n'autorise que les origines déclarées Gravité : majeure.

```
curl -s -o /dev/null -w '%{http_code}\n' "$BASE/maintenance" #
sans Origin
curl -s -o /dev/null -w '%{http_code}\n' "$BASE/maintenance" -H 'Origin: http://localhost:3000'
curl -s -o /dev/null -w '%{http_code}\n' "$BASE/maintenance" -H 'Origin: http://localhost:3001'
curl -s -o /dev/null -w '%{http_code}\n' "$BASE/maintenance" -H 'Origin: http://exemple-
malveillant.test'
```

**Attendu : 200** sans `Origin` (curl, serveur à serveur — accepté par conception), **200** pour les deux fronts déclarés (3000 et 3001, sur `localhost`, `192.168.x.x` ou `10.x.x.x`), et un **refus** pour l'origine inconnue (« Not allowed by CORS »).

**À retenir pour les campagnes futures** : un nouveau front, ou un même front sur un nouveau port, doit être ajouté à la liste de la passerelle — sinon **tous** ses appels échouent, y compris à travers le proxy du serveur Next (l'en-tête `Origin` du navigateur traverse le proxy).

---

## 7. Idempotence et concurrence

Ce chapitre répond à une seule question : **que se passe-t-il quand le même geste arrive deux fois ?** — parce que le réseau a bégayé, parce que l'utilisateur a double-cliqué, parce qu'un client mobile a rejoué une requête en zone blanche, ou parce que deux personnes agissent en même temps.

La réponse attendue est toujours l'une des deux suivantes, jamais une troisième :

- **le geste est idempotent** : le second appel répond comme le premier, sans doubler l'effet ;
- **le geste est conditionnel** : le second appel reçoit un **conflit typé**, et rien n'a bougé.

### 7.1 Rejeu séquentiel

---

**API-IDEM-01 — Une intention de paiement ne sert qu'une fois** **Gravité : bloquante** · **Prérequis** : un deal créé avec l'intention `$PI` (API-DEAL-04).

```
show -X POST "$BASE/deals" -b shipper.txt -H 'Content-Type: application/json' -d "{
  \"tripId\": \"$TRIP_PERKG\", \"paymentIntentId\": \"$PI\", \"expectedTotalCents\": $TOTAL,
  \"product\": \"PARCEL\", \"family\": \"CLOTHES_TEXTILE\", \"sizeClass\": \"M\", \"weightKg\": 3,
  \"protection\": \"BASIC\", \"description\": \"Rejeu volontaire de la création.\",
  \"declaredValueCents\": 15000, \"photoUrls\": [],
  \"recipient\":
    { \"firstName\": \"Marie\", \"lastName\": \"Kabeya\", \"phoneE164\": \"+243812345678\", \"email\": \"marie.k@example.com\" },
  \"charterAccepted\": true, \"termsAccepted\": true}"
curl -s -b shipper.txt "$BASE/me/bookings" | jq '[.bookings[] |
  select(.paymentIntentId=="'$PI'")] | length'
```

**Attendu — HTTP 409 PAYMENT\_ALREADY\_USED**, et un **seul** deal portant cette intention.

**La règle pour un client** : perdre la réponse d'un `POST /deals` n'autorise **jamais** à recréer. Le bon geste est de **relire** `GET /me/bookings` : le deal existe très probablement déjà. Recréer produirait un double débit.

---

**API-IDEM-02 — Rejouer une acceptation ne réaccepte pas** **Gravité : bloquante**.

```
D=$(bid gru-pending)
show -X POST "$BASE/deals/$D/accept" -b ines.txt -H 'Content-Type: application/json' -d
 '{"charterAccepted":true}'
ACCEPT_E_A=$(curl -s -b ines.txt "$BASE/deals/$D" | jq -r '.deal.acceptedAt')
show -X POST "$BASE/deals/$D/accept" -b ines.txt -H 'Content-Type: application/json' -d
 '{"charterAccepted":true}'
ACCEPT_E_B=$(curl -s -b ines.txt "$BASE/deals/$D" | jq -r '.deal.acceptedAt')
echo "$ACCEPT_E_A / $ACCEPT_E_B"
```

**Attendu :** premier appel **200**, second **409 TRANSITION\_NOT\_ALLOWED**, et surtout **acceptedAt** **identique avant et après**. La date d'acceptation ne bouge pas, et le paiement **n'est pas capturé deux fois** — le vérifier sur la fiche argent de l'administration (GET /api/admin/deals/{id}/money) : une seule capture.

**Le mécanisme :** toute transition est une **écriture conditionnelle** — statut attendu + verrou optimiste sur l'horodatage de mise à jour. Rejouer n'est donc jamais destructeur.

### API-IDEM-03 — Rejouer une remise ne relivre pas Gravit  : bloquante.

```
D=$(bid bzv-picked)
show -X POST "$BASE/deals/$D/deliver" -b carrier.txt -H 'Content-Type: application/json' \
 -d '{"code":"742891","photoUrls":[]}'
LIVRE_A=$(curl -s -b carrier.txt "$BASE/deals/$D" | jq -r '.deal.deliveredAt')
show -X POST "$BASE/deals/$D/deliver" -b carrier.txt -H 'Content-Type: application/json' \
 -d '{"code":"742891","photoUrls":[]}'
LIVRE_B=$(curl -s -b carrier.txt "$BASE/deals/$D" | jq -r '.deal.deliveredAt')
echo "$LIVRE_A / $LIVRE_B"
```

**Attendu :** premier **200**, second **409 TRANSITION\_NOT\_ALLOWED**, **deliveredAt** **inchang **, et **payoutDueAt** **non repouss **. Un rejeu qui d calerait la fen tre de v rification volerait quatre jours   l'Exp diteur.

**Point d'attention suppl mentaire :** le rejeu ne doit **pas** consommer une tentative de code (le compteur de tentatives rat es reste   z ro). Le v rifier en encha nant un mauvais code apr s le rejeu : **attemptsLeft** doit valoir **2**, pas moins.

### API-IDEM-04 — Les gestes d clar s idempotents le sont vraiment Gravit  : majeure.

```
idem () { # idem <verbe> <chemin> <pot>  → deux appels, deux codes
  printf '%-46s %s %s\n' "$2" \
    "$(curl -s -o /dev/null -w '%{http_code}' -X "$1" "$BASE$2" -b "$3")" \
    "$(curl -s -o /dev/null -w '%{http_code}' -X "$1" "$BASE$2" -b "$3")"
}
idem POST    "/trips/$TRIP_PERKG/favorite"      shipper.txt
idem DELETE  "/trips/$TRIP_PERKG/favorite"      shipper.txt
idem POST    "/users/$SLUG/follow"              shipper3.txt
idem DELETE  "/users/$SLUG/follow"              shipper3.txt
idem PATCH   "/me/notifications/read-all"      shipper.txt
idem POST    "/messages/conversations/$CONV/read"  carrier.txt
idem POST    "/deals/$DEAL_PICKED/tracking-link"  shipper.txt
```

**Attendu :** **le m me code aux deux appels** sur chaque ligne (200 ou 201), et **aucun doublon** cr  . En particulier, le lien de suivi renvoie **le m me jeton** deux fois : il n'y a **qu'un lien par deal**.

**API-IDEM-05 — Rejouer un événement consommé ne le rejoue pas deux fois** Gravité : majeure · **Prérequis** : Redpanda et le consommateur actifs, sinon ►►.

Le consommateur **revendique** l'identifiant de l'événement **avant** tout traitement. Un doublon est donc ignoré silencieusement, et la matérialisation des notifications est de toute façon un « insère ou met à jour » sur le couple (événement, destinataire).

**Preuve la plus simple, par le webhook email — qui est idempotent par contrat :**

```
# Deux fois exactement le même événement signé
for i in 1 2; do
  curl -s -o /tmp/body.json -w '%{http_code} ' -X POST "$BASE/webhooks/email/resend" \
    -H 'Content-Type: application/json' \
    -H "svix-id: msg_recette_idem" -H "svix-timestamp: $(date +%s)" -H "svix-signature: $SIG" \
    --data-binary @/tmp/evenement.json
  jq -c '{ok, status, suppressed}' /tmp/body.json
done
```

**Attendu : 200** aux deux appels ; `suppressed: true` au premier et `suppressed: false` au second — l'effet ne s'applique qu'une fois. Un événement de type inconnu répond `{ "ok": true, "ignored": "<type>" }`, en **200** : un webhook n'est jamais une erreur pour l'émetteur, sinon il retenterait indéfiniment.

**Côté événements internes**, la preuve se fait dans le cahier n° 4 en relisant la collection des événements consommés : une seule ligne par couple (groupe de consommateurs, identifiant d'événement).

---

## 7.2 Concurrency réelle

---

**API-IDEM-06 — Deux acceptations simultanées : une seule gagne** Gravité : bloquante.

Le cas réel : le Voyageur double-clique, ou son téléphone et son ordinateur envoient la requête en même temps.

```
D=$(bid bzv-pending)
for i in 1 2 3; do
  curl -s -o "/tmp/acc$i.json" -w "%{http_code}\n" -X POST "$BASE/deals/$D/accept" \
    -b carrier.txt -H 'Content-Type: application/json' -d '{"charterAccepted":true}' &
done; wait
jq -c '{status, code: .details.code}' /tmp/acc1.json /tmp/acc2.json /tmp/acc3.json
```

**Attendu : exactement un 200**, et deux **409** `TRANSITION_NOT_ALLOWED`.

**Le contrôle décisif est l'effet de bord :**

```
curl -s -b admin.txt "$BASE/admin/deals/$D/money" | jq '{captures: [.movements[]? |
select(.kind=="CAPTURE")] | length}'
```

**Attendu : une seule capture.** Deux gagnants, ou deux captures, seraient une anomalie **bloquante** : l'Expéditeur serait débité deux fois. Le mécanisme qui l'interdit est le verrou optimiste sur l'horodatage de mise à jour, à l'intérieur de la transaction.

---

**API-IDEM-07 — Deux réservations simultanées sur les derniers kilos** Gravité : bloquante.

```
# Deux Expéditeurs réservent en même temps un poids qui ne tient pas deux fois
RESTE=$(curl -s "$BASE/trips/$TRIP_PERKG/public" | jq -r '.trip.remainingKg')
POIDS=$(( RESTE )) # tout le reste : une seule des deux demandes peut passer
# ... créer deux intentions (une par Expéditeur), puis :
( curl -s -o /tmp/b1.json -w '%{http_code}\n' -X POST "$BASE/deals" -b shipper.txt -H 'Content-Type: application/json' -d "$CORPS1" ) &
( curl -s -o /tmp/b2.json -w '%{http_code}\n' -X POST "$BASE/deals" -b shipper3.txt -H 'Content-Type: application/json' -d "$CORPS2" ) &
wait
jq -c '{bookingId, code: .details.code}' /tmp/b1.json /tmp/b2.json
curl -s "$BASE/trips/$TRIP_PERKG/public" | jq '.trip.remainingKg'
```

**Attendu :** un **201** et un **409 CAPACITY\_EXCEEDED**. Et surtout : **remainingKg** ne devient jamais négatif. La réservation des kilos est une **écriture conditionnelle** dans la même transaction que la création du deal.

**L'invariant à retenir :** les kilos réservés d'un trajet valent toujours la somme des poids de ses deals **actifs** ( **PENDING** , **ACCEPTED** , **PICKED\_UP** , **DELIVERED** , **DISPUTED** ). Le vérifier après la fiche :

```
curl -s -b carrier.txt "$BASE/deals?tripId=$TRIP_PERKG" \
  | jq '[.deals[] | select(.status|IN("PENDING","ACCEPTED","PICKED_UP","DELIVERED","DISPUTED")) | .weightKg] | add'
```

#### API-IDEM-08 — Deux acceptations de rendez-vous simultanées Gravit  : majeure.

```
M=$(curl -s -b shipper2.txt "$BASE/messages/conversations/$CONV" | jq -r '[.meetups[] | select(.status=="PROPOSED")][0].id')
for i in 1 2; do
  curl -s -o "/tmp/m$i.json" -w '%{http_code}\n' -X POST \
    "$BASE/messages/conversations/$CONV/meetups/$M/accept" -b shipper2.txt &
done; wait
jq -c '{status, message}' /tmp/m1.json /tmp/m2.json
```

**Attendu :** un **200** avec `status: "ACCEPTED"`, un **400** « This meeting was just changed » — la garde optimiste **répond** plutôt que d'écraser. `acceptedAt` ne doit exister qu'une fois.

#### API-IDEM-09 — Deux régénérations simultanées ne comptent qu'une fois Gravit  : majeure.

```
D=$(bid bzv-tracking)
AVANT=$(curl -s -b shipper2.txt "$BASE/deals/$D" | jq -r '.deal.codeRegenerationsLeft // empty')
for i in 1 2; do
  curl -s -o "/tmp/r$i.json" -w '%{http_code}\n' -X POST "$BASE/deals/$D/code/regenerate" -b shipper2.txt &
done; wait
jq -c '{deliveryCode: (.deliveryCode|type), left: .codeRegenerationsLeft, code: .details.code}' /tmp/r1.json /tmp/r2.json
```

**Attendu :** un **seul** appel réussit, et le compteur ne descend **que d'une unité**. Deux clics égalent une régénération : c'est la garde optimiste sur le compteur.

#### API-IDEM-10 — Deux rafraîchissements de session simultanés Gravit  : majeure.

```
cp shipper.txt r1.txt; cp shipper.txt r2.txt
( curl -s -o /dev/null -w '%{http_code}\n' -X POST "$BASE/auth/refresh" -b r1.txt -c r1.txt ) &
( curl -s -o /dev/null -w '%{http_code}\n' -X POST "$BASE/auth/refresh" -b r2.txt -c r2.txt ) &
wait
curl -s -o /dev/null -w 'r1 → %{http_code}\n' "$BASE/auth/me" -b r1.txt
curl -s -o /dev/null -w 'r2 → %{http_code}\n' "$BASE/auth/me" -b r2.txt
```

**Attendu :** au plus **une** session utilisable ensuite. Le second rafraîchissement, s'il a lu le même identifiant de session révoqué entre-temps, répond **401** — ce qui est correct : un jeton de rafraîchissement ne sert qu'une fois.

**La conséquence pour un client :** ne lancer **qu'un seul** rafraîchissement à la fois et mettre les requêtes concurrentes en file d'attente. C'est exactement ce que fait le client du front, avec en plus un disjoncteur de 30 secondes après un échec — sans quoi une page montant dix composants authentifiés déclencherait dix cycles « 401 → rafraîchir → 401 ».

## 8. Webhooks

Deux flux entrants, deux mécanismes de signature, deux chemins d'accès différents.

### 8.1 Stripe — un appel serveur à serveur, hors passerelle

La route est `POST http://localhost:6003/webhooks/stripe`. **Elle ne passe jamais par la passerelle**, pour une raison technique précise : la signature porte sur les **octets bruts** du corps, et la passerelle analyse puis re-séréalise le JSON, ce qui la casserait. Côté deal-service, la route est montée **avant** l'analyseur JSON, sur un lecteur de corps brut.

**Deux points d'entrée Stripe, une seule URL :** les événements de la plateforme et ceux des comptes connectés arrivent au même endroit ; la signature est essayée avec l'un des deux secrets, puis avec l'autre.

**API-HOOK-01 — Mettre en place l'écoute locale Gravit  :** majeure · **Pr requis :** la CLI Stripe et un compte de test ; sinon ►►.

```
stripe login
stripe listen --forward-to localhost:6003/webhooks/stripe
# La CLI affiche le secret de signature : le poser dans .env
# STRIPE_WEBHOOK_SECRET=whsec_...
# puis red marrer le deal-service.
```

**Attendu :** la CLI affiche « Ready! Your webhook signing secret is whsec\_... ». Chaque  v nement transmis appara t dans son journal avec le code renvoy  par le service.

**API-HOOK-02 — Les quatre r ponses possibles du webhook Stripe Gravit  :** bloquante.

| R ponse                       | Sens                                                               | Cons quence c t  Stripe                                                    |
|-------------------------------|--------------------------------------------------------------------|----------------------------------------------------------------------------|
| <b>200</b> { received: true } | Trait , ou <b>ignor  volontairement</b> (type non pris en charge). | Ne renverra pas.                                                           |
| <b>400</b>                    | En-t te absent ou signature invalide.                              | Aucun r essai utile.                                                       |
| <b>501</b>                    | <code>STRIPE_WEBHOOK_SECRET</code> absent (fournisseur factice).   | L'endpoint <b>existe</b> mais refuse plut t que d'accepter sans signature. |

| Réponse | Sens                                   | Conséquence côté Stripe                        |
|---------|----------------------------------------|------------------------------------------------|
| 500     | Échec transitoire (base indisponible). | <b>Stripe réessaie</b> — c'est le filet voulu. |

```
stripe trigger payment_intent.canceled
stripe trigger account.updated
# Un type non traité : doit répondre 200 sans effet
stripe trigger charge.succeeded
```

**Attendu : 200** pour les trois. Un type non traité répond **200 sans effet** — c'est volontaire : un 4xx ferait retenter Stripe indéfiniment sur un événement dont on ne veut rien faire.

### API-HOOK-03 — Les cinq événements Stripe traités et leur effet Gravité : majeure.

| Type Stripe                              | Effet dans Yamba                                                                                                                                                                                      | Comment le vérifier                                                               |
|------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| payment_intent.canceled                  | L'autorisation est morte (expiration ~7 j, annulation fournisseur) : un deal <b>PENDING</b> qui la porte est <b>annulé par le système. Idempotent.</b>                                                | GET /api/deals/{id} → status: "CANCELLED", fermé par SYSTEM.                      |
| payment_intent.amount_capturable_updated | Accusé simple : l'autorisation est posée. La création du deal reste pilotée par POST /deals.                                                                                                          | Aucun changement de statut attendu.                                               |
| account.updated (compte connecté)        | Les drapeaux du Voyageur (chargesEnabled, payoutsEnabled, detailsSubmitted) <b>suivent Stripe</b> sans repasser par l'onboarding ; un compte devenu prêt fait <b>repartir ses versements bloqués.</b> | GET /api/carrier/onboarding/stripe/status ; file d'exception de l'administration. |
| transfer.reversed                        | Le transfert au Voyageur a été renversé : le deal est marqué.                                                                                                                                         | File « Transferts renversés » de GET /api/admin/finances/queue.                   |
| payout.failed (compte connecté)          | La banque du Voyageur a refusé le virement : il est prévenu.                                                                                                                                          | Notification et email au Voyageur.                                                |

**Le principe qui gouverne tout ce tableau** : entre la base et Stripe, **c'est Stripe qui a l'argent**. L'état de Yamba converge vers le sien ; le webhook est la source de vérité de l'état du paiement.

### API-HOOK-04 — Rejouer un événement Stripe ne double aucun effet Gravité : bloquante.

```
stripe events resend evt_XXXXXXX
stripe events resend evt_XXXXXXX
curl -s -b admin.txt "$BASE/admin/deals/$D/history" | jq ' [.items[] | select(.kind=="WEBHOOK")] | length'
```

**Attendu : 200** aux deux envois, et **un seul** effet métier (une seule annulation, un seul marquage). L'annulation par `payment_intent.canceled` passe par la machine à états : rejouée sur un deal déjà annulé, elle est refusée sans dommage.

## 8.2 Email (Resend) — via la passerelle, signature Svix

La route est `POST /api/webhooks/email/resend`, proxifiée vers le notification-service. Ici, le corps brut est **conservé** par l'analyseur JSON (option de vérification), ce qui permet de valider la signature **Svix** tout en profitant du corps analysé.

Trois en-têtes : `svix-id`, `svix-timestamp`, `svix-signature`. Secret : `RESEND_WEBHOOK_SECRET`.

### API-HOOK-05 — Les trois réponses du webhook email Gravit  : majeure.

| Cas                                       | R ponse                                                                                            |
|-------------------------------------------|----------------------------------------------------------------------------------------------------|
| <code>RESEND_WEBHOOK_SECRET</code> absent | <b>503</b> { "message": "Webhook not configured (RESEND_WEBHOOK_SECRET)." }                        |
| Signature invalide ou absente             | <b>401</b> { "message": "Invalid signature.", "reason": "..." }                                    |
| Signature valide                          | <b>200</b> { ok, status, suppressed } — ou { ok: true, ignored: "<type>" } pour un type non trait  |

### API-HOOK-06 — Fabriquer un  v nement email sign  Gravit  : majeure · Pr requis :

`RESEND_WEBHOOK_SECRET` configur  ; sinon ►►.

La signature Svix est un HMAC-SHA256, encod  en base64, calcul  sur la cha ne `<svix-id>.<svix-timestamp>.<corps brut>`, avec la partie base64 du secret (`whsec_<base64>`) comme cl . Script de recette :

```
SECRET_B64="${RESEND_WEBHOOK_SECRET#whsec_}"
ID="msg_recette_$(date +%s)"
TS=$(date +%s)
cat > /tmp/evenement.json <<'JSON'
{"type":"email.delivered","data":{"email_id":"em_recette_0001","to":
["aminata.shipper@seed.yamba.dev"]}}
JSON
BODY=$(cat /tmp/evenement.json)
SIG="v1,$(printf '%s.%s.%s' "$ID" "$TS" "$BODY" \
| openssl dgst -sha256 -mac HMAC -macopt "hexkey:$(printf %s "$SECRET_B64" | base64 -d | xxd -
p -c 256)" -binary \
| base64)"

show -X POST "$BASE/webhooks/email/resend" -H 'Content-Type: application/json' \
-H "svix-id: $ID" -H "svix-timestamp: $TS" -H "svix-signature: $SIG" \
--data-binary @/tmp/evenement.json
```

**Attendu — HTTP 200** { "ok": true, "status": "DELIVERED", "suppressed": false }.

**Effet de bord :** la trace `EmailDelivery` correspondant à ce `providerMessageId` passe au statut « livré », avec sa date.

---

#### API-HOOK-07 — Un rebond dur met l'adresse sur la liste de suppression Gravit  : bloquante.

Rejouer le script pr c dent avec ce corps :

```
{ "type": "email.bounced",
  "data": { "email_id": "em_recette_0002", "to": ["aminata.shipper@seed.yamba.dev"],
    "bounce": { "type": "hard" } } }
```

**Attendu — HTTP 200** { "ok": true, "status": "BOUNCED", "suppressed": true }.

**Effet de bord — c'est le point de la fiche :**

```
curl -s -b admin.txt "$BASE/admin/users/${uid aminata}" | jq '{emailSuppressedAt,
emailSuppressedReason}'
```

**Attendu :** une date et la raison `HARD_BOUNCE` (ou `COMPLAINT` avec `email.complained`).

**La cons quence sur tout le reste de la plateforme : chaque r solveur de destinataires doit ignorer les comptes effac s ET les adresses supprim es.** Un flux d'email qui l'oublie est un d faut. Le v rifier : d clencher une transition qui envoie un email au membre supprim , puis constater qu'**aucun** message ne part pour cette adresse.

**Effet de bord   annuler :** lever la suppression avant de continuer — `DELETE /api/admin/users/{id}/email-suppression`.

---

#### API-HOOK-08 — Un  v nement email inconnu ou rejou  r pond 200 sans effet Gravit  : majeure.

Rejouer avec `"type": "email.opened"`, puis rejouer **deux fois** le rebond dur de la fiche pr c dente.

**Attendu :** { "ok": true, "ignored": "email.opened" } en **200** pour le premier ; **200** aux deux rejeux du rebond, avec `suppressed: true` puis `suppressed: false` — l'adresse n'est supprim e qu'une fois.

**Types trait s :** `email.delivered`, `email.bounced`, `email.complained`. Tous les autres (`email.sent`, `email.opened`, `email.clicked`, `email.delivery_delayed`) sont **ignor s volontairement**.

---

## 9. Consignation

### 9.1 Tableau de suivi

  remplir int gralement. Une campagne sans tableau rempli n'est pas une campagne.

#### En-t te de campagne

| Champ             | Valeur |
|-------------------|--------|
| Date              |        |
| Testeur           |        |
| Branche / version |        |

| Champ                   | Valeur                                                                                      |
|-------------------------|---------------------------------------------------------------------------------------------|
| Base de données         |                                                                                             |
| Fournisseur de paiement | STRIPE / FAKE                                                                               |
| Broker d'événements     | actif / absent                                                                              |
| Fournisseur d'email     | resend / smtp (Mailpit) / fake                                                              |
| Jeu d'essai rejoué à    |                                                                                             |
| smoke-services.sh       | 6/6  / ... |

## Fiches

| Fiche       | Intitulé                              | Gravité   | Attendu                       | Obtenu | Verdict | Anomalie |
|-------------|---------------------------------------|-----------|-------------------------------|--------|---------|----------|
| API-GW-01   | Sonde publique ok                     | majeure   | 200 ok , 5 services           |        |         |          |
| API-GW-02   | Sonde down service coupé              | majeure   | 503 down                      |        |         |          |
| API-GW-03   | Santé de la passerelle                | mineure   | 200 ok                        |        |         |          |
| API-GW-04   | /health des 5 services                | mineure   | 200 ×5                        |        |         |          |
| API-GW-05   | État public de maintenance            | mineure   | 200 enabled: false            |        |         |          |
| API-GW-06   | Écriture refusée en maintenance       | majeure   | 503 MAINTENANCE               |        |         |          |
| API-GW-07   | Lectures et exceptions en maintenance | majeure   | 200 ×4                        |        |         |          |
| API-GW-08   | Plafonds 100 / 1 000                  | majeure   | 100 · 1000 · 100              |        |         |          |
| API-GW-09   | Identifiant de corrélation            | majeure   | posé et renvoyé               |        |         |          |
| API-GW-10   | x-locale pilote le formatage          | mineure   | dates et textes différents    |        |         |          |
| API-GW-11   | Langue non supportée                  | mineure   | 400 LOCALE_UNSUPPORTED        |        |         |          |
| API-GW-12   | Locales exotiques tolérées            | mineure   | 200 ×5                        |        |         |          |
| API-GW-13   | details.code arrive au client         | bloquante | 409/403/400 avec code         |        |         |          |
| API-GW-14   | Erreurs par champ                     | mineure   | 400 + errors                  |        |         |          |
| API-GW-15   | Aucune erreur non gérée               | bloquante | 0 occurrence                  |        |         |          |
| API-GW-16   | 401 sans session                      | bloquante | 401 ×4                        |        |         |          |
| API-GW-17   | 403 non partie prenante               | bloquante | 403, corps vide de données    |        |         |          |
| API-GW-18   | 404 sans divulgation                  | bloquante | 404 / 400                     |        |         |          |
| API-GW-19   | 409 conflit typé                      | majeure   | 409<br>TRANSITION_NOT_ALLOWED |        |         |          |
| API-GW-20   | Pagination recherche                  | majeure   | pages disjointes              |        |         |          |
| API-GW-21   | Pagination du fil                     | mineure   | vers le passé                 |        |         |          |
| API-GW-22   | Listes bornées                        | mineure   | ≤ 50                          |        |         |          |
| API-AUTH-01 | Démarrer une inscription              | majeure   | 200, rien en base             |        |         |          |
| API-AUTH-02 | Consentement obligatoire              | majeure   | 400                           |        |         |          |
| API-AUTH-03 | Email déjà pris                       | majeure   | 409                           |        |         |          |

| Fiche       | Intitulé                          | Gravité          | Attendu                        | Obtenu | Verdict | Anomalie |
|-------------|-----------------------------------|------------------|--------------------------------|--------|---------|----------|
| API-AUTH-04 | Mot de passe faible               | mineure          | 400 type: "password"           |        |         |          |
| API-AUTH-05 | Code faux, compteur               | majeure          | 401 OTP_INCORRECT              |        |         |          |
| API-AUTH-06 | Bon code, sans session            | majeure          | 201, 0 cookie                  |        |         |          |
| API-AUTH-07 | Jetons en cookies seulement       | <b>bloquante</b> | 200, corps sans jeton          |        |         |          |
| API-AUTH-08 | 401 indistinguable                | <b>bloquante</b> | 401 identiques                 |        |         |          |
| API-AUTH-09 | /auth/me sans secret              | <b>bloquante</b> | 200, fuites: []                |        |         |          |
| API-AUTH-10 | Rotation révoque l'ancien         | majeure          | 200 puis 401                   |        |         |          |
| API-AUTH-11 | Mes appareils                     | majeure          | liste + révocation             |        |         |          |
| API-AUTH-12 | Mot de passe révoque les sessions | majeure          | 200 puis 401                   |        |         |          |
| API-AUTH-13 | Déconnexion                       | majeure          | 200 puis 401                   |        |         |          |
| API-AUTH-14 | Mot de passe oublié muet          | <b>bloquante</b> | 200 identiques                 |        |         |          |
| API-AUTH-15 | Profil éditable, slug stable      | mineure          | 200, slug inchangé             |        |         |          |
| API-AUTH-16 | Date de naissance invalide        | mineure          | 400 + code de règle            |        |         |          |
| API-AUTH-17 | Avatar signé, URL gardée          | majeure          | 200 / 400                      |        |         |          |
| API-AUTH-18 | Profil public / masqué            | <b>bloquante</b> | 404 tiers, hidden propriétaire |        |         |          |
| API-AUTH-19 | Abonnement                        | mineure          | 200/201, pas de doublon        |        |         |          |
| API-AUTH-20 | Préférences et consentement       | mineure          | 200 + ConsentLog               |        |         |          |
| API-AUTH-21 | Geste sensible sans fenêtre       | <b>bloquante</b> | 403 SUDO_REQUIRED              |        |         |          |
| API-AUTH-22 | Ouvrir puis rejouer               | <b>bloquante</b> | 200                            |        |         |          |
| API-AUTH-23 | Fenêtre liée à une session        | <b>bloquante</b> | 403 sur l'autre                |        |         |          |
| API-AUTH-24 | Export sans données d'autrui      | <b>bloquante</b> | aucune fuite                   |        |         |          |
| API-AUTH-25 | Export trop fréquent              | mineure          | 400 EXPORT_RATE_LIMITED        |        |         |          |
| API-AUTH-26 | Bloqueurs d'effacement            | majeure          | 200 + liste fermée             |        |         |          |

| Fiche       | Intitulé                               | Gravité          | Attendu                    | Obtenu | Verdict | Anomalie |
|-------------|----------------------------------------|------------------|----------------------------|--------|---------|----------|
| API-AUTH-27 | Effacement bloqué                      | majeure          | 409 ERASURE_BLOCKED        |        |         |          |
| API-AUTH-28 | Effacement effectif                    | <b>bloquante</b> | 200 puis 401               |        |         |          |
| API-AUTH-29 | Alertes de route, cycle                | mineure          | 201/200                    |        |         |          |
| API-AUTH-30 | Gardes des alertes                     | majeure          | 400 · 409 à 20             |        |         |          |
| API-AUTH-31 | Signaler un trajet                     | majeure          | 201                        |        |         |          |
| API-AUTH-32 | Gardes du signallement                 | majeure          | 400/400/404/409            |        |         |          |
| API-AUTH-33 | Onboarding Voyageur                    | majeure          | 200 ×4                     |        |         |          |
| API-AUTH-34 | Tableau de bord sans compte            | mineure          | 409 STRIPE_ACCOUNT_MISSING |        |         |          |
| API-TRIP-01 | Recherche publique                     | majeure          | 200 ×2                     |        |         |          |
| API-TRIP-02 | Filtres durs                           | <b>bloquante</b> | aucun trajet interdit      |        |         |          |
| API-TRIP-03 | Filtres invalides ignorés              | mineure          | 200                        |        |         |          |
| API-TRIP-04 | Paramètres de prix publics             | majeure          | 200                        |        |         |          |
| API-TRIP-05 | Facettes                               | mineure          | 200, 9 compteurs           |        |         |          |
| API-TRIP-06 | Trajet public                          | majeure          | 200 / 404 / 400            |        |         |          |
| API-TRIP-07 | Favori idempotent et gardé             | majeure          | 200 · 403 · 404 · 409      |        |         |          |
| API-TRIP-08 | Brouillon puis publication             | majeure          | 201 puis 200               |        |         |          |
| API-TRIP-09 | Portes de publication                  | majeure          | 400                        |        |         |          |
| API-TRIP-10 | allowedActions fait foi                | majeure          | 400 hors liste             |        |         |          |
| API-TRIP-11 | Cycle de vie complet                   | majeure          | 200 / 400 cohérents        |        |         |          |
| API-TRIP-12 | <b>Annulation refusée, deal vivant</b> | <b>bloquante</b> | 409 TRIP_HAS_ACTIVE_DEALS  |        |         |          |
| API-TRIP-13 | Modification, capacité immuable        | majeure          | 200 / 400                  |        |         |          |
| API-TRIP-14 | Trajet d'autrui                        | <b>bloquante</b> | refus ×3, prix inchangé    |        |         |          |
| API-TRIP-15 | Suppression et annulation              | mineure          | 200 puis 404               |        |         |          |

| Fiche       | Intitulé                         | Gravité          | Attendu                  | Obtenu | Verdict | Anomalie |
|-------------|----------------------------------|------------------|--------------------------|--------|---------|----------|
| API-TRIP-16 | Documents et déduplication       | majeure          | 201 puis 200             |        |         |          |
| API-TRIP-17 | Signature de téléversement       | majeure          | 401 / 200                |        |         |          |
| API-TRIP-18 | Suppression idempotente          | mineure          | 200 x2                   |        |         |          |
| API-DEAL-01 | Intention de paiement            | majeure          | 201, rien persisté       |        |         |          |
| API-DEAL-02 | Les neuf refus typés             | <b>bloquante</b> | 409 + code               |        |         |          |
| API-DEAL-03 | Divergence de devis              | <b>bloquante</b> | 409 QUOTE_DIVERGENCE     |        |         |          |
| API-DEAL-04 | Créer un deal                    | majeure          | 201, kilos réservés      |        |         |          |
| API-DEAL-05 | Instantanés immuables            | <b>bloquante</b> | devis inchangé           |        |         |          |
| API-DEAL-06 | Deux vues, listes blanches       | <b>bloquante</b> | fuites: []               |        |         |          |
| API-DEAL-07 | Acceptation, capture             | majeure          | 200 ACCEPTED             |        |         |          |
| API-DEAL-08 | Refus d'acceptation              | majeure          | 409 / 403                |        |         |          |
| API-DEAL-09 | Refus du Voyageur                | majeure          | 200, remboursement total |        |         |          |
| API-DEAL-10 | Aperçu = montant appliqué        | <b>bloquante</b> | égalité                  |        |         |          |
| API-DEAL-11 | Inspection complète exigée       | <b>bloquante</b> | 400 / 400 / 200          |        |         |          |
| API-DEAL-12 | <b>Code absent de la réponse</b> | <b>bloquante</b> | corps sans code          |        |         |          |
| API-DEAL-13 | Refus à la récupération          | majeure          | 200, remboursement       |        |         |          |
| API-DEAL-14 | Séquence des jalons              | majeure          | 409 / 409 / 200 / 409    |        |         |          |
| API-DEAL-15 | Remise, verrou du code           | <b>bloquante</b> | 3 essais puis verrou     |        |         |          |
| API-DEAL-16 | Régénération, limite 5           | majeure          | 5x200 puis 409           |        |         |          |
| API-DEAL-17 | Confirmation, versement          | majeure          | 200 SENT                 |        |         |          |
| API-DEAL-18 | Litige, versement gelé           | majeure          | 200 FROZEN               |        |         |          |
| API-DEAL-19 | Déclaration du Voyageur          | majeure          | 201 puis 409             |        |         |          |
| API-DEAL-20 | Notation double aveugle          | majeure          | 201, autre note cachée   |        |         |          |

| Fiche        | Intitulé                       | Gravité          | Attendu                  | Obtenu | Verdict | Anomalie |
|--------------|--------------------------------|------------------|--------------------------|--------|---------|----------|
| API-DEAL-21  | Lien de suivi unique           | majeure          | même jeton               |        |         |          |
| API-DEAL-22  | Page destinataire minimale     | <b>bloquante</b> | fuites: []               |        |         |          |
| API-DEAL-23  | Listes bornées au propriétaire | <b>bloquante</b> | aucun croisement         |        |         |          |
| API-MSG-01   | Fil créé au premier accès      | majeure          | 200, même id             |        |         |          |
| API-MSG-02   | Pas de fil avant acceptation   | <b>bloquante</b> | 403 ×3                   |        |         |          |
| API-MSG-03   | Code refusé, contact marqué    | <b>bloquante</b> | 400 / 201 flagged        |        |         |          |
| API-MSG-04   | Fenêtre d'écriture             | majeure          | clés stables + 400       |        |         |          |
| API-MSG-05   | Marquer lu                     | mineure          | 200 idempotent           |        |         |          |
| API-MSG-06   | Rendez-vous et créneaux        | majeure          | 201 / 400 ×5             |        |         |          |
| API-MSG-07   | Contre-proposition             | majeure          | remplace                 |        |         |          |
| API-MSG-08   | Accepter celle de l'autre      | majeure          | 400 / 200 / 400          |        |         |          |
| API-MSG-09   | Numéro pas avant l'heure       | <b>bloquante</b> | 400 T00_EARLY            |        |         |          |
| API-MSG-10   | Signaler un message            | majeure          | 400/400/201/409          |        |         |          |
| API-MSG-11   | Réponses rapides               | mineure          | 200, clés stables        |        |         |          |
| API-MSG-12   | Lecture admin journalisée      | majeure          | 200 + ligne de journal   |        |         |          |
| API-NOTIF-01 | Lire ses notifications         | mineure          | 200, ≤ 50                |        |         |          |
| API-NOTIF-02 | Marquer lu, idempotent         | mineure          | 200 ×4                   |        |         |          |
| API-NOTIF-03 | Quatre statuts distincts       | <b>bloquante</b> | 403/404/400/401          |        |         |          |
| API-NOTIF-04 | Reflet d'une transition        | majeure          | compteur augmente        |        |         |          |
| API-SEC-01   | 401 sur 15 routes              | <b>bloquante</b> | 401 ×15                  |        |         |          |
| API-SEC-02   | Jetons invalides               | <b>bloquante</b> | 401 ×4, 200 optionnelles |        |         |          |
| API-SEC-03   | Traversée horizontale          | <b>bloquante</b> | 403 partout              |        |         |          |
| API-SEC-04   | Membre ≠ administration        | <b>bloquante</b> | 401 ×14                  |        |         |          |
| API-SEC-05   | Permission manquante           | <b>bloquante</b> | 403 + permission         |        |         |          |
| API-SEC-06   | Restreint / suspendu           | <b>bloquante</b> | 403 · 401                |        |         |          |
| API-SEC-07   | Cinq gestes sensibles          | <b>bloquante</b> | 403 ×5                   |        |         |          |

| Fiche       | Intitulé                  | Gravité          | Attendu               | Obtenu | Verdict | Anomalie |
|-------------|---------------------------|------------------|-----------------------|--------|---------|----------|
| API-SEC-08  | Balayage du code          | <b>bloquante</b> | 0 occurrence          |        |         |          |
| API-SEC-09  | Destinataire non exposé   | <b>bloquante</b> | fuites: []            |        |         |          |
| API-SEC-10  | Aucun secret technique    | <b>bloquante</b> | aucune ligne          |        |         |          |
| API-SEC-11  | Aucune trace de pile      | <b>bloquante</b> | aucune ligne          |        |         |          |
| API-SEC-12  | Signature email invalide  | <b>bloquante</b> | 401, aucun effet      |        |         |          |
| API-SEC-13  | Signature Stripe invalide | <b>bloquante</b> | 400 / 501             |        |         |          |
| API-SEC-14  | Aucune énumération        | <b>bloquante</b> | 404 identiques        |        |         |          |
| API-SEC-15  | CORS restreint            | majeure          | 200 / refus           |        |         |          |
| API-IDEM-01 | Intention consommée       | <b>bloquante</b> | 409, 1 seul deal      |        |         |          |
| API-IDEM-02 | Rejeu d'acceptation       | <b>bloquante</b> | 409, 1 capture        |        |         |          |
| API-IDEM-03 | Rejeu de remise           | <b>bloquante</b> | 409, dates inchangées |        |         |          |
| API-IDEM-04 | Gestes idempotents        | majeure          | mêmes codes           |        |         |          |
| API-IDEM-05 | Rejeu d'événement         | majeure          | 1 seul effet          |        |         |          |
| API-IDEM-06 | Deux acceptations         | <b>bloquante</b> | 1×200, 2×409          |        |         |          |
| API-IDEM-07 | Deux réservations         | <b>bloquante</b> | 201 + 409, kg ≥ 0     |        |         |          |
| API-IDEM-08 | Deux acceptations RDV     | majeure          | 200 + 400             |        |         |          |
| API-IDEM-09 | Deux régénérations        | majeure          | -1 seulement          |        |         |          |
| API-IDEM-10 | Deux rafraîchissements    | majeure          | ≤ 1 session vivante   |        |         |          |
| API-HOOK-01 | Écoute Stripe locale      | majeure          | secret affiché        |        |         |          |
| API-HOOK-02 | Quatre réponses Stripe    | <b>bloquante</b> | 200/400/501/500       |        |         |          |
| API-HOOK-03 | Cinq événements traités   | majeure          | effets attendus       |        |         |          |
| API-HOOK-04 | Rejeu Stripe              | <b>bloquante</b> | 1 seul effet          |        |         |          |
| API-HOOK-05 | Trois réponses email      | majeure          | 503/401/200           |        |         |          |
| API-HOOK-06 | Événement email signé     | majeure          | 200 DELIVERED         |        |         |          |
| API-HOOK-07 | Rebond dur → suppression  | <b>bloquante</b> | 200 suppressed:true   |        |         |          |
| API-HOOK-08 | Événement inconnu / rejeu | majeure          | 200 sans effet        |        |         |          |

## 9.2 Fiche d'anomalie

Une anomalie par fiche KO, numérotée `ANO-API-<nn>`.

```
ANO-API-<nn>
Fiche           : API-XXX-nn
Gravité         : bloquante / majeure / mineure
Environnement  : branche, base, fournisseur de paiement, broker, fournisseur d'email
Appel exact     : la commande curl complète, copiable et rejouable
Corps envoyé    : le JSON envoyé (secrets caviardés)
Attendu        : code HTTP + champs du corps + effet de bord
Obtenu         : code HTTP + corps intégral + effet de bord constaté
x-correlation-id : la valeur lue dans les en-têtes de la réponse
Reproductible   : oui (n/n essais) / intermittent (n/m) / une fois
Impact         : ce qu'un client mal intentionné ou maladroit peut faire à cause du défaut
Piste          : fichier et fonction suspectés, si le testeur en a une
```

### Trois règles de rédaction :

1. **Toujours joindre l'identifiant de corrélation.** C'est la clé partagée entre la réponse, les journaux serveur, les événements et Sentry. Une anomalie sans lui coûte une demi-journée.
2. **Toujours joindre la commande exacte,** avec ses identifiants réels. Un développeur doit pouvoir la coller telle quelle.
3. **Ne jamais joindre de secret :** caviarder les jetons, les cookies, les clés de fournisseur. Ne jamais joindre un export de données personnelles complet — décrire le champ fautif suffit.

## 9.3 Critères de sortie

La campagne est **acceptée** quand les quatre conditions suivantes sont réunies.

1. **Zéro anomalie bloquante ouverte.** Sans exception et sans dérogation. Une bloquante est, par définition, une garde de sécurité ou d'argent qui ne tient pas côté serveur : la publier revient à publier la faille.
2. **Toutes les fiches marquées bloquantes sont jouées et OK.** Une bloquante en ►► n'est pas un succès : c'est une couverture manquante, et elle doit être justifiée par écrit (configuration absente) **et** rejouée sur un environnement qui la permet avant la mise en production. Les fiches concernées, dans l'ordre de priorité :
  - **Le contrat d'erreur** : API-GW-13 (`details.code` arrive au client), API-GW-15, API-SEC-11.
  - **La sémantique** : API-GW-16, API-GW-17, API-GW-18, API-SEC-14.
  - **Les frontières** : API-SEC-01, API-SEC-03, API-SEC-04, API-SEC-05, API-SEC-06, API-DEAL-23.
  - **Les secrets** : API-SEC-08 (code de livraison), API-SEC-09 (destinataire), API-SEC-10, API-DEAL-06, API-DEAL-12, API-DEAL-22.
  - **L'argent** : API-DEAL-02, API-DEAL-03, API-DEAL-05, API-DEAL-10, API-DEAL-15, API-TRIP-12, API-IDEM-01, API-IDEM-02, API-IDEM-03, API-IDEM-06, API-IDEM-07.
  - **La fenêtre sensible** : API-AUTH-21, API-AUTH-22, API-AUTH-23, API-SEC-07.
  - **Les signatures** : API-SEC-12, API-SEC-13, API-HOOK-02, API-HOOK-04, API-HOOK-07.
3. **Les anomalies majeures sont arbitrées, une par une,** par le responsable produit et le responsable technique : corrigée avant la livraison, ou acceptée avec un contournement documenté et une échéance. Une majeure « oubliée » n'existe pas : elle est soit corrigée, soit inscrite au registre.
4. **Les mineures sont inscrites au journal de dette** avec leur fiche d'origine. Les écarts de sémantique connus du trip-service (400 au lieu de 403 ou 404) y figurent déjà : les redécouvrir à chaque campagne est normal, les signaler comme neufs ne l'est pas.

**Ce que la campagne ne prouve pas, et qu'il faut dire explicitement dans le rapport :**

- Rien sur la **charge** ni sur la **tenue en durée** : le limiteur a été vérifié comme garde, pas comme protection contre un déni de service distribué.
- Rien sur les **tâches planifiées** (expiration à 24 h, versement à 4 jours, révélation des avis à 14 jours, purges de rétention) : elles relèvent du cahier n° 4.
- Rien sur la **cohérence des événements** au-delà de leur effet observable par l'API : la boîte d'envoi, le relais, le dédoublonnage et les événements parqués relèvent du cahier n° 4.
- Rien sur les **écrans**. Une API conforme derrière un écran fautif reste un défaut — pour le cahier n° 1 ou n° 2.

---

*Fin du cahier de recette n° 3. Toute divergence entre ce document et le comportement observé doit être traitée comme un défaut du cahier ou du code, jamais comme une interprétation du testeur.*